

# Apply-volgorde voor een Bottom-up SDD Compilatie

Gijs Vliegen

Thesis voorgedragen tot het behalen  
van de graad van Master of Science  
in de ingenieurswetenschappen:  
computerwetenschappen, hoofdoptie  
Artificiële intelligentie

**Promotor:**

Prof. dr. L. De Raedt

**Evaluator:**

Prof. dr. M. Denecker

**Begeleider:**

Dr. ir. J. Bekker

Dr. ir. V. Derkinderen

© Copyright KU Leuven

Zonder voorafgaande schriftelijke toestemming van zowel de promotor als de auteur is overnemen, kopiëren, gebruiken of realiseren van deze uitgave of gedeelten ervan verboden. Voor aanvragen tot of informatie i.v.m. het overnemen en/of gebruik en/of realisatie van gedeelten uit deze publicatie, wend u tot het Departement Computerwetenschappen, Celestijnenlaan 200A bus 2402, B-3001 Leuven, +32-16-327700 of via e-mail [info@cs.kuleuven.be](mailto:info@cs.kuleuven.be).

Voorafgaande schriftelijke toestemming van de promotor is eveneens vereist voor het aanwenden van de in deze masterproef beschreven (originele) methoden, producten, schakelingen en programma's voor industrieel of commercieel nut en voor de inzending van deze publicatie ter deelname aan wetenschappelijke prijzen of wedstrijden.

# Inhoudsopgave

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| Voorwoord                                                           | iii       |
| Gebruik externe tools                                               | iv        |
| Lijst van figuren en tabellen                                       | v         |
| Lijst van afkortingen en symbolen                                   | vii       |
| Samenvatting                                                        | ix        |
| <b>1 Inleiding</b>                                                  | <b>1</b>  |
| <b>2 SDD als nieuwe voorstelling van een kennisbasis</b>            | <b>3</b>  |
| 2.1 Propositionele logica . . . . .                                 | 3         |
| 2.2 SDD als deelverzameling van d-DNNF en superverzameling van OBDD | 4         |
| 2.3 Vtree . . . . .                                                 | 6         |
| 2.4 Apply-operatie . . . . .                                        | 7         |
| <b>3 Probleemstelling</b>                                           | <b>10</b> |
| 3.1 Bottom-up compilatie . . . . .                                  | 10        |
| 3.2 Formele probleemstelling . . . . .                              | 11        |
| 3.3 Voorbeeld . . . . .                                             | 13        |
| 3.4 In de praktijk . . . . .                                        | 13        |
| <b>4 Gerelateerd onderzoek</b>                                      | <b>15</b> |
| 4.1 Vtree heuristieken . . . . .                                    | 16        |
| 4.2 Compilers . . . . .                                             | 17        |
| 4.3 Fasetransities . . . . .                                        | 18        |
| 4.4 Conjunctieve normaalvorm . . . . .                              | 19        |
| <b>5 Heuristieken</b>                                               | <b>21</b> |
| 5.1 Bovengrens na Apply . . . . .                                   | 21        |
| 5.2 Heuristieken op basis van vtree . . . . .                       | 27        |
| 5.3 Probabiliteit variabelenvoorkomens . . . . .                    | 31        |
| <b>6 Experimenten</b>                                               | <b>33</b> |
| 6.1 Werkwijze . . . . .                                             | 35        |
| 6.2 SDD- en PySDD-bibliotheek . . . . .                             | 37        |
| 6.3 Resultaten gebalanceerde vtree . . . . .                        | 37        |
| 6.4 Resultaten rechtslineaire vtrees . . . . .                      | 41        |
| 6.5 Resultaten linkslineaire vtrees . . . . .                       | 41        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 6.6      | Combinatie van heuristieken . . . . .           | 44        |
| 6.7      | Resultaten willekeurig aantal clauses . . . . . | 48        |
| <b>7</b> | <b>Besluit</b>                                  | <b>51</b> |
| 7.1      | Toekomstvisie . . . . .                         | 52        |
|          | <b>Bibliografie</b>                             | <b>53</b> |
| <b>A</b> | <b>Engelse termen</b>                           | <b>56</b> |
| <b>B</b> | <b>Resultaten sectie 6.7</b>                    | <b>58</b> |

# Voorwoord

Bij het voltooien van deze thesis wil ik mijn oprechte dank uitspreken aan iedereen die mij gedurende dit proces heeft ondersteund en geïnspireerd.

In het bijzonder wil ik mijn promotor, prof. dr. Luc De Raedt, bedanken voor zijn onmisbare begeleiding, waardevolle inzichten en voortdurende steun.

Daarnaast ben ik bijzonder dankbaar voor de assistenten, dr. ir. Jessa Bekker en dr. ir. Vincent Derkinderen, voor hun constructieve terugkoppeling, aanmoediging en technische ondersteuning. Hun deskundige adviezen en bereidheid om te helpen hebben een grote bijdrage geleverd aan de totstandkoming van deze thesis.

Tenslotte wil ik mijn familie en vrienden bedanken voor hun onvoorwaardelijke steun en begrip tijdens deze intensieve periode. Hun aanmoediging heeft mij gemotiveerd om door te zetten en dit werk tot een goed einde te brengen.

Aan iedereen die op welke manier dan ook heeft bijgedragen aan dit werk, mijn oprechte dank.

*Gijs Vliegen*

# Gebruik externe tools

## Generatieve AI

Voor het opstellen van dit werk werd gebruik gemaakt van ChatGPT, zowel inhoudelijk als praktisch. We geven een exhaustieve lijst van zaken waarvoor deze tool gebruikt werd:

1. Vertalen van technisch jargon, een lijst van vertaalde woorden is terug te vinden in appendix A.
2. Beter verwoorden van zinnen.
3. Informatie verkrijgen over elementaire logische wiskunde.
4. Assistentie bij het coderen in Python: voor plots, numpy en parallelisme.
5. Assistentie bij het schrijven in  $\text{\LaTeX}$ : voor (aangepaste) commando's en opmaak.

Altijd werd deze hulp zeer kritisch verwerkt, enkel eenvoudige zaken waarvan de correctheid eenvoudig gecontroleerd kon worden, werden gebruikt.

## Vlaams Supercomputer Centrum

De infrastructuur en dienstverlening gebruikt in dit werk werd voorzien door het VSC (Vlaams Supercomputer Centrum), gefinancierd door het FWO en de Vlaamse overheid.

# Lijst van figuren en tabellen

## Lijst van figuren

|     |                                                                                                                 |    |
|-----|-----------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Rechtslineaire en gebalanceerde vtree gebruikt voor figuur 2.2. . . . .                                         | 6  |
| 2.2 | OBDD, rechtslineaire SDD en gebalanceerde SDD voor de formule<br>$f = (A \wedge B) \vee (C \wedge D)$ . . . . . | 7  |
| 3.1 | Een mogelijke bottom-up compilatie van een rooted-DAG . . . . .                                                 | 12 |
| 3.2 | Variatie in compilatietijd $ct$ voor een probleeminstantie met<br>gebalanceerde vtree. . . . .                  | 14 |
| 5.1 | Deel van SDD $\Psi$ , met vtreenknoten $v^p$ , $v$ , $v^l$ en $v^r$ . . . . .                                   | 24 |
| 5.2 | Voorbeeld van hoe de normalisatie het aantal elementen beïnvloedt . . .                                         | 27 |
| 6.1 | Stappenplan voor experimenten. . . . .                                                                          | 35 |
| 6.2 | Variatie van willekeurige compilatiesequenties samen met de<br>compilatietijden van heuristieken . . . . .      | 49 |

## Lijst van tabellen

|     |                                                                                                                 |    |
|-----|-----------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Bewerkingen en query's voor een kennisbasis . . . . .                                                           | 1  |
| 2.1 | Waarheidstabellen met operaties negatie, conjunctie en disjunctie. . . .                                        | 4  |
| 2.2 | De wetten van De Morgan en operaties met elementaire waarden. . . .                                             | 4  |
| 5.1 | Kans op voorkomen van elke variabele in een 3-CNF $f$ met $r \times \nu$<br>clausules voor $\nu = 24$ . . . . . | 32 |
| 6.1 | Heuristieken gebruikt in de experimenten . . . . .                                                              | 34 |
| 6.2 | Resultaten met gebalanceerde vtree en $r = 0.5, 1, 1.5, 2$ . . . . .                                            | 38 |
| 6.3 | Resultaten met gebalanceerde vtree en $r = 2.5, 3, 3.5, 4$ . . . . .                                            | 39 |
| 6.4 | Resultaten met gebalanceerde vtree en $r = 4.5$ . . . . .                                                       | 40 |
| 6.5 | Resultaten met rechtslineaire vtree en $r = 0.5, 1, 1.5, 2$ . . . . .                                           | 42 |
| 6.6 | Resultaten met rechtslineaire vtree en $r = 2.5, 3, 3.5, 4$ . . . . .                                           | 43 |
| 6.7 | Resultaten met rechtslineaire vtree en $r = 4.5$ . . . . .                                                      | 44 |

|      |                                                                      |    |
|------|----------------------------------------------------------------------|----|
| 6.8  | Resultaten met linkslineaire vtree en $r = 0.5, 1, 1.5, 2$ . . . . . | 45 |
| 6.9  | Resultaten met linkslineaire vtree en $r = 2.5, 3, 3.5, 4$ . . . . . | 46 |
| 6.10 | Resultaten met linkslineaire vtree en $r = 4.5$ . . . . .            | 47 |
| 6.11 | Gemiddelde scores voor experimenten met willekeurig aantal clauses . | 49 |
| A.1  | Vertalingen van Engels jargon. . . . .                               | 57 |
| B.1  | Resultaten voor gebalanceerde vtrees en $\nu = 28$ . . . . .         | 58 |
| B.2  | Resultaten voor rechtslineaire vtrees en $\nu = 28$ . . . . .        | 60 |
| B.3  | Resultaten voor linkslineaire vtrees en $\nu = 10$ . . . . .         | 63 |
| B.4  | Resultaten voor willekeurige vtrees en $\nu = 16$ . . . . .          | 65 |

# Lijst van afkortingen en symbolen

## Afkortingen

|             |                                                 |
|-------------|-------------------------------------------------|
| d-DNNF      | Deterministic Decomposable Negation Normal Form |
| OBDD        | Ordered Binary Decision Diagram                 |
| SDD         | Sentential Decision Diagram                     |
| vtree       | Variabelenboom                                  |
| rooted-DAG  | Gewortelde Gerichte Acyclische Graaf            |
| CNF         | Conjunctieve Normaalvorm                        |
| DNF         | Disjunctieve Normaalvorm                        |
| SAT         | Booleaans Geldigheidsprobleem                   |
| $t(\alpha)$ | Compilatietijd van een SDD $\alpha$             |
| ct          | Totale compilatietijd                           |
| RANDOM      | Heuristiek: 3                                   |
| VP          | Heuristiek: 1                                   |
| KE          | Heuristiek: 4                                   |
| EL          | Heuristiek: 5                                   |
| TD          | Heuristiek: 6                                   |
| BU-LR       | Heuristiek: 6.1                                 |
| BU-RL       | Heuristiek: 6.2                                 |
| BU + EL     | Heuristiek: 7                                   |

## Symbolen

|                               |                                                                   |
|-------------------------------|-------------------------------------------------------------------|
| $\gamma_i, \delta_i$          | SDD in de context van bottom-up compilatie                        |
| $\alpha, \beta, \Omega, \Psi$ | SDD                                                               |
| $v$                           | Vtreetknoop                                                       |
| $v^l$                         | Linkerkind van vtreetknoop $v$                                    |
| $v^r$                         | Rechterkind van vtreetknoop $v$                                   |
| $v^p$                         | Ouder van vtreetknoop $v$                                         |
| $p, r$                        | Primes                                                            |
| $s, t$                        | Subs                                                              |
| $\nu$                         | Aantal variabelen in een probleeminstantie                        |
| $\neg$                        | Negatie                                                           |
| $\wedge$                      | Conjunctie                                                        |
| $\vee$                        | Disjunctie                                                        |
| $\triangle$                   | Symmetrisch verschil                                              |
| $\top$                        | Waar                                                              |
| $\perp$                       | Onwaar                                                            |
| $\phi$                        | Ratio van het aantal variabelen $\nu$ , gebruikt in heuristiek 7. |

# Samenvatting

Tijdens de bottom-up compilatie van logische circuits naar behandelbare voorstellingen zoals SDD's, OBDD's en d-DNNF's worden veel tussenresultaten berekend. Door middel van een goede Apply-volgorde kunnen we de compilatietijd van de SDD versnellen. In dit werk worden verschillende heuristieken voorgesteld om een Apply-volgorde te bepalen voor de bottom-up compilatie. We identificeren twee types heuristieken, heuristieken gebaseerd op een variabelenvolgorde afhankelijk van de vtree en heuristieken gebaseerd op een bovengrens na Apply. Empirisch wordt vastgesteld dat afhankelijk van de probleeminstantie een van de twee types best gebruikt kan worden. Er wordt tenslotte een voorstel gedaan om de twee types te combineren.

# Hoofdstuk 1

## Inleiding

In allerlei vakgebieden van computerwetenschappen worden logische circuits gebruikt, bijvoorbeeld binnen Probabilistische Inferentie voor het tellen van mogelijkheden [Der23]. Een logisch circuit bestaat uit verschillende variabelen. Elke variabele kan waarheidswaarde *Waar* of *Onwaar* hebben, de waarheidswaarde van het volledige circuit is een functie van de waarden van zijn variabelen. Een logisch circuit wordt ook wel een kennisbasis genoemd.

Er kunnen verschillende soorten bewerkingen of query's worden uitgevoerd op een kennisbasis [DM11], zie tabel 1.1 voor enkele voorbeelden van eenvoudige bewerkingen en query's. Door een combinatie van eenvoudige bewerkingen kunnen geavanceerde toepassingen zoals de eerder genoemde probabilistische inferentie, foutendetectie en diagnose van systemen en stochastische beperking-optimalisatie-problemen gesimuleerd worden [Bry86, LBD<sup>+</sup>17]. Kennisbasissen kunnen op verschillende manieren worden voorgesteld, een bepaald type voorstellingen wordt ook wel een compilatietaal genoemd. Een bepaalde voorstelling kan efficiënt zijn voor specifieke query's of bewerkingen (met polynomiale tijdscomplexiteit). Deze efficiëntie kan echter gepaard gaan met nadelen, zoals minder efficiënte uitvoering van andere query's of bewerkingen, een hoger geheugengebruik, enzovoort. Het vakgebied dat deze voorstellingen bestudeert, wordt kenniscompilatie genoemd. Elke toepassing vereist andere query's of bewerkingen met tot gevolg dat er heel wat voorstellingen zijn bedacht en bestudeerd,

| Bewerkingen             | Query's                      |
|-------------------------|------------------------------|
| Conditionering          | Consistentiecontrole         |
| Vergeten                | Geldigheidscontrole          |
| Vergeten van singletons | Clausale-implicatie-controle |
| Conjunctie              | Implicatiecontrole           |
| Begrensde conjunctie    | Equivalentiecontrole         |
| Disjunctie              | Zin-implicatie-controle      |
| Begrensde disjunctie    | Modeltelling                 |
| Negatie                 | Modelenumeratie              |

TABEL 1.1: Bewerkingen en query's voor een kennisbasis

---

zie [DM11] voor een overzicht.

Er bestaan twee algemene termen om compilatietalen te labelen, bondigheid als maat voor de grootte van de representatie en behandelbaarheid voor de verzameling van bewerkingen/query's die in polytijd kunnen worden uitgevoerd. In het algemeen geldt: hoe meer behandelbaar een voorstelling is, hoe minder bondig [DM11]. Twee type voorstellingen die een hoge behandelbaarheid hebben en dus voor veel toepassingen gebruikt kunnen worden, maar toch ook relatief bondig zijn, zijn de Deterministic Decomposable Negation Normal Form (d-DNNF) [Dar00] en de Ordered Binary Decision Diagram (OBDD) [Bry86]. De OBDD is een deelverzameling van de d-DNNF. De OBDD heeft dus alle eigenschappen van de d-DNNF en nog enkele andere nuttige eigenschappen, maar is als gevolg minder bondig. Recent is er een nieuwe veelbelovende representatie voorgesteld, de Sentential Decision Diagram (SDD) [Dar11]. De SDD is een deelverzameling van de d-DNNF, maar een superversameling van de OBDD, met interessante eigenschappen van beide voorstellingen. Er is al aangetoond dat de SDD bondiger is dan de OBDD [Bov16a] en de meer algemene BDD [Bov16b].

Om een kennisbasis om te zetten naar een bepaalde voorstelling, moet die gecompileerd worden. Voor SDD's, OBDD's en d-DNNF's kan dit zowel bottom-up als top-down. Tijdens de bottom-up compilatie van een kennisbasis naar een bepaalde representatie zoals de SDD worden er veel tussenresultaten bekomen. De tussenresultaten hangen af van de volgorde waarin een kennisbasis bottom-up wordt gecompileerd. Voor praktische toepassingen is een lage compilatietijd belangrijk. Dat kunnen we bereiken door een compilatie waarbij de tussenresultaten en daarmee ook het eindresultaat zo snel mogelijk berekend kunnen worden. Voorlopig is er nog geen duidelijke manier om een goede volgorde te zoeken. Zowel in [NW07] als in [HD04] wordt dit probleem aangekaart. In dit werk gaan we opzoek naar heuristieken om een volgorde te bepalen, met als doel de totale compilatietijd te minimaliseren. We zullen de volgorde onder andere bepalen aan de hand van de variabelen aanwezig in de SDD's, de grootte en structuur van de SDD's en de vtree die ze respecteren.

## Hoofdstuk 2

# SDD als nieuwe voorstelling van een kennisbasis

Deze sectie geeft belangrijke theorie om de SDD te begrijpen. Deze theorie komt voornamelijk uit het originele werk waarin de SDD werd geïntroduceerd [Dar11].

Eerst en vooral een paar opmerkingen over notatie. In context van een bottom-up compilatie zal  $\gamma_i$  of  $\delta_i$  gebruikt worden om SDD's te noteren, in andere gevallen worden de letters  $\alpha, \beta, \Omega$  en  $\Psi$  gebruikt. Hoofdletters zoals  $X$  zijn voor variabelen; vetgedrukte hoofdletters  $\mathbf{X}$  zijn voor verzamelingen van variabelen; kleine letters zoals  $x$  zijn voor een instantie van een variabele;  $\top$  wordt gebruikt voor de waarheidswaarde *Waar* en  $\perp$  voor *Onwaar*.

### 2.1 Propositionele logica

Sentential Decision Diagrams zijn een bepaalde voorstelling van een logische formule. We leggen kort uit wat een logische formule is. De basis van een logische formule is een propositie, een propositie kan waar zijn, aangeduid met  $\top$ , of onwaar, aangeduid met  $\perp$ . Enkele voorbeelden van proposities zijn "Het regent", "Jan is 10 jaar oud" en "De deur is gesloten". Een variabele  $P$  is een propositie die waar of onwaar kan zijn. Als we willen zeggen dat  $P$  waar is, duiden we dat aan met de literal  $p$ . Als we willen zeggen dat  $P$  onwaar is, doen we dat met de literal  $\neg p$ . Variabelen kunnen gecombineerd worden met logische operatoren. We hebben al een operator gebruikt in de vorige zin, namelijk de negatie ( $\neg$ ). Naast de negatie zijn er nog twee belangrijke operatoren, de conjunctie ( $\wedge$ ) en disjunctie ( $\vee$ ). De conjunctie van twee variabelen is enkel waar als beide variabelen waar zijn, de disjunctie is waar als minstens één van de twee variabelen waar is. In tabel 2.1 geven we twee waarheidstabellen, die geven een overzicht van alle mogelijke waarheidswaarden van de variabelen samen met de resulterende waarheidswaarde na een bepaalde operatie.

Met behulp van variabelen en logische operatoren kunnen we logische formules construeren. Een formule over de variabelen  $\mathbf{X}$  kan geschreven worden als  $f(\mathbf{X})$ . Een logische formule  $f(\mathbf{X})$  kan op verschillende manieren voorgesteld worden; de wetten van De Morgan zijn daar een voorbeeld van, zie figuur 2.2.

| $P$     | $\neg P$ | $P$     | $Q$     | $P \wedge Q$ | $P \vee Q$ |
|---------|----------|---------|---------|--------------|------------|
| $\perp$ | $\top$   | $\perp$ | $\perp$ | $\perp$      | $\perp$    |
| $\top$  | $\perp$  | $\top$  | $\perp$ | $\perp$      | $\top$     |
| $\perp$ | $\top$   | $\perp$ | $\top$  | $\perp$      | $\top$     |
| $\top$  | $\perp$  | $\top$  | $\top$  | $\top$       | $\top$     |

TABEL 2.1: Waarheidstabellen met operaties negatie, conjunctie en disjunctie.

$$\begin{array}{ll}
\neg(P \wedge Q) = \neg P \vee \neg Q & P \wedge \top = P \quad P \wedge \perp = \perp \\
\neg(P \vee Q) = \neg P \wedge \neg Q & P \vee \top = \top \quad P \vee \perp = P
\end{array}$$

TABEL 2.2: De wetten van De Morgan en operaties met elementaire waarden.

Een veelgebruikt type formule is de Conjunctieve Normaalvorm (CNF). Een CNF bestaat uit de conjunctie van disjunctieclausules. Een disjunctieclausule is een logische formule die enkel uit de disjunctie van (genegateerde) variabelen bestaat, bijvoorbeeld  $(P \vee \neg Q \vee R)$ . Een formule in K-CNF is een CNF waarbij elke clausule uit exact K variabelen bestaat. Zo bestaat er voor elke mogelijke formule een voorstelling in 3-CNF.

De tegenpool van de disjunctieclausule is de conjunctieclausule en als tegenpool van de Conjunctieve Normaalvorm bestaat de Disjunctieve Normaalvorm (DNF). Een DNF is een logische formule die bestaat uit de disjuncties van conjunctieclausules. Een CNF kan getransformeerd worden naar een DNF door de negatie te nemen en vervolgens De Morgan's wetten en distributiviteit toe te passen en vice versa.

Als we veronderstellen dat een variabele  $P$  waar is in de formule  $f$ , schrijven we dat als  $f \mid P$ , lees "formule  $f$  gegeven  $P$ ".

## 2.2 SDD als deelverzameling van d-DNNF en superverzameling van OBDD

De meest belangrijke eigenschappen van d-DNNF zijn splitsbaarheid en determinisme, deze eigenschappen maken dat de voorstelling een grote behandelbaarheid heeft [CD08]. Twee belangrijke eigenschappen van de OBDD die de d-DNNF niet heeft zijn canoniciteit en een polytijd Apply-operatie. Canoniciteit houdt in dat een bepaalde kennisbasis altijd op dezelfde manier zal worden voorgesteld. Hierdoor kan equivalentiecontrole in polytijd en is zoeken naar een optimale representatie (door de variabelenvolgorde te veranderen) mogelijk. Een polytijd Apply-operatie houdt in dat twee OBDD's samengevoegd kunnen worden (de exclusieve disjunctie, disjunctie of conjunctie van twee OBDD's) in polytijd. Deze eigenschap maakt mogelijk dat een OBDD bottom-up kan gecompileerd worden. De OBDD is minder bondig dan de d-DNNF, maar door de canoniciteit en een polytijd Apply-operatie is ze in de praktijk wel het meest populair.

De Sentential Decision Diagram (SDD) heeft als eigenschappen gestructureerde splitsbaarheid [PD08] en sterk determinisme [PD10]. Deze eigenschappen zijn strikter dan splitsbaarheid en determinisme an sich. De SDD-taal is dus een strikte deelver-

zameling van de d-DNNF-taal. Verder is de SDD-taal een strikte superverzameling van de OBDD-taal. De SDD behoudt bepaalde belangrijke eigenschappen van de OBDD, namelijk een polytijd Apply-operatie en onder bepaalde voorwaarden canoniciteit. Het is al gebleken dat de SDD bondiger is dan de OBDD in termen van de padbreedte [Dar11] en voor bepaalde kennisbasissen is de SDD exponentieel meer compact [XCD12]. Voorbeelden van een OBDD en SDD zijn terug te vinden in figuur 2.2.

### 2.2.1 Sterk determinisme en partitionen

OBDD's zijn gebaseerd op Shannon-decomposities, hierbij wordt een formule  $f$  opgesplitst in twee elementen  $(X, f \mid X), (\neg X, f \mid \neg X)$  op basis van de variabele  $X$ . De SDD breidt dit idee uit door niet enkel te splitsen op de waarde van één variabele  $X$ , maar door te splitsen op de waarde van meerdere variabelen, een zogenaamde zin of sentence.

**Definitie 1 (( $\mathbf{X}, \mathbf{Y}$ )-decompositie [Dar11])** *Gegeven een Booleaanse functie  $f$  over variabelen  $\mathbf{X}, \mathbf{Y}$ , met  $\mathbf{X} \cap \mathbf{Y} = \emptyset$ . Als  $f = (p_1(\mathbf{X}) \wedge s_1(\mathbf{Y})) \vee \dots \vee (p_n(\mathbf{X}) \wedge s_n(\mathbf{Y}))$ , dan is  $\{(p_1, s_1), \dots, (p_n, s_n)\}$  een  $(\mathbf{X}, \mathbf{Y})$ -decompositie van  $f$ .*

Als ook nog geldt dat  $p_i \wedge p_j = \perp$  voor elke  $i \neq j$  dan wordt de decompositie sterk deterministisch op  $\mathbf{X}$  genoemd. Elk paar  $(p_i, s_i)$  wordt een element genoemd, met  $p_i$  de prime en  $s_i$  de sub. De grootte van de decompositie is het aantal elementen erin.

**Definitie 2 ( $\mathbf{X}$ -partitie [Dar11])** *Gegeven een  $(\mathbf{X}, \mathbf{Y})$ -decompositie  $\alpha$  van een functie  $f$  die sterk deterministisch is.  $\alpha$  wordt een  $\mathbf{X}$ -partitie van  $f$  genoemd a.s.a. de primes een partitie vormen.*

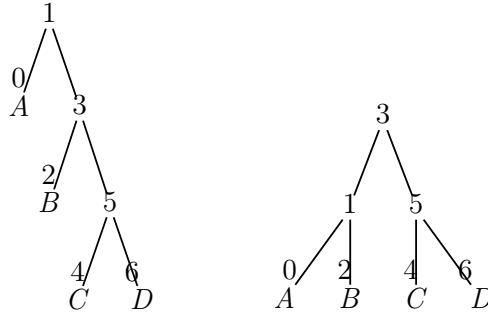
Een partitie heeft als eigenschappen:

1. Elke prime is niet logisch onwaar.
2. Elk paar van twee verschillende primes kunnen niet door dezelfde instantie van variabelen waar zijn.
3. De disjunctie van alle primes is logisch waar.

Als geldt dat elke sub in de  $\mathbf{X}$ -partitie verschillend is ( $s_i \neq s_j$  als  $i \neq j$ ), dan is  $\alpha$  **gecomprimeerd**. Het is bewezen dat een Booleaanse functie  $f(\mathbf{X}, \mathbf{Y})$  exact één gecomprimeerde  $\mathbf{X}$ -partitie heeft [Dar11]. Omwille van deze eigenschap is een gecomprimeerde SDD canoniek.

**Eigenschap 1 ( $\mathbf{X}$ -partitie van  $f \circ g$  [Dar11])** *Neem  $\circ$  een Booleaanse operator, en  $\{(p_1, s_1), \dots, (p_n, s_n)\}$  en  $\{(q_1, r_1), \dots, (q_m, r_m)\}$   $\mathbf{X}$ -partities van respectievelijk  $f(\mathbf{X}, \mathbf{Y})$  en  $g(\mathbf{X}, \mathbf{Y})$ , dan is  $\{(p_i \wedge q_j, s_i \circ r_j) \mid p_i \wedge q_j \neq \perp\}$  een  $\mathbf{X}$ -partitie van  $f \circ g$ .*

Eigenschap 1 is zeer essentieel voor de bruikbaarheid van de SDD. Het maakt een polytijd Apply-operatie mogelijk.



FIGUUR 2.1: Rechtslineaire en gebalanceerde vtree gebruikt voor figuur 2.2.

## 2.3 Vtree

In een OBDD wordt gesplitst op variabelen. De volgorde waarop wordt gesplitst, is gespecificeerd in de variabelenvolgorde. Voor een SDD moet ook een volgorde worden vastgelegd, maar er wordt gesplitst op meerdere variabelen in een keer. Hiervoor is de vtree verantwoordelijk. Een vtree voor variabelen  $\mathbf{X}$  is een binaire boom, waarbij de bladeren een één-op-één correspondentie hebben met de variabelen in  $\mathbf{X}$ . In figuur 2.1 kunnen we twee voorbeelden van een vtree zien. We refereren naar een knoop in de vtree of de deelboom geworteld in die knoop met  $v$ . Het linker- en rechterkind of de linker- en rechterdeelboom van  $v$  duiden we aan met  $v^l$  en  $v^r$ . De ouder van  $v$ , indien die bestaat, duiden we aan met  $v^p$ . Het verschil tussen linkse en rechtse kindknopen in de vtree is belangrijk omwille van het verschil tussen primes en subs. We specificeren vier verschillende types vtree:

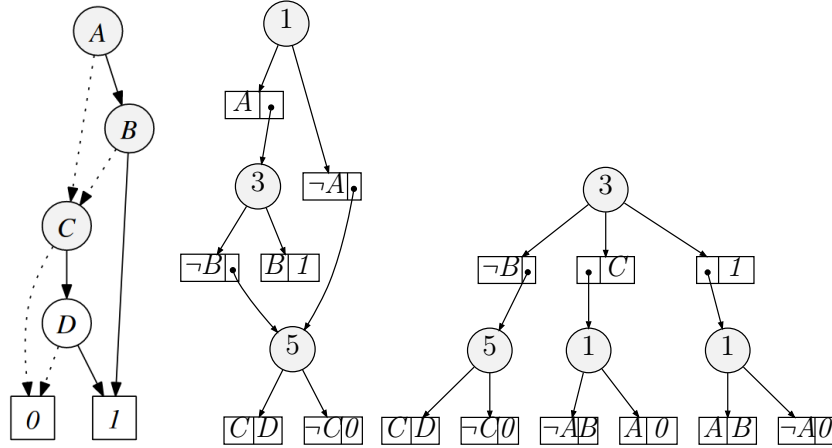
- Rechtslineaire vtree: elk linkerkind is een bladknoop.
- Linkslinaire vtree: elk rechterkind is een bladknoop.
- Gebalanceerde vtree: elke bladknoop zit op dezelfde diepte, voor zover mogelijk is.
- Willekeurige vtree.

Een SDD die een rechtslineaire vtree respecteert, is een OBDD.

Nu we de vtree geïntroduceerd hebben, kunnen we de SDD formeel definiëren. De originele SDD paper [Dar11] introduceert een afbeelding  $\langle \cdot \rangle$  van SDD's naar Booleaanse formules. Die herhalen we hier omdat ze essentieel is om SDD's te begrijpen.

**Definitie 3 (Sentential Decision Diagram [Dar11])**  $\alpha$  is een SDD die vtree  $v$  respecteert a.s.a.

1.  $\alpha = \perp$  of  $\alpha = \top$ ,  $\langle \perp \rangle = \text{Onwaar}$ ,  $\langle \top \rangle = \text{Waar}$ .
2.  $\alpha = X$  of  $\alpha = \neg X$  en  $v$  is een blad met variabele  $X$ .  $\langle X \rangle = X$ ,  $\langle \neg X \rangle = \neg X$ .



FIGUUR 2.2: OBDD, rechtslineaire SDD en gebalanceerde SDD voor de formule  $f = (A \wedge B) \vee (C \wedge D)$ . De getallen bij de SDD's geven aan welke vtrees (deel)SDD respecteert.

3.  $\alpha = \{(p_1, s_1), \dots, (p_n, s_n)\}$ , elke  $p_i$  is een SDD die een deelboom van  $v^l$  respecteert en elke  $s_i$  is een SDD die een deelboom van  $v^r$  respecteert. Bovendien moeten  $\langle p_1 \rangle, \dots, \langle p_n \rangle$  een partitie vormen.  $\langle \alpha \rangle = \bigvee_{i=1}^n \langle p_i \rangle \wedge \langle s_i \rangle$ .

Een belangrijke type SDD's zijn gecomprimeerde SDD's. Een SDD is gecomprimeerd a.s.a. voor elke decompositie  $\{(p_1, s_1), \dots, (p_n, s_n)\}$  in de SDD geldt dat  $s_i \not\equiv s_j$  voor  $i \neq j$ . De grootte van een SDD  $\alpha$  wordt gedefinieerd als de som van de groottes van al zijn decomposities en wordt geschreven als  $|\alpha|$ .

Een SDD is genormaliseerd voor een vtree  $v$  als de primes genormaliseerd zijn voor vtrees  $v^l$  en de subs voor  $v^r$ . Het verschil is dat de primes of subs geen deelboom van  $v^l$  of  $v^r$  respecteren, maar de vtrees  $v^l, v^r$  zelf.

## 2.4 Apply-operatie

Een Apply-operatie heeft als invoer twee kennisbasissen en een Booleaanse operatie, en als uitvoer de combinatie van die twee kennisbasissen volgens die operatie. Gegeven SDD's  $\langle \alpha \rangle = A$  en  $\langle \beta \rangle = B$ , dan is  $\langle \text{Apply}(\alpha, \beta, \circ) \rangle = A \circ B$ . Hoe de Apply-operatie werkt, hangt af van het type kennisbasis. Zoals eerder vermeld is er voor OBDD's een polytijd Apply-operatie. Het is relatief makkelijk om een algoritme te verzinnen dat een Booleaanse operatie op twee SDD's uitvoert in polytijd met behulp van de eerder geziene eigenschap 1 van  $\mathbf{X}$ -partities. Die eigenschap vereist wel dat beide SDD's genormaliseerd zijn voor dezelfde vtree. In algoritme 1 wordt pseudocode gegeven voor zo een Apply-operatie. Het resultaat van die Apply-operatie is een SDD, maar deze SDD is niet per se gecomprimeerd, zelfs als de invoerSDD's gecomprimeerd zijn. Op lijn 12 kan het voorkomen dat de berekende sub  $s$  al in  $\gamma$  zit en opnieuw toegevoegd wordt, dit is in strijd met de voorwaarden voor een gecomprimeerde

SDD. Een niet-gecomprimeerde SDD is niet canoniek. Zoals eerder vermeld is canoniciteit in de praktijk zeer interessant, dus kunnen we niet de standaard Apply-operatie gebruiken. De aanpassingen aan de pseudocode om canoniciteit te behouden zijn minimaal. We moeten enkel bij het toevoegen van  $(p, s)$  detecteren of er een element  $(q, s)$  al in  $\gamma$  zit. Als dat zo is moeten we  $(\text{Apply}(p, q, \vee), s)$  toevoegen aan  $\gamma$ . Dit aangepaste algoritme zullen we de Compressie-Apply-operatie noemen. Het is gebleken dat de Compressie-Apply-operatie niet in polytijd uitvoerbaar is [dBD15]. Het bewijs hiervan is gebaseerd op bepaalde logische formules waarvan de SDD's na de Compressie-Apply-operatie exponentieel groter zijn. Wel is uit datzelfde werk gebleken dat in de praktijk een Compressie-Apply-operatie wel degelijk verkiesbaar is. Aangezien in de praktijk canoniciteit een belangrijke eigenschap is zullen we in dit werk veronderstellen dat elke SDD een gecomprimeerde SDD is en met de Apply-operatie bedoelen we de compressie-behoudende versie. Als nadeel moeten we nu veronderstellen dat elke Apply-operatie van twee SDD's in theorie exponentieel veel tijd en geheugen nodig kan hebben.

---

**Algorithm 1**  $\text{Apply}(\alpha, \beta, \circ)$ :  $\alpha$  en  $\beta$  zijn SDD's genormaliseerd voor dezelfde vtreesknoop,  $\circ$  is een Booleaanse operator [Dar11].

---

**Cache**(., ., .) = **nil** bij start.

**Expand**( $\gamma$ ) geeft  $\{(\top, \top)\}$  terug als  $\gamma = \top$ ;  $\{(\top, \perp)\}$  als  $\gamma = \perp$ ; anders  $\gamma$ .

**UniqueD**( $\gamma$ ) geeft  $\top$  terug als  $\gamma = \{(\top, \top)\}$ ;  $\perp$  als  $\gamma = \{(\top, \perp)\}$ ; anders de unieke SDD met elementen  $\gamma$ .

```

1: if  $\alpha$  en  $\beta$  zijn constanten of variabelen then
2:   return  $\alpha \circ \beta$ 
3: else if Cache( $\alpha, \beta, \circ$ )  $\neq$  nil then
4:   return Cache( $\alpha, \beta, \circ$ )
5: else
6:    $\gamma \leftarrow \{\}$ 
7:   for all elementen  $(p_i, s_i)$  in Expand( $\alpha$ ) do
8:     for all elementen  $(q_j, r_j)$  in Expand( $\beta$ ) do
9:        $p \leftarrow \text{Apply}(p_i, q_j, \wedge)$ 
10:      if  $p$  is consistent then
11:         $s \leftarrow \text{Apply}(s_i, r_j, \circ)$ 
12:        voeg element  $(p, s)$  toe aan  $\gamma$ 
13:      end if
14:    end for
15:  end for
16: end if
17: return Cache( $\alpha, \beta, \circ$ )  $\leftarrow$  UniqueD( $\gamma$ )

```

---

Met behulp van de Apply-operatie kunnen we de negatie van een SDD meer in detail bekijken. Neem SDD  $\alpha$ , dan is  $\neg\alpha = \alpha \oplus \top = \text{Apply}(\alpha, \top, \oplus)$ . Met behulp van eigenschap 1 kunnen we de operatie doorvoeren naar de elementen van  $\alpha$ . Neem

$\alpha = \{(p_1, s_1), \dots, (p_n, s_n)\}$  en  $\top = \{\top, \top\}$ .

$$\begin{aligned}\neg\alpha &= \alpha \oplus \top = \{(p_1, s_1), \dots, (p_n, s_n)\} \oplus \{(\top, \top)\} \\ \neg\alpha &= \{(p_i \wedge \top, s_i \oplus \top) \mid i \in \{1, 2, \dots, n\}\} \\ \neg\alpha &= \{(p_i, \neg s_i) \mid i \in \{1, 2, \dots, n\}\}\end{aligned}\tag{2.1}$$

**Eigenschap 2 (Negatie van een SDD)** *Neem  $\alpha = \{(p_1, s_1), \dots, (p_n, s_n)\}$  een SDD, dan is  $\neg\alpha = \{(p_1, \neg s_1), \dots, (p_n, \neg s_n)\}$  de inverse SDD.*

De negatie van  $\alpha$  kan recursief verder gedaan worden op de subs  $s_i$  van  $\alpha$ .

## Hoofdstuk 3

# Probleemstelling

Er is in het verleden veel onderzoek gedaan om compacte OBDD's efficiënt te bekomen. De SDD is in theorie bondiger dan de OBDD, maar er moeten verschillende zaken onderzocht worden om dat in de praktijk waar te maken. Onderzoeksonderwerpen zijn bijvoorbeeld het effect van de vtree op de grootte van de SDD, omvangslimieten, gespecialiseerde compilers, et cetera. In dit werk onderzoeken we het probleem van het kiezen van een Apply-volgorde tijdens de bottom-up compilatie van een kennisbasis naar een SDD. Het kiezen van de Apply-volgorde bepaalt welke tussenresultaten bekomen worden en bepaalt mee de totale compilatietijd.

### 3.1 Bottom-up compilatie

Een kennisbasis, logische formule of logisch circuit kan in het algemeen als een Gewortelde Gerichtte Acyclische Graaf (rooted-DAG) beschouwd worden. De rooted-DAG bestaat uit knopen die een conjunctie, disjunctie of negatie voorstellen van de kinderen van die knoop. De bladeren van de rooted-DAG stellen variabelen voor. Elke knoop stelt dus een deel formule voor van de kennisbasis. Een bottom-up compilatie zet eerst alle bladeren van de rooted-DAG om naar SDD's, die SDD's worden door middel van Apply-operaties en negaties samengevoegd volgens de rooted-DAG. Op die manier wordt de SDD-representatie van elke knoop van de rooted-DAG berekend, en dat van onder naar boven. Een algoritme voor bottom-up compilatie van een rooted-DAG is beschreven in algoritme 2 en een diagram van een deel van een bottom-up compilatie is te vinden in figuur 3.1.

In de praktijk is gebleken dat SDD's die tijdens de compilatie worden berekend, zogenaamde tussenresultaten, variëren in berekeningstijd. Willen we een efficiënte compilatie, dan moeten we een Apply-volgorde vinden die snel berekend kan worden. De SDD's die een knoop in de rooted-DAG voorstellen moeten verplicht berekend worden in de bottom-up compilatie (in figuur 3.1 de SDD's  $\gamma, \gamma_1, \gamma_2, \gamma_3, \gamma_4$  en  $\gamma_5$ ). Die SDD's zijn onvermijdelijk nodig om andere SDD's hogerop in de rooted-DAG te berekenen (tenzij de structuur van de rooted-DAG wordt veranderd). In het algoritme komen deze onvermijdelijke SDD's overeen met de SDD's die worden gegeven als

---

**Algorithm 2 Bottom-up compilatie**( $n$ ):  $n$  is in het begin de rootknoop van de rooted-DAG.

---

**Heuristiek**(SDD-lijst) kiest 2 SDD's uit SDD-lijst volgens de heuristiek. **SDD** stelt de functionaliteit van de SDD-bibliotheek voor

```

1: if  $n.\text{operatie} = \text{Atoom}$  then
2:   return  $\text{SDD.Literal}(n.\text{variabele})$ 
3: end if
4: if  $n.\text{operatie} = \text{Negatie}$  then
5:   return  $\text{SDD.Negatie}(\text{Bottom-up compilatie}(n.\text{kinderen}[0]))$ 
6: end if
7:  $\text{kindCompilaties} \leftarrow \text{map}(\text{Bottom-up compilatie}, n.\text{kinderen})$ 
8: while  $\text{kindCompilaties}$  bevat meer dan één element do
9:    $\gamma_i, \gamma_j \leftarrow \text{kindCompilaties.pop}(), \text{kindCompilaties.pop}()$ 
10:   $\gamma_k \leftarrow \text{Apply}(\gamma_i, \gamma_j, n.\text{operatie})$ 
11:   $\text{kindCompilaties} \leftarrow (\text{kindCompilaties} \cup \gamma_k) / \{\gamma_i, \gamma_j\}$ 
12: end while
13: return  $\text{kindCompilaties}[0]$ 

```

---

resultaat in regels 2, 5 of 13. Als knoop  $n$  een variabele voorstelt, doet het algoritme een initialisatie. Stelt  $n$  de negatie voor, dan moet de negatie uitgevoerd worden. Beide bewerkingen zijn triviaal omwille van eigenschap 2.

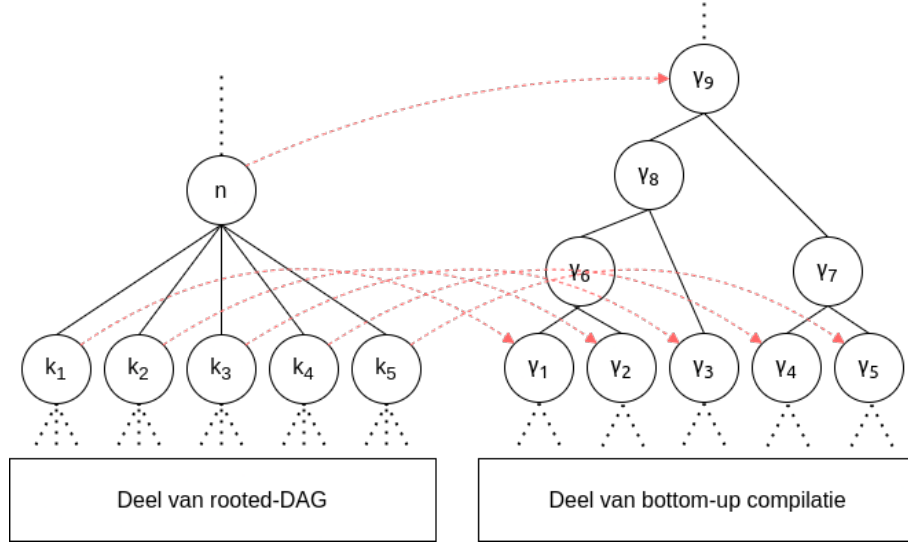
Waar we de bottom-up compilatie kunnen optimaliseren is bij de berekening van de tussenresultaten  $\gamma_k$  als knoop  $n$  een andere operatie voorstelt. We kunnen telkens maar twee SDD's samennemen met behulp van de Apply-operatie. In figuur 3.1 worden in volgorde de paren  $(\gamma_1, \gamma_2)$ ,  $(\gamma_4, \gamma_5)$ ,  $(\gamma_6, \gamma_3)$  en  $(\gamma_8, \gamma_7)$  gekozen. Als een knoop  $n$  van de rooted-DAG meer dan twee kinderen  $k_i$  heeft, moet er een keuze gemaakt worden welke kinderen met welke worden Applied (Deze keuze gebeurt op regel 9 in algoritme 2). Niet elke Apply-operatie duurt even lang en de mogelijke tussenresultaten kunnen verschillen in grootte. Het eindresultaat, de SDD die overeenkomt met knoop  $n$ , zal altijd dezelfde zijn wegens de canoniciteit van SDD's. De volgorde waarop de kinderen van knoop  $n$  worden samengevoegd, is te kiezen. We verkiezen een volgorde die zo snel mogelijk berekend kan worden.

## 3.2 Formele probleemstelling

Het probleem kunnen we formeel als volgt stellen.

**Definitie 4 (Bottom-up knoopcompilatie)** *De bottom-up compilatie van één knoop  $n = \bigcirc_{i=1}^N k_i$ , met  $\bigcirc$  de functionele compositie van een logische operatie, wordt geschreven als een sequentie van SDD's  $\gamma_1, \dots, \gamma_M$ , met  $\gamma_M \equiv n$  en voor elke  $i \in \{1, 2, \dots, M\}$ :*

1.  $\gamma_i \equiv k_i$  voor  $i \in \{1, 2, \dots, N\}$ , of



FIGUUR 3.1: Een mogelijke bottom-up compilatie van een rooted-DAG:  $\gamma_9$  is de SDD die de formule onder  $n$  voorstelt. Elk kind  $k_i$  van knoop  $n$  is gecompileerd naar een SDD  $\gamma_i$ .

2.  $\gamma_i = \text{Apply}(\gamma_j, \gamma_k, \circ)$  met  $j, k < i$  en  $\gamma_j$  en  $\gamma_k$  zijn genormaliseerd voor dezelfde vtree, of
3.  $\gamma_i \equiv \gamma_j$  met  $j < i$  en  $\gamma_i$  en  $\gamma_j$  zijn genormaliseerd voor verschillende vtrees.

De derde mogelijkheid in deze definitie komt overeen met het dynamisch aanpassen van de vtree tijdens de compilatie. Dat zullen we niet gebruiken in dit werk, zodat de keuze van de heuristiek het enige is dat effect heeft op de compilatietijd (meer uitleg hierover is te vinden in sectie 4.1). We zullen enkel de eerste twee mogelijkheden gebruiken en elke  $\gamma_i$  slechts één keer gebruiken als parameter voor de Apply-operatie. Met die aannames zullen er  $N - 2$  tussenresultaten  $\{\gamma_{N+1} \dots \gamma_{2 \times N - 2}\}$  berekend worden voordat  $\gamma_M$  berekend kan worden,  $M = 2 \times N - 1$ . We willen de totale compilatietijd  $ct$  van knoop  $n$  minimaliseren. Die wordt als volgt gemeten:

$$ct = \sum_{i=N+1}^M t(\gamma_i) \quad (3.1)$$

met  $t(\gamma_i)$  de tijd nodig om  $\gamma_i$  te berekenen.

We verwachten dat de groottes van de tussenresultaten een grote invloed zullen hebben op de totale compilatietijd, kleinere SDD's kunnen sneller berekend worden. Naast de compilatietijd zullen we dus ook deze groottes meten.

### 3.3 Voorbeeld

Neem de logische formule  $f = (A \vee B) \wedge (B) \wedge (\neg B \vee C)$ . We willen de SDD  $\alpha$  overeenkomstig met deze formule berekenen. Eerst compileren we de drie deelformules van  $f$  naar een SDD.

$$\gamma_1 = \{(A, \top), (\neg A, B)\} \equiv (A \vee B) \quad (3.2)$$

$$\gamma_2 = B \quad (3.3)$$

$$\gamma_3 = \{(B, C), (\neg B, \top)\} \equiv (\neg B \vee C) \quad (3.4)$$

Nadat deze drie SDD's berekend zijn moet er een keuze gemaakt worden, berekenen we eerst  $\gamma_1 \wedge \gamma_2$ ,  $\gamma_1 \wedge \gamma_3$  of  $\gamma_2 \wedge \gamma_3$ .

$$\gamma_4' = \gamma_1 \wedge \gamma_2 = B \quad (3.5)$$

$$\gamma_4'' = \gamma_1 \wedge \gamma_3 = \{(\neg A, \{(B, C), (\neg B, \perp)\}), (A, \{(B, C), (\neg B, \top)\})\} \quad (3.6)$$

$$\gamma_4''' = \gamma_2 \wedge \gamma_3 = \{(B, C), (\neg B, \perp)\} \quad (3.7)$$

In de laatste stap moeten we de SDD van de overgebleven deelformule toevoegen aan  $\gamma_4$ . Dit resultaat zal altijd hetzelfde zijn omwille van de canoniciteit.

$$\gamma_5 = \{(B, C), (\neg B, \perp)\} \equiv f \quad (3.8)$$

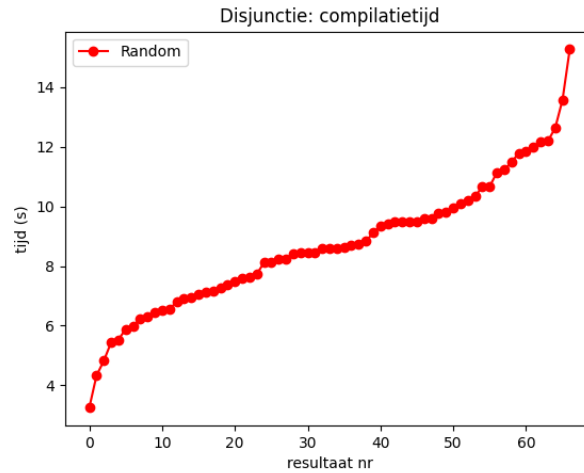
In dit voorbeeld zien we dat  $\gamma_4''$  duidelijk groter is dan  $\gamma_4'$ ,  $\gamma_4'''$  en zelfs  $\gamma_5$ . We verwachten dat een Apply-operatie met  $\gamma_4''$  langer duurt dan een Apply-operatie met  $\gamma_4'$  of  $\gamma_4'''$  vanwege de grootte.

### 3.4 In de praktijk

Één voorbeeld is niet representatief voor de algemene situatie, om uit te testen hoe groot dit probleem is in de praktijk hebben we een klein experiment uitgevoerd. We genereren de kinderen  $k_i$  van een logische formule  $n = \bigcirc_{i=1}^N k_i$  en compileren die naar de SDD's  $\gamma_i$ , daarna proberen we verschillende willekeurige compilatiesequentie's  $\gamma_{N+1}, \dots, \gamma_M$  en meten we de compilatietijd  $ct$ .

Hoe we  $k_i$  genereren en andere details zijn terug te vinden in de sectie Experimenten, de willekeurige sequentie wordt later bepaald in heuristiek 3. Wat voor nu belangrijk is, is dat we één probleeminstantie  $n = \bigcirc_{i=1}^N k_i$  hebben. De verschillende  $k_i$  worden gegenereerd als 3-CNF's, met een bepaald aantal variabelen  $\nu = 28$  en een willekeurig aantal clausules  $y \in \{1, 2, \dots, 4.5 * \nu\}$  voor elke  $k_i$ . We compileren telkens met een willekeurige volgorde, we krijgen dus willekeurige sequentie's van  $\gamma_{N+1}, \dots, \gamma_M$ . Van elke compilatie meten we de tijd  $ct$ . Voor deze probleeminstantie zijn de SDD's genormaliseerd voor een gebalanceerde vtree.

We geven in figuur 3.2 de variatie weer van 67 willekeurige sequenties, dat doen we door de compilatietijden te sorteren en zo te plotten. Voor deze probleeminstantie zien we dat de slechtste compilatiesequentie meer dan vier keer trager is dan de beste.



FIGUUR 3.2: Variatie in compilatietijd  $ct$  voor een probleeminstantie met gebalanceerde vtree.

Dit is slechts voor één probleeminstantie, voor andere probleeminstanties is er soms meer en soms minder variatie. We hebben dit experiment 100 keer uitgevoerd per type vtree, dus met 400 verschillende probleeminstanties in totaal. Gemiddeld is de traagste compilatietijd 262% zo traag als de snelste compilatietijd voor gebalanceerde vtrees. Voor willekeurige en rechte lineaire vtrees is dit gemiddelde ongeveer gelijk. Voor linkslineaire vtrees verschilt de traagste tijd gemiddeld 561% met de snelste tijd.

## Hoofdstuk 4

# Gerelateerd onderzoek

In de vorige secties hebben we de theorie van de SDD en het probleem van de Apply-volgorde besproken, die zijn belangrijk om de heuristieken te begrijpen die we later zullen introduceren. Nu bekijken we relevant onderzoek specifiek voor het probleem van de Apply-volgorde. We bespreken enkele compilers, heuristieken voor een optimale vtree en een fasetransitie bij de grootte van kenniscompilaties. Een aanzienlijk deel van de hier aangehaalde bronnen beperken zich tot formules in Conjunctieve Normaalvorm. In dit werk limiteren we ons juist niet tot enkel CNF's, maar tot algemene rooted-DAG's, toch zullen we er enige aandacht aan besteden omdat we CNF's wel gebruiken om SDD's te genereren.

In principe zijn de hieropvolgende technieken/heuristieken terug te brengen tot algemene heuristieken voor Beperking-Tevredenstellingsproblemen (CSP) zoals beschreven in [RN20]:

- **Opdelen in deelproblemen** door onafhankelijke componenten te vinden: Als twee SDD's  $\alpha$  en  $\beta$  geen variabelen met elkaar delen, zijn ze onafhankelijk. Volledig onafhankelijke  $\gamma_i$  vinden zal meestal niet mogelijk zijn voor grote  $N$ , maar wellicht zijn er paren  $(\gamma_i, \gamma_j)$  met weinig overeenkomstige variabelen en paren met veel overeenkomstige variabelen.
- **Componenten met minst aantal mogelijke waarden** eerst bepalen: De grootte van een SDD  $\alpha$  is het aantal elementen per vtreesknoop. Hoe het aantal elementen verandert na Apply met SDD  $\beta$  hangt af van de elementen die  $\beta$  bevat.
- **Componenten die in veel beperkingen een rol spelen** eerst bepalen: Een variabele aan de linkerzijde van de vtree (prime) is onderhevig aan meer restricties dan een variabele aan de rechterzijde (sub). De diepte van een variabele in de vtree speelt ook een rol.

Een resultaat om in het achterhoofd te houden werd geleverd door De Colnet en Mengel [dCM21]. Zij tonen aan dat de bottom-up compilatie naar structured-DNNF's (str-DNNF) van bepaalde formules exponentiële tijdscomplexiteit heeft, ondanks het

bestaan van een constante str-DNNF representatie. De exponentiële tijdscomplexiteit volgt uit tussenresultaten van exponentiële grootte, ongeacht de volgorde van Apply-operaties. str-DNNF is een superverzameling van SDD, dit resultaat geldt dus ook voor SDD's.

## 4.1 Vtree heuristieken

De OBDD heeft een variabelenvolgorde. Deze volgorde heeft een grote invloed op de totale grootte van de voorstelling. Daarom is er al veel onderzoek gedaan naar het zoeken van volgordes die resulteren in een relatief kleine OBDD. Zo is gebleken dat een optimale variabelenvolgorde vinden een NP-Compleet probleem is [BW96]. Omwille van die reden zijn er heuristieken ontwikkeld. Statistische heuristieken zoals MINCE [AMS04] en FORCE [AMS03] bepalen vòòr de compilatie een goede volgorde, gebaseerd op de partitionering van variabelen volgens een Min-Cut. Dynamische heuristieken zoals het zeefalgoritme [Rud93] passen de variabelenvolgorde aan tijdens de compilatie, gebaseerd op wat er al gecompileerd is. De ene techniek sluit de andere niet uit, er kan eerst statisch naar een goede startvolgorde gezocht worden, die volgorde kan dynamisch dan nog geoptimaliseerd worden. Een variabelenvolgorde kan ook beschouwd worden als een rechte lijn vtree.

Voor de SDD geldt dat de vtree een grote invloed heeft op de grootte van de compilatie [CD13a]. Er kan, net zoals bij de OBDD, op voorhand statisch een vtree bepaald worden aan de hand van structuur in de rooted-DAG of de vtree kan dynamisch aangepast worden tijdens de compilatie van de rooted-DAG naar SDD.

Er zijn al verschillende statistische minimalisatieheuristieken onderzocht voor vtrees. Meestal in context van een specifiek type probleem met al dan niet een uitbreiding van de SDD als compilatierepresentatie. Een vtree-heuristiek voor symbolische modelverificatie wordt gegeven in [VL20]. Twee gelijkaardige heuristieken worden gegeven in [LBdB17] en [HKZDM19]. De vtrees worden in deze twee laatste gevallen gebruikt voor de structuur van de PSDD en P-XSDD, beide uitbreidingen op de SDD. De drie heuristieken gebruiken gelijkaardige technieken als de statistische variabelenvolgordeheuristieken bij de OBDD, gebaseerd op een min-cut. Darwiche geeft in [Dar11] een statistische vtreeheuristiek voor de compilatie van CNF's, minfill vtrees.

Een techniek voor dynamische minimalisatie is voorgesteld in [CD13a]. Er wordt een manier aangehaald om de ruimte van vtrees te doorzoeken. Zo kan er tijdens de compilatie naar een betere vtree gezocht worden. Een compilatie-algoritme voor CNF's dat deze dynamische minimalisatie gebruikt, werd ook voorgesteld.

Een betere vtree zoeken kan best op speciale momenten plaatsvinden, waarbij vermoedelijk de SDD's die de vtree respecteren, niet meer (veel) veranderen. Stel de vtree  $v$  van een SDD  $\gamma_i$  wordt dynamisch geoptimaliseerd. Stel ook dat er nog een tweede SDD is  $\gamma_j$  die variabelen met  $\gamma_i$  deelt. De vtree waarvoor  $\gamma_j$  is genormaliseerd, moet dezelfde deeltvtree onder  $v$  hebben als de vtree waarvoor  $\gamma_i$  genormaliseerd is.  $\gamma_j$  zal aangepast moeten worden voordat Apply op  $\gamma_i$  en  $\gamma_j$  kan worden uitgevoerd.

Niet alleen kost de aanpassing aan  $\gamma_j$  extra tijd, na Apply moet de vtree onder  $v$  opnieuw dynamisch geoptimaliseerd worden.

De SDD  $\gamma_i$  genormaliseerd voor vtreenknoop  $v$  zal niet meer veranderen als elke SDD  $\gamma_j$  met variabelen in de deelboom  $v$  opgenomen is in  $\gamma_i$ , zelfs als  $\gamma_i$  met andere SDD's wordt Applied. Zodra  $\gamma_i$  niet of bijna niet meer zal veranderen, kunnen we de deelboom  $v$  optimaliseren met de garantie dat er geen of weinig aanpassingen moeten gemaakt worden aan andere SDD's en dat de deelboom onder  $v$  optimaal blijft. De optimalisatie is "nevenkostenvrij".

De situatie waarin we de vtree nevenkostenvrij of bijna nevenkostenvrij kunnen optimaliseren komt normaal gezien niet veel voor in een rooted-DAG. Voor de compilatie van een CNF naar een SDD is de dynamische minimalisatie wel bruikbaar. In een CNF-compiler worden enkel clauses bij elkaar gevoegd om de compilatie te verkrijgen. De kans dat we (bijna alle) clauses kunnen partitioneren over de vtree is groot, dit is exact wat gedaan wordt in heuristiek 1. In combinatie met de heuristiek kan de vtree-optimalisatie vaak toegepast worden én zijn de nevenkosten klein. Het resultaat is dat tijdens de compilatie de vtree deelboom per deelboom van onder naar boven dynamisch geoptimaliseerd wordt.

Het effect van de dynamische optimalisatie van de vtree tijdens de compilatie van een algemene rooted-DAG is nog niet bestudeerd, ondanks dat de techniek wel bestaat. Los daarvan heeft dynamische minimalisatie een invloed op de compilatietijd, terwijl we enkel het effect van de heuristieken op de compilatietijd willen meten. Daarom zullen we in dit werk gebruik maken van een vooraf bepaalde vtree (gebalanceerd, linkslineair,...) met willekeurige variabelenvolgorde en passen we geen dynamische minimalisatie toe.

## 4.2 Compilers

De huidige state of the art SDD-compiler voor CNF's is een top-down compiler. Die werd voorgesteld door Oztok en Darwiche [OD18] en maakt gebruik van DPLL. De compilatietaal van deze compiler is Decision-SDD. Een top-down compiler heeft niet het probleem van een Apply-volgorde.

Een bottom-up compiler voor CNF's werd voorgesteld door Choi en Darwiche in [CD13a]. Deze compiler doet aan dynamische minimalisatie door tijdens de compilatie aanpassingen aan de vtree te maken. Verder gebruikt de compiler twee heuristieken om een Apply-volgorde op te leggen aan alle clauses. De eerste heuristiek is beschreven in 1.

**Heuristiek 1 (Vtree partitionering)** *Wijs elke clause  $c$  toe aan de laagste vtreenknoop  $v$  die de variabelen van  $c$  bevat. Gebruik de vtree als recursieve partitionering van alle clauses, met elke vtreenknoop  $v$  verantwoordelijk voor een verzameling van clauses. Doorloop de partitionering bottom-up van links naar rechts. De heuristiek compileert recursief de clauses gelinkt aan de kinderen van  $v$ .*

Zelfs als de clauses gepartitioneerd worden, kan het zijn dat er meer dan 2 clauses gelinkt worden aan dezelfde vtreeknoop  $v$ . In dat geval moet er nog altijd een keuze gemaakt worden welke clauses eerst Applied worden. De compiler zal in dat geval een tweede, interne, heuristiek raadplegen. In het werk wordt een tweede heuristiek gegeven, zie heuristiek 2, maar deze werkt enkel op CNF's. Verder in dit werk zullen we die vervangen met onze eigen heuristieken.

**Heuristiek 2 (Clause-lengte)** *De clauses van vtreeknoop  $v$  worden van kort naar lang (aantal variabelen) doorlopen.*

Heuristiek 1 wordt ook gebruikt in JUICE, een Julia package voor logische en probabilistische circuits [DKL<sup>+</sup>21]. Hiervoor wordt de heuristiek uitgebreid om te werken op rooted-DAG's, niet enkel op CNF's. Er wordt geen interne heuristiek gebruikt.

Verder zijn er twee compilers voor specifieke probleeminstanties ontwikkelt. Een compiler voor probabilistische grafische modellen [CKD13] en een compiler voor probabilistische logische programma's [VRdBR14]. Beide compilers maken gebruik van structuur die voortkomt uit het type invoer om efficiënte compilatie mogelijk te maken. Deze structuur wordt ook gebruikt om een volgorde te geven aan de Apply-operaties. De compilers verwachten een specifiek soort invoer, deze veronderstelling kan niet worden gemaakt voor de compilatie van een algemene rooted-DAG.

Het probleem van grote tussenresultaten komt ook voor bij de constructie van OBDD's. Daarom kijken we ook naar OBDD-compilers. Neem bijvoorbeeld MINCE [AMS04], naast een heuristiek voor de variabelenvolgorde worden ook heuristieken voorgesteld voor de volgorde waarin de clauses worden samengevoegd. Één voorstel werkt met de hypergraaf van de formule, elke knoop van de rooted-DAG is een knoop in deze hypergraaf. Voor elke variabele  $X$  wordt een verbinding gelegd tussen de knopen die  $X$  bevatten. Op die hypergraaf wordt gebalanceerde min-cut uitgevoerd om de clauses op te splitsen. Een tweede voorstel, voor CNF's, legt een volgorde op aan de clauses gebaseerd op de variabele van elke clause, die het laatste in de variabelenvolgorde voorkomt. De clauses met variabelen die laat in de variabelenvolgorde voorkomen, zullen het eerst samengevoegd worden.

### 4.3 Fasetransities

In het domein van SAT-problemen is er een concept genaamd fasetransitie. De moeilijkheid van het vinden van een bevredigende oplossing verandert abrupt voor bepaalde waarden van een parameter [CA96b], [MSL92]. In  $K$ -SAT draait het om het vinden van een toewijzing van waarheidswaarden aan variabelen in een formule  $f$  in  $K$ -CNF zodat de formule geldig is.

De fasetransitie conjunctuur is gelinkt aan de verhouding  $r$  van het aantal clauses  $y$  ten opzichte van het aantal variabelen  $\nu$ ,  $r = y/\nu$ . De conjunctuur stelt dat voor  $K$ -SAT er een kritieke waarde  $r^*$  is. Neem  $f$  een CNF met ratio  $r$ . Als  $r < r^*$  en  $n \leftarrow \infty$ , is er met kans  $P = 1$  een combinatie van waarheidswaarden zodat  $f$  geldig is. Als  $r > r^*$  en  $n \leftarrow \infty$  is er met kans  $P = 1$  geen combinatie van waarheidswaarden

zodat  $f$  geldig is. In de praktijk is gemerkt dat oplossingsstrategieën voor  $K$ -SAT vooral moeite hebben met formules waarvoor  $r \approx r^*$ , als  $r \not\approx r^*$  is een oplossing vinden relatief makkelijk. We krijgen een makkelijk-moeilijk-makkelijk patroon.

In veel andere problemen is er een soortgelijk kritiek punt voor een bepaalde parameter, waarbij instanties met die parameter op het kritieke punt, moeilijk oplosbaar zijn. Als de parameter onder het kritieke punt ligt is het probleem onderbepaald en bijgevolg makkelijk oplosbaar. Boven het kritieke punt is het probleem overbepaald en zijn er bijgevolg weinig oplossingen, een goed algoritme kan dan ook makkelijk een oplossing vinden [CKT91].

Er is ook een fasetransitie waar te nemen bij de groottes van kenniscompilaties en de compilatietijd, met als parameter de verhouding  $r$  van aantal clausules tot aantal variabelen van een CNF. Empirische studies naar de grootte van DNNF-compilaties werden gedaan in [GWY15] en naar de grootte en compilatietijd van SDD's in [GRM20].

**Hypothese 1 (fasetransitie in compilatiegrootte [GWY15])** *Gegeven een aantal variabelen  $\nu$  en een compilatietaal  $L$ ,  $L \subseteq DNNF$ , er bestaat een positief reëel getal  $r_c$  zodat voor elk paar CNF's  $(C_1, C_2)$  geldt dat als*

- $r_{C_1} < r_{C_2} < r_c$ : *De verwachte grootte van compilatie  $L_{C_1}$  is kleiner dan de verwachte grootte van compilatie  $L_{C_2}$ .*
- $r_c < r_{C_1} < r_{C_2} < r^*$ : *De verwachte grootte van compilatie  $L_{C_1}$  is groter dan de verwachte grootte van compilatie  $L_{C_2}$ .*

met  $r_{C_i}$  de ratio  $r$  voor CNF  $C_i$  en  $L_{C_i} \equiv C_i$ .

De kritieke waarde  $r_c$  voor 3-CNF wordt geschat op 1,8.

## 4.4 Conjunctieve normaalvorm

Om onze experimenten te doen hebben we probleeminstanties nodig die bestaan uit een reeks SDD's waarop de Apply-operatie moet worden uitgevoerd in een bepaalde volgorde. Deze SDD's compileren we vanuit willekeurig gegenereerde CNF's. Dat doen we omdat CNF's relatief makkelijk zonder bias genereerbaar zijn. De generatie gebeurt zoals beschreven in [MSL92]. We kunnen voor elke CNF drie parameters kiezen.  $k$  voor de lengte van de clausules,  $y$  voor het aantal clausules en  $\nu$  voor het aantal variabelen. Elke CNF wordt gegenereerd door  $y$  clausules uniform en onafhankelijk te genereren. Elke clausule wordt gegenereerd door zonder teruglegging willekeurig  $k$  unieke variabelen te selecteren uit  $\nu$  variabelen en elke variabele met kans 0.5 negatief te maken. De reden dat we met willekeurige probleeminstanties werken is als volgt.

- We kunnen zoveel willekeurige instanties genereren als we nodig hebben.
- De instanties zijn aanpasbaar aan de hand van de drie parameters  $k, \nu$  en  $y$ .

- Willekeurige instanties hebben geen structuur die gebruikt kan worden.

Tenslotte geven we nog een citaat uit [CA96a]. "there is an argument that randomly generated problems represent a “core” of hard satisfiability problems. Certainly real problems have structure, regularity, symmetries, etc., and algorithms will have to make use of this structure to simplify the problems. However, once all the structure is “squeezed out”, the remainder will be a problem that requires search, and if all the structure is used up then the remainder will presumably be a random problem."

## Hoofdstuk 5

# Heuristieken

De heuristieken die we in deze sectie beschrijven dienen om telkens twee SDD's  $\gamma_i$  en  $\gamma_j$  te verwijderen uit een verzameling van SDD's, die Applied moeten worden. Het resultaat  $\gamma_i \circ \gamma_j$  wordt toegevoegd aan de verzameling van SDD's, zodat de verzameling nu een element minder bevat. Dit proces komt overeen met de tweede optie in definitie 4. Het zoeken wordt herhaald tot de verzameling slechts uit één SDD bestaat, alle SDD's zijn dan samengevoegd tot één SDD. In Algoritme 2 van de bottom-up compilatie moet de heuristiek als extra parameter worden meegegeven en moet regel 9 worden aangepast naar  $\gamma_i, \gamma_j \leftarrow \text{Heuristiek}(\text{kindCompilaties})$  om de heuristiek te gebruiken. Heuristiek 1 is moeilijker te beschrijven op deze manier. Voor deze heuristiek wordt een gespecialiseerd algoritme gegeven in sectie 5.2.1. Van elke heuristiek geven we de tijdscomplexiteit, die bovenop de tijdscomplexiteit van de compilatie komt. De tijdscomplexiteit zal onder andere in functie zijn van  $N$ , het aantal SDD's dat Applied moet worden en  $\nu$ , het aantal variabelen aanwezig in de probleeminstantie.

We gebruiken heuristieken om een compilatiesequentie te genereren waarvan we de compilatietijd  $ct$  kunnen meten. Deze meting zegt echter niks als we geen compilatiesequentie hebben waarmee we kunnen vergelijken. Als basisgeval implementeren we een willekeurige heuristiek.

**Heuristiek 3 (Willekeurig)** *kies willekeurig twee SDD's*

### 5.1 Bovengrens na Apply

Een eerste heuristiek die we voorstellen is gebaseerd op de maximale grootte na Apply  $|\gamma_i \circ \gamma_j|$ . We vermoeden dat kleine tussenresultaten gunstiger zijn voor de compilatietijd, daarom willen we eerst de Apply-operatie doen met de kleinste maximale grootte. Uit eigenschap 1 volgt dat voor de polytijd Apply-operatie geldt:

$$|\gamma_i \circ \gamma_j| \leq |\gamma_i| * |\gamma_j| \quad (5.1)$$

Deze bovengrens geldt niet voor de Compressie-Apply-operatie (niet polytijd), desondanks is deze bovengrens wel een goede schatting. In de praktijk kunnen we deze

heuristisch implementeren door alle SDD's te sorteren op hun grootte en telkens de twee kleinsten te kiezen.

**Heuristiek 4 (Kleinste eerst)** *Kies de twee SDD's met de kleinste grootte.*

Het berekenen van de grootte van een SDD  $\alpha$  heeft een tijdscomplexiteit van  $O(|\alpha|)$ . De complexiteit van deze heuristiek is  $O(\max_{\gamma_i}(|\gamma_i|) \times N \times \log(N))$ . In de praktijk is het mogelijk om de grootte van een SDD tijdens de uitvoering van een programma bij te houden, zodat het opvragen hiervan in constante tijd kan gebeuren. Een correcte implementatie brengt de tijdscomplexiteit naar  $O(N \times \log(N))$ .

Heuristiek 4 is niet optimaal, er zijn betere, strengere bovengrenzen en er zijn andere zaken waarmee rekening kan gehouden worden, zoals de variabelen die in de SDD aanwezig zijn. We doen een tweede poging door middel van een strengere bovengrens.

### 5.1.1 Bovengrens door elementen

De grootte van een SDD  $\alpha$  is de som van de groottes van de decomposities. We kunnen dit ook anders schrijven:

$$|\alpha| = \sum_{v \in V} \#el(\alpha, v) \quad (5.2)$$

met  $\#el(\alpha, v)$  het aantal elementen onder vtreesknoop  $v$  in SDD  $\alpha$  en  $V$  de verzameling van alle vtreesknopen. Als we algoritme 1 bestuderen, zien we dat het aantal recursieve oproepen afhangt van het aantal elementen per vtreesknoop  $v$ . Het aantal elementen dat wordt toegevoegd aan  $\alpha \circ \beta$  is bijgevolg gelimiteerd:

$$\#el(\alpha \circ \beta, v) \leq \#el(\alpha, v) \times \#el(\beta, v) \quad (5.3)$$

We vullen vergelijking 5.3 in in vergelijking 5.2 om de volgende bovengrens te bekomen.

$$|\alpha \circ \beta| \leq \sum_{v \in V} \#el(\alpha, v) \times \#el(\beta, v) \quad (5.4)$$

Net zoals vergelijking 5.1 geldt deze bovengrens enkel voor de polytijd Apply-operatie, toch zullen we hem gebruiken.

### 5.1.2 Extra bovengrens door primes

Er is een tweede, meer theoretische, bovengrens voor het aantal elementen in een vtreesknoop  $v$ , gebaseerd op de variabelenverzameling van de primes.

**Definitie 5 (Maximum grootte van een partitie)** *Een partitie over  $\nu$  variabelen bestaat maximaal uit  $2^\nu$  primes.*

**Bewijs door inductie:** De grootste partitie over 0 variabelen is de partitie met een prime:  $\{\top\}$ . Neem aan dat een partitie over  $\nu - 1$  variabelen maximaal uit  $2^{\nu-1}$  primes bestaat. Neem de partitie  $PAR_{\nu-1}$  een partitie van maximale grootte

$$PAR_{\nu-1} = \{P_1, P_2, \dots, P_{2^{\nu-1}}\} \quad (5.5)$$

We weten door de maximale grootte van  $PAR_{\nu-1}$  dat voor eender welke prime  $Q_{\nu-1}$  over  $\nu - 1$  variabelen moet gelden dat er een prime  $P_i \in PAR_{\nu-1}$  bestaat zodat  $P_i \wedge Q_{\nu-1} = \perp$  of  $P_i$  en  $Q_{\nu-1}$  zijn voldaanbaar door dezelfde instantie  $I$  van variabelen.

We breiden  $PAR_{\nu-1}$  uit naar een partitie over  $\nu$  variabelen door de nieuwe variabele  $O$  toe te voegen.

$$PAR_{\nu} = \{P_1 \wedge O, P_1 \wedge \neg O, P_2 \wedge O, P_2 \wedge \neg O, \dots, P_{2^{\nu-1}} \wedge O, P_{2^{\nu-1}} \wedge \neg O\} \quad (5.6)$$

De grootte van  $PAR_{\nu}$  is  $2^{\nu-1} \times 2 = 2^{\nu}$ . Eender welke prime  $Q_{\nu}$  over  $\nu$  variabelen kunnen we schrijven als de conjunctie van een prime  $Q_O$  over de variabele  $O$  en een prime  $Q_{\nu-1} = Q_{\nu} \mid Q_O$ :

$$\begin{aligned} Q_{\nu} &= Q_{\nu-1} \wedge O \text{ of,} \\ Q_{\nu} &= Q_{\nu-1} \wedge \neg O \end{aligned} \quad (5.7)$$

Er zijn twee gevallen: Ofwel is er een  $P_i \in PAR_{\nu-1}$  zodat  $Q_{\nu-1}$  en  $P_i$  voldaanbaar zijn door dezelfde instantie  $I$  van variabelen. Dan geldt dat voor  $Q_{\nu} = Q_{\nu-1} \wedge O$  de instantie  $I \wedge O$  zowel  $Q_{\nu}$  als  $P_i \wedge O \in PAR_{\nu}$  voldaanbaar maakt. Voor  $Q_{\nu} = Q_{\nu-1} \wedge \neg O$  maakt de instantie  $I \wedge \neg O$  zowel  $Q_{\nu}$  als  $P_i \wedge \neg O \in PAR_{\nu}$  voldaanbaar.

Het andere geval is wanneer er een prime  $P_i \in PAR_{\nu-1}$  is zodat  $P_i \wedge Q_{\nu-1} = \perp$ . Voor elke mogelijke  $Q_{\nu}$  bestaat er een prime  $(P_i \wedge P_O) \in PAR_{\nu}$  zodat  $(P_i \wedge P_O) \wedge Q_{\nu} = \perp$ , met  $P_O = O$  of  $\neg O$ .

$$\begin{aligned} (Q_{\nu-1} \wedge O) \wedge (P_i \wedge \neg O) &= O \wedge \neg O = \perp \\ (Q_{\nu-1} \wedge \neg O) \wedge (P_i \wedge O) &= O \wedge \neg O = \perp \end{aligned} \quad (5.8)$$

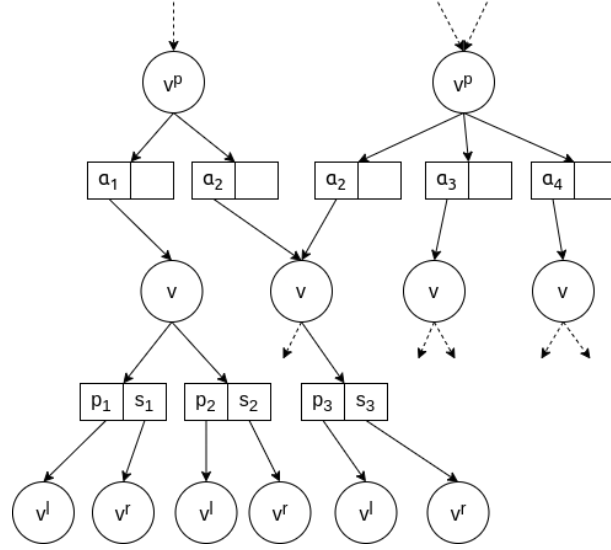
Er kan geen prime  $Q_{\nu}$  toegevoegd worden aan  $PAR_{\nu}$  zonder de voorwaarden van een partitie te breken, dit bewijst dat  $PAR_{\nu}$  een partitie van maximale grootte is.  $\square$

Neem  $\{(p_1(\mathbf{X}), s_1(\mathbf{Y})), \dots, (p_n(\mathbf{X}), s_n(\mathbf{Y}))\}$  en  $\{(q_1(\mathbf{X}), r_1(\mathbf{Y})), \dots, (q_m(\mathbf{X}), r_m(\mathbf{Y}))\}$  de elementen van SDD's  $\alpha(\mathbf{X}, \mathbf{Y})$  en  $\beta(\mathbf{X}, \mathbf{Y})$ .  $\alpha$  en  $\beta$  zijn genormaliseerd voor vtreesknoop  $v$ , met de  $\mathbf{X}$  de variabelen in  $v^l$  en  $\mathbf{Y}$  de variabelen in  $v^r$ . Elke  $p_i$  en  $q_j$  is een SDD over de variabelen  $\mathbf{X}$ . Het kan zijn dat een variabele  $X \in \mathbf{X}$  in geen enkele  $p_i$  of  $q_j$  voorkomt. We schrijven daarom dat elke  $p_i$  een SDD is over de variabelen  $\mathbf{X}_{\alpha}$  en elke  $q_j$  een SDD over de variabelen  $\mathbf{X}_{\beta}$ . De primes van een SDD moeten een partitie vormen. Volgens definitie 5 kan een partitie over de variabelen  $\mathbf{X}$  maximaal uit  $2^{|\mathbf{X}|}$  primes bestaan.

$$\#el(\alpha, v) \leq 2^{|\mathbf{X}_{\alpha}|} \quad \#el(\beta, v) \leq 2^{|\mathbf{X}_{\beta}|} \quad (5.9)$$

We zijn geïnteresseerd in het aantal elementen van  $\alpha \circ \beta$  onder vtreesknoop  $v$ .

$$\alpha \circ \beta = \{(p_i \wedge q_j, s_i \circ r_j) \mid p_i \wedge q_j \neq \perp\} \quad (5.10)$$

FIGUUR 5.1: Deel van SDD  $\Psi$ , met vtreenknoten  $v^p$ ,  $v$ ,  $v^l$  en  $v^r$ .

Na de Apply-operatie moeten de primes  $p_i \wedge q_j$  een partitie moeten vormen over de variabelen  $\mathbf{X}_\alpha \cup \mathbf{X}_\beta$ . Het aantal elementen onder vtreenknoop  $v$  voor  $\alpha \circ \beta$  is hierdoor begrensd.

$$\#el(\alpha \circ \beta, v) \leq 2^{|\mathbf{X}_\alpha \cup \mathbf{X}_\beta|} \quad (5.11)$$

Vergelijking 5.11 geldt enkel omdat  $v$  de wortelknoop van  $\alpha$  en  $\beta$  is. Het kan zijn dat  $\alpha$  en  $\beta$  respectievelijk deel zijn van de grotere SDD's  $\Psi$  en  $\Omega$  zoals in figuur 5.1. Neem  $v^p$  de ouderknoop van vtreenknoop  $v$ . Elk element in  $\Psi$  onder vtreenknoop  $v^p$  bevat een  $\alpha_i$  als prime of als sub als  $v$  het linker- dan niet het rechterkind van  $v^p$  is. In totaal zijn er maximum  $\#el(\Psi, v^p)$  verschillende  $\alpha_i$ . Elke  $\alpha_i$  kan bestaan uit andere elementen. Hetzelfde geldt voor de verschillende  $\beta_i$  in  $\Omega$ .

$$\#el(\Psi, v) \leq \#el(\Psi, v^p) \times 2^{|\mathbf{X}_\alpha|} \quad \#el(\Omega, v) \leq \#el(\Omega, v^p) \times 2^{|\mathbf{X}_\beta|} \quad (5.12)$$

In deze vergelijking zijn  $\mathbf{X}_\alpha$  en  $\mathbf{X}_\beta$  de verzamelingen van variabelen die een voorkomen hebben in een prime van respectievelijk  $\alpha_i$  en  $\beta_i$ . In de Apply-operatie  $\Psi \circ \Omega$  zal elk element onder vtreenknoop  $v^p$  van  $\Psi$  met elk element onder  $v^p$  van  $\Omega$  Applied worden. Voor elk van die Apply-operaties geldt vergelijking 5.11. We kunnen een vergelijking opschrijven voor het aantal elementen in  $\Psi \circ \Omega$  onder  $v$ :

$$\#el(\Psi \circ \Omega, v) \leq \#el(\Psi \circ \Omega, v^p) \times 2^{|\mathbf{X}_\alpha \cup \mathbf{X}_\beta|} \quad (5.13)$$

We hebben nu wel een bovengrens voor  $\#el(\Psi \circ \Omega, v)$ , maar de vraag is of die ook nuttig is, is deze bovengrens strenger dan die in vergelijking 5.3? Als we de vergelijkingen van 5.12 invoegen in vergelijking 5.3 krijgen we

$$\#el(\Psi \circ \Omega, v) \leq \#el(\Psi, v^p) \times \#el(\Omega, v^p) \times 2^{|\mathbf{X}_\alpha| + |\mathbf{X}_\beta|}. \quad (5.14)$$

De bovengrens in 5.13 kan kleiner zijn dan de bovengrens in 5.14 als  $\mathbf{X}_\alpha \cap \mathbf{X}_\beta \neq \emptyset$ . Hoe kleiner het symmetrisch verschil  $\mathbf{X}_\alpha \Delta \mathbf{X}_\beta$  is, hoe beter vergelijking 5.13 de bovengrens gaat beïnvloeden.

### 5.1.3 Extra bovengrens door subs

Er kan een tweede soortgelijke bovengrens gevonden worden in termen van de subs.

**Definitie 6 (Aantal verschillende SDD's over  $\nu$  variabelen)** *Er bestaan  $2^{2^\nu}$  verschillende gecomprimeerde SDD's over  $\nu$  variabelen*

**Bewijs:** Een kennisbasis over  $\nu$  variabelen bestaat uit een combinatie van werelden, elke wereld is een instantie van de  $\nu$  variabelen. In een wereld is elke variabele ofwel waar ofwel onwaar, we hebben  $2^\nu$  mogelijke werelden. Ofwel zit een wereld in de kennisbasis, ofwel niet, we kunnen op die manier  $2^{2^\nu}$  verschillende kennisbasissen creëren. Wegens de canoniciteit van gecomprimeerde SDD's kunnen we een één-op-één afbeelding maken tussen gecomprimeerde SDD's en kennisbasissen. Het aantal verschillende SDD's is bijgevolg gelijk aan het aantal verschillende kennisbasissen.  $\square$

Neem opnieuw  $\{(p_1(\mathbf{X}), s_1(\mathbf{Y})), \dots (p_n(\mathbf{X}), s_n(\mathbf{Y}))\}$  en  $\{(q_1(\mathbf{X}), r_1(\mathbf{Y})), \dots (q_m(\mathbf{X}), r_m(\mathbf{Y}))\}$  de elementen van SDD's  $\alpha(\mathbf{X}, \mathbf{Y})$  en  $\beta(\mathbf{X}, \mathbf{Y})$ . elke  $s_i$  en  $r_j$  is een SDD over de variabelen  $\mathbf{Y}$ . Zoals bij de primes schrijven we specifieker dat elke  $s_i$  een SDD over  $\mathbf{Y}_\alpha$  en elke  $r_j$  een SDD over  $\mathbf{Y}_\beta$  is. De subs van een SDD moeten geen partitie vormen, dus elke  $s_i$  kan eender welke SDD zijn. Als de subs  $s_i$  en  $s_j$  gelijk zijn worden de elementen  $(p_i, s_i)$  en  $(p_j, s_j)$  samengenomen naar  $(p_i \vee p_j, s_i)$ . Het aantal elementen onder  $v$  is bijgevolg gelimiteerd door het aantal mogelijke verschillende subs. Met definitie 6 kunnen we volgende ongelijkheden opstellen.

$$\#el(\alpha, v) \leq 2^{(2^{|\mathbf{Y}_\alpha|})} \quad \#el(\beta, v) \leq 2^{(2^{|\mathbf{Y}_\beta|})} \quad (5.15)$$

Op dezelfde wijze als bij de primes kunnen we een bovengrens bekomen voor het aantal elementen van  $\alpha \circ \beta$ , zoals in 5.11, maar dan voor  $\mathbf{Y}_\alpha$  en  $\mathbf{Y}_\beta$  en met een extra macht.

$$\#el(\alpha \circ \beta, v) \leq 2^{(2^{|\mathbf{Y}_\alpha \cup \mathbf{Y}_\beta|})} \quad (5.16)$$

Die kunnen we dan weer uitbreiden naar het meer algemene geval  $\Psi \circ \Omega$ .

$$\#el(\Psi \circ \Omega, v) \leq \#el(\Psi \circ \Omega, v^p) \times 2^{(2^{|\mathbf{Y}_\alpha \cup \mathbf{Y}_\beta|})} \quad (5.17)$$

Deze bovengrens is in de praktijk minder nuttig dan 5.13, omdat de bovengrens een macht extra heeft in de vergelijking.

De drie bovengrenzen in 5.3, 5.13 en 5.17 geven ons de vergelijking

$$\#el(\alpha \circ \beta, v) \leq \text{Min} \left\{ \begin{array}{l} \#el(\alpha, v) \times \#el(\beta, v) \\ \#el(\alpha \circ \beta, v^p) \times 2^{|\mathbf{X}_\alpha \cup \mathbf{X}_\beta|} \\ \#el(\alpha \circ \beta, v^p) \times 2^{(2^{|\mathbf{Y}_\alpha \cup \mathbf{Y}_\beta|})} \end{array} \right\} \quad (5.18)$$

Met  $v^p$  de ouderknoop van  $v$ ,  $\mathbf{X}_\alpha$  en  $\mathbf{X}_\beta$  de verzameling van variabelen in  $\alpha$  en  $\beta$  onder vtreesknoop  $v^l$  en  $\mathbf{Y}_\alpha$  en  $\mathbf{Y}_\beta$  de verzameling van variabelen in  $\alpha$  en  $\beta$  onder vtreesknoop  $v^r$ .

We merken op dat de bovengrens op  $\#el(\alpha \circ \beta, v)$  afhangt van de bovengrens op  $\#el(\alpha \circ \beta, v^p)$ . In de praktijk is het dus belangrijk dat de vtree van boven naar onder wordt doorlopen tijdens het berekenen van de totale bovengrens.

**Heuristiek 5 (Elementgewijze bovengrens)** *Kies de twee SDD's  $\gamma_i, \gamma_j$  waarvoor  $\sum_{v \in V} \#el(\gamma_i \circ \gamma_j, v)$  de kleinste bovengrens heeft.*

Net zoals bij heuristiek 4 geldt de gegeven bovengrens enkel voor de polytijd Apply-operatie. In de gecomprimeerde versie is de bovengrens in vergelijking 5.18 niet strikt (vergelijking 5.3 geldt dan niet). Toch vermoeden we dat deze heuristiek goed kan werken.

#### 5.1.4 Praktische implementatie

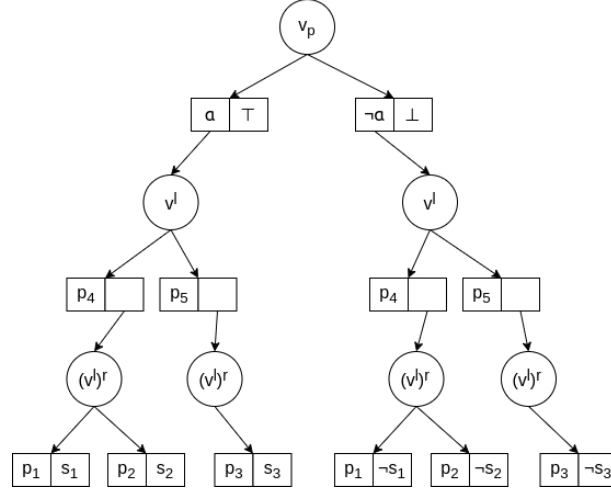
In een computerprogramma kunnen we een SDD  $\alpha$  volledig doorlopen en in een lijst opslaan hoeveel elementen de SDD bevat per vtreesknoop, de tijdscomplexiteit hiervan is  $O(|\alpha|)$ . In theorie zou deze telling ook bijgehouden kunnen worden tijdens de uitvoering van Apply-operaties, zodat het niet apart berekend moet worden. De lijsten van twee SDD's kunnen we paarsgewijs vermenigvuldigen om een eerste schatting te krijgen van het aantal elementen na de Apply-operatie. De lijsten bevatten een getal voor elke vtreesknoop, de lengte van de lijsten is  $O(\nu)$ , met  $\nu$  het aantal variabelen in de kennisbasis.

Een belangrijk gegeven bij de telling is als volgt. Stel een SDD  $\gamma_i$  is niet genormaliseerd voor vtreesknoop  $v$ , maar het linkerkind  $v^l$ . Dan is  $\{(\gamma_i, \top), (\neg\gamma_i, \perp)\}$  een SDD genormaliseerd voor vtreesknoop  $v$ . Ook al lijkt het initieel alsof de SDD maar één element heeft onder  $v$ , omdat  $\gamma_i$  een prime is en de primes een partitie moeten vormen, heeft de SDD twee elementen onder  $v$ . Verder weten we door eigenschap 2 dat de negatie doorgevoerd kan worden in de subs. Om een correcte telling te bekomen moet het aantal elementen worden verdubbeld voor elke vtreesknoop op het meest rechtse pad van de vtree  $v^l$ , inclusief  $v^l$  zelf ( $\neg\gamma_i$  is namelijk genormaliseerd voor  $v^l$ ), in figuur 5.2 geven we een voorbeeld.

Naast het tellen van de elementen per vtreesknoop kunnen we ook een lijst opvragen van alle variabelen die aanwezig zijn in een SDD. Deze lijst moeten we dan omzetten naar een lijst van variabelen per vtreesknoop. Met die informatie kunnen we vergelijking 5.18 gebruiken om de eerste schatting van elk paar SDD's bij te stellen, de tijdscomplexiteit hiervan is  $O(\nu)$ .

Van elk paar SDD's berekenen we de bovengrens van de grootte na Apply, deze bovengrenzen sorteren we en we kiezen het paar SDD's met de kleinste bovengrens. Het sorteren kost  $O(N^2 \times \log(N^2))$  tijd.

De totale tijdscomplexiteit van deze heuristiek is  $O(\max_{\gamma_i}(|\gamma_i|) \times N + (\nu + \log(N^2)) \times N^2)$ . De complexiteit van deze heuristiek wordt net zoals heuristiek 4 gedomineerd door de term  $(\max_{\gamma_i}(|\gamma_i|))$ . Voor deze heuristiek kan het nuttig zijn om het aantal elementen per vtreesknoop automatisch bij te houden zodat dit in constante tijd kan worden opgevraagd.



FIGUUR 5.2: Voorbeeld van hoe de normalisatie het aantal elementen beïnvloedt. SDD  $\alpha$  is genormaliseerd voor vtreenknoop  $v^l$ . SDD  $\{(\alpha, \top), (\neg\alpha, \perp)\}$  is genormaliseerd voor vtreenknoop  $v$ . De negatie wordt doorgevoerd naar de subs in  $(v^l)^r$  terwijl de primes gelijk blijven tussen  $\alpha$  en  $\neg\alpha$ .

## 5.2 Heuristieken op basis van vtree

### 5.2.1 Partitionering volgens de vtree

Deze subsectie beschrijft de praktische implementatie van heuristiek 1. Zoals eerder vermeld wordt die gebruikt in JUICE [DKL<sup>+</sup>21] en de CNF-compiler van [CD13b]. In tegenstelling tot andere heuristieken legt deze heuristiek geen directe volgorde op aan de SDD's  $\gamma_i$ , maar worden de SDD's recursief gepartitioneerd volgens de vtree. Daarom geven we een specifiek algoritme voor de bottom-up compilatie met deze heuristiek in Algoritme 3. In de laatste lijn, waarbij de SDD's voor dezelfde vtreenknoop zijn genormaliseerd, wordt algoritme 2 opgeroepen met een secundaire heuristiek. De complexiteit van de heuristiek hangt af van de complexiteit van **Partitioneer**(SDDs, vtreenknoop  $v$ ). De opsplitsing van de kinderen gebeurt door te kijken of het kind genormaliseerd is voor een vtreenknoop in de linkerdeelboom of rechterdeelboom van  $v$ . De SDD-bibliotheek die we gebruiken voorziet rechtstreekse functionaliteit om te testen of een vtree een deelboom is van een andere vtree, in constante tijd. Zonder die functionaliteit is de tijdscomplexiteit lineair in de grootte van de vtree  $O(\nu)$ . Een oproep van **Partitioneer** kost  $O(N)$  tijd. Door de recursiviteit van **PartitieCompilatieKnoop** kan **Partitioneer** meerdere keren opgeroepen worden op een lijst van SDD's. Het aantal recursieve oproepen is gelimiteerd door de diepte van de vtree, die op zijn beurt gelimiteerd is door het aantal variabelen  $\nu$ . De tijdscomplexiteit van deze heuristiek is bijgevolg  $O(N * \nu)$ , hierbij is niet de tijdscomplexiteit van de secundaire heuristiek gerekend.

---

**Algorithm 3** **PartitieCompilatie**(knoop): knoop is in het begin de rootknoop van de rooted-DAG.

---

**Partitioneer**(kinderen,  $v$ ) geeft drie lijsten terug, één met alle kinderen die onder  $v^l$  zitten, één met alle kinderen onder  $v^r$ , en de overige kinderen. **interneHeuristiek** is de secundaire heuristiek die wordt gebruikt. **SDD** stelt de functionaliteit van de SDD-bibliotheek voor.

```

1: if knoop.operatie = Atoom then
2:   return SDD.Literal(knoop.variabele)
3: end if
4: if knoop.operatie = Negatie then
5:   return SDD.Negatie(PartitieCompilatie(knoop.kinderen[0]))
6: end if
7: kindSDDs = map(PartitieCompilatie, knoop.kinderen)
8: return PartitieCompilatieKnoop(kindSDDs, knoop.vtreeknoop)

```

**PartitieCompilatieKnoop**(kindSDDs, heuristiek,  $v$ )

```

1: links, rechts, midden = Partitioneer(kindSDDs,  $v$ )
2: if links bevat mistens één element then
3:   midden  $\leftarrow$  midden  $\cup$  PartitieCompilatieKnoop(links,  $v^l$ )
4: end if
5: if rechts bevat mistens één element then
6:   midden  $\leftarrow$  midden  $\cup$  PartitieCompilatieKnoop(rechts,  $v^r$ )
7: end if
8: return Bottom-upCompilatie(midden, interneHeuristiek)

```

---

### 5.2.2 Variabelenvolgorde op basis van de vtree

We hebben al heuristieken bepaald op basis van een bovengrens en op basis van de vtree. In deze subsectie proberen we heuristieken te bepalen op basis van de variabelen in de SDD's en waar die variabelen zich bevinden in de vtree.

We bekijken de Apply-operatie van SDD  $\alpha$  met variabele  $X$ .

$$\alpha = \{(p_1, s_1), \dots, (p_n, s_n)\} = \{(p_i, s_i) \mid i \in \{1, 2, \dots, n\}\} \quad (5.19)$$

$\alpha$  is genormaliseerd voor vtreeknoop  $v$ . Als  $X$  een variabele onder  $v^l$  is dan geldt:

$$\alpha \circ X = \alpha \circ \{(X, \top), (\neg X, \perp)\} \quad (5.20)$$

We kunnen dit uitschrijven met behulp van eigenschap 1.

$$\alpha \circ X = \{(p_i \wedge X, s_i \circ \top), (p_i \wedge \neg X, s_i \circ \perp) \mid i \in \{1, 2, \dots, n\}\} \quad (5.21)$$

De operatie  $\circ$  kan een conjunctie of een disjunctie voorstellen. We kunnen de elementaire gelijkheden toepassen op basis van de operatie, zie tabel 2.2. In geval van een conjunctie wordt vergelijking 5.21

$$\alpha \wedge X = \{(p_i \wedge X, s_i), (p_i \wedge \neg X, \perp) \mid i \in \{1, 2, \dots, n\}\} \quad (5.22)$$

De primes  $p_i \wedge \neg X$  hebben dezelfde sub, om compressie te behouden zullen ze samengenomen worden door middel van een disjunctie. Omdat we weten dat alle primes  $p_i$  een partitie moeten vormen kunnen we de vergelijking verder versimpelen.

$$\bigvee_{i=1}^n (p_i \wedge \neg X) = \bigvee_{i=1}^n (p_i) \wedge \neg X = \top \wedge \neg X = \neg X \quad (5.23)$$

We kunnen dus vergelijking 5.22 nog verder versimpelen naar

$$\alpha \wedge X = \{(p_i \wedge X, s_i) \mid i \in \{1, 2, \dots, n\}\} \cup \{(\neg X, \perp)\} \quad (5.24)$$

Voor de disjunctie wordt vergelijking 5.21

$$\alpha \vee X = \{(p_i \wedge X, \top), (p_i \wedge \neg X, s_i) \mid i \in \{1, 2, \dots, n\}\} \quad (5.25)$$

Op gelijkaardige manier als bij vergelijking 5.22 kunnen we vergelijking 5.25 verder versimpelen naar

$$\alpha \vee X = \{(p_i \wedge \neg X, s_i) \mid i \in \{1, 2, \dots, n\}\} \cup \{(X, \top)\} \quad (5.26)$$

Het kan ook zijn dat de variabele  $X$  onder  $v^r$  valt in plaats van  $v^l$ , dan krijgen we een andere situatie.

$$\alpha \circ X = \alpha \circ (\top \wedge X) \quad (5.27)$$

$$\alpha \circ X = \{(p_i, s_i \circ X) \mid i \in \{1, 2, \dots, n\}\} \quad (5.28)$$

In dit geval kan het zijn dat de sub  $s_i \circ X$  gelijk is aan sub  $s_j \circ X$  en zullen primes  $p_i$  en  $p_j$  samengenomen worden. Deze compressie-behoudende stap is exact wat kan leiden tot exponentiële groei van de grootte van  $\alpha \circ X$  [dBD15].

De verschillende besproken gevallen lijsten we hier op in tabelvorm:

| $\alpha \circ X$ | $X \in v^l$                                        | $X \in v^r$               |                            |
|------------------|----------------------------------------------------|---------------------------|----------------------------|
| Conjunctie       | $\{(p_i \wedge X, s_i)\} \cup \{(\neg X, \perp)\}$ | $\{(p_i, s_i \wedge X)\}$ | $i \in \{1, 2, \dots, n\}$ |
| Disjunctie       | $\{(p_i \wedge \neg X, s_i)\} \cup \{(X, \top)\}$  | $\{(p_i, s_i \vee X)\}$   |                            |

(5.29)

Zolang  $X$  voorkomt in  $v^l$ , dus als prime, groeit de grootte van de SDD maar lineair. Als  $X$  in  $v^r$  voorkomt kan het zijn dat subs gelijk worden en primes samengenomen moeten worden.

We kunnen tabel 5.29 nog verder analyseren, bijvoorbeeld wat er gebeurt als  $X$  al in  $\alpha$  voorkomt of hoe de hoogte van vtreesknoop  $v$  belangrijk is (aantal knopen tussen  $v$  en de wortelknoop). Voor deze sectie proberen we het echter op een simpele manier te bekijken.

Al de vergelijkingen en de tabel zijn veralgemeenbaar naar  $\alpha \circ \beta$ , met  $\beta$  een SDD genormaliseerd voor  $v^l$  of  $v^r$ . We concluderen dat in algoritme 1, zolang  $\beta$  onder  $v^l$  valt, de SDD maximaal lineair groeit in grootte. Zodra  $\beta$  onder  $v^r$  valt, kan de compressiestap optreden. Als  $\beta$  niet genormaliseerd is voor  $v^l$  of  $v^r$ , maar  $v$ , kunnen we niks speciaals zeggen over de Apply-operatie. Deze veralgemening versterkt de theoretische onderbouwing voor heuristiek 1.

De veralgemening is nuttig, maar niet nuttig als de SDD's niet  $v^l$  of  $v^r$ , maar  $v$  respecteren. Daarom kijken we naar de individuele variabelenvoorkomens in een SDD. Gebaseerd op tabel 5.29 kunnen we een heuristiek bedenken: Apply eerst die SDD's, die voorkomens hebben van variabelen die links in de vtree voorkomen. In onze experimenten werkte dit minder goed voor niet-gebalanceerde vtrees. We moeten ook rekening houden met hoe hoog elke variabele in de vtree voorkomt. Voor deze heuristiek geven we prioriteit aan variabelen die hoog voorkomen.

**Heuristiek 6 (Top-down variabelenvolgorde)** *Leg een volgorde op aan alle variabelen door de vtree breedte-eerst van links naar rechts te doorlopen. Kies de twee SDD's die een variabele bevatten die als eerste voorkomt in de volgorde.*

De implementatie van deze heuristiek vereist het sorteren van de SDD's en het sorteren vereist de vergelijking van de variabelenvoorkomens. Het berekenen van het aantal variabelenvoorkomens in een SDD  $\alpha$  kost  $O(|\alpha|)$  tijd. De SDD-bibliotheek die we gebruiken slaat bepaalde structuren op tijdens de uitvoering van een programma, waardoor de lijst van variabelen in constante tijd kan worden opgevraagd. Het vergelijken van twee lijsten kan in lineaire tijd. De tijdscomplexiteit is bijgevolg  $O(N * \log(N) * \nu)$ .

De heuristiek geeft een volgorde aan de Apply-operaties dmv. de meest belangrijke variabele in de SDD (Bij gelijke voorkomens wordt naar de tweede belangrijkste variabele gekeken enz.). Met veel andere zaken wordt niet rekening gehouden in deze heuristiek, toch zijn de resultaten interessant zullen we verder zien.

### 5.2.3 Inverse variabelenvolgorde

We herbekijken de tweede techniek voorgesteld bij MINCE (zie sectie 4.2). De techniek legt een volgorde op aan de clausules gebaseerd op de variabele met het laatste voorkomen in de variabelenvolgorde. De uitbreiding van deze techniek naar rooted-DAGs en vtrees gaat als volgt: Doorloop de vtree breedte-eerst om een variabelenvolgorde te bekomen. Apply eerst die SDD's, die een voorkomen hebben van een variabele die het laatste voorkomt in de volgorde. Dit is het omgekeerde van wat we doen in de vorige subsectie 5.2.2. Nu is nog de vraag of we best de vtree van links naar rechts doorlopen of van rechts naar links. De vorige sectie geeft voorkeur aan primes, dus dan zouden we van rechts naar links moeten doorlopen (aangezien het laatste voorkomen telt). De heuristiek voorgesteld in MINCE geeft automatisch prioriteit aan variabelen in de rechtertakken, omdat die bij de OBDD altijd lager liggen in de volgorde (elke linkertak in de vtree bevat slechts één variabele).

**Heuristiek 6.1 (Bottom-up variabelenvolgorde LR)** *Leg een volgorde op aan alle variabelen door de vtree breedte-eerst van links naar rechts te doorlopen. Kies de twee SDD's die een variabele bevatten die als laatste voorkomt in de volgorde.*

**Heuristiek 6.2 (Bottom-up variabelenvolgorde RL)** *Leg een volgorde op aan alle variabelen door de vtree breedte-eerst van rechts naar links te doorlopen. Kies de twee SDD's die een variabele bevatten die als laatste voorkomt in de volgorde.*

Deze twee heuristieken hebben dezelfde tijdscomplexiteit als heuristiek 6. Heuristieken 6, 6.1 en 6.2 resulteren in een incrementele compilatie, zie definitie 7. Dat komt omdat de SDD  $\gamma_i \circ \gamma_j$  met hoge kans de variabelen bevat die in  $\gamma_i$  of in  $\gamma_j$  voorkomen. Als  $\gamma_i$  en  $\gamma_j$  geprioriteerd worden door de heuristiek, zal  $\gamma_i \circ \gamma_j$  ook geprioriteerd worden.

**Definitie 7 (Incrementele compilatie)** *Een compilatiesequentie  $\gamma_1, \dots, \gamma_M$  noemen we incrementeel, als*

1.  $\gamma_{N+1} = \text{Apply}(\gamma_i, \gamma_j, \circ)$ , met  $i, j \leq N$
2.  $\gamma_i = \text{Apply}(\gamma_{i-1}, \gamma_j)$ , met  $j < N + 1 < i$
3. elke  $\gamma_i$  wordt maar één keer gebruikt als parameter van de Apply-operatie.

### 5.3 Probabiliteit variabelenvoorkomens

De heuristieken 6, 6.1 en 6.2 hangen sterk af van de aanwezige variabelen in de SDD's. Als de SDD's alle variabelen bevatten van de probleeminstantie, zullen heuristieken 6, 6.1 en 6.2 resulteren in dezelfde compilatiesequentie. Ook het effect van heuristiek 1 wordt kleiner naarmate er meer variabelen in de SDD's  $\gamma_i$  aanwezig zijn.

We berekenen de kans dat alle variabelen aanwezig zijn in een arbitraire K-CNF over  $\nu$  variabelen en met  $r \times \nu$  clausules. De kans dat één clausule  $c$  van  $K$  variabelen de variabele  $P$  niet bevat kan als volgt berekend worden:

$$\text{Prob}(P \notin \text{vars}(c)) = \frac{\nu-1}{\nu} \times \frac{\nu-2}{\nu-1} \times \dots \times \frac{\nu-K}{\nu-K+1} = \frac{\nu-K}{\nu} \quad (5.30)$$

Met  $\text{vars}(c)$  de verzameling van variabelen  $\mathbf{X}$  waarvoor geldt dat voor elke  $X \in \mathbf{X}$ :  $c \not\models c \mid X$ . De kans dat een K-CNF  $f$  met  $r \times \nu$  clausules de variabele  $P$  niet bevat berekenen we door de verschillende kansen te vermenigvuldigen:

$$\text{Prob}(P \notin \text{vars}(f)) = \left( \frac{\nu-K}{\nu} \right)^{r \times \nu} \quad (5.31)$$

De kans dat een K-CNF  $f$  minstens één voorkomen heeft van de variabele  $P$  is de inverse van 5.31:

$$\text{Prob}(P \in \text{vars}(f)) = 1 - \left( \frac{\nu-K}{\nu} \right)^{r \times \nu} \quad (5.32)$$

De kans dat een K-CNF alle  $\nu$  variabelen bevat is moeilijk exact te berekenen omdat het voorkomen van de ene variabele de kans beïnvloedt van het voorkomen van een andere variabele. We kunnen een schatting maken van de kans door onafhankelijkheid van de voorkomens van variabelen te veronderstellen:

$$\text{Prob}(|\text{vars}(f)| = \nu) \approx \left( 1 - \left( \frac{\nu-K}{\nu} \right)^{r \times \nu} \right)^\nu \quad (5.33)$$

In tabel 5.1 geven we de waarden voor  $\nu = 24$ ,  $K = 3$  en een aantal ratio's. We

| ratio $r$ | $Prob( vars(f)  = \nu)$ |
|-----------|-------------------------|
| 0.25      | 0.5512046813964844      |
| 0.5       | 0.7985827619995689      |
| 0.75      | 0.9096048864993564      |
| 1.0       | 0.9594310962362776      |
| 1.25      | 0.9817928659099648      |
| 1.5       | 0.9918287234552058      |
| 1.75      | 0.9963327693396816      |
| 2.0       | 0.9983541640474098      |

TABEL 5.1: Kans op voorkomen van elke variabele in een 3-CNF  $f$  met  $r \times \nu$  clauses voor  $\nu = 24$ .

merken dat de kans al voor kleine  $r$  naar 1 convergeert. We kunnen dus verwachten dat heuristiek 1 bijna geen effect zal hebben voor  $r > 1$ . De heuristieken gebaseerd op een variabelenvolgorde zullen waarschijnlijk samenvallen voor  $r > 1$ , ze zullen resulteren in dezelfde compilatievolgorde.

## Hoofdstuk 6

# Experimenten

Om onze heuristieken te testen, passen we ze telkens toe tijdens de bottom-up compilatie van een probleeminstantie. Even ter herhaling, een probleeminstantie bestaat uit een aantal elementen. We hebben vergelijking  $n = \bigcirc_{i=1}^N k_i$ . We willen de SDD-compilatie van formule  $n$  berekenen, elke  $k_i$  is reeds gecompileerd naar een SDD.  $\bigcirc$  stelt een logische operatie voor. Voor onze experimenten hebben we een groot aantal probleeminstanties nodig, die genereren we willekeurig. We hebben verschillende parameters voor een probleeminstantie: het aantal  $k_i = N$ , het aantal variabelen in  $n = \nu$  en de operatie  $\bigcirc$  (conjunctie of disjunctie).

Het eerste dat we verklaren is dat we onze heuristieken enkel voor één operatie moeten testen en niet voor zowel de conjunctie als de disjunctie. Wanneer we een SDD genereren vanuit een 3-CNF moeten we rekening houden met de ratio  $r$  van de CNF, omwille van de twee fasetransities gezien in sectie 4.3. Wanneer we van twee CNF's  $f$  en  $g$  de conjunctie  $h = f \wedge g$  nemen, is de ratio van  $h$  de som van de ratio van de CNF's  $r_h = r_f + r_g$ . Als we de conjunctie van  $n$  CNF's met ratio  $r$  nemen, krijgen we een CNF met ratio  $n \times r$ . Als de ratio over de 3-SAT fasetransitie gaat is de CNF met hoge kans triviaal Onwaar, dat willen we vermijden omdat de Apply-operaties dan ook triviaal zijn, zie 2.2. Als we disjunctie nemen tussen twee CNF's hebben we dit probleem niet, enkel als we een groot aantal CNF's (met lage ratio) samennemen komen we mogelijks tot de triviale Waar formule.

Als we niet willen dat de ratio te groot wordt na het uitvoeren van verschillende conjuncties, moeten we werken met DNF's. Als we de conjunctie van DNF's nemen hebben we niet het probleem dat de formule triviaal Onwaar wordt. We weten dat een CNF kan omgezet worden naar een DNF door middel van negatie, de wetten van De Morgan en distributiviteit. De Morgan en distributiviteit behouden gelijkheid tussen formules. Omwille van de canoniciteit van SDD's, zijn genegateerde CNF's voldoende om SDD's te bekomen waarop de conjunctie kan worden uitgevoerd.

We verklaren dat het testen van de disjunctie op CNF's gelijk is aan het testen van de conjunctie op genegateerde CNF's. We genereren verschillende formules  $k_i$  met behulp van CNF's. Ofwel voeren we de disjunctie uit op SDD's  $\gamma_1, \dots, \gamma_N$ , met

| Heuristiek | Beschrijving                         | Afkorting |
|------------|--------------------------------------|-----------|
| 3          | Willekeurig                          | RANDOM    |
| 1 + 4      | Partitionering + Kleinste Eerst      | VP + KE   |
| 1 + 5      | Partitionering + Elementenbovengrens | VP + EL   |
| 6          | Top-down                             | TD        |
| 6.1        | Bottom-up met focus op subs          | BU-LR     |
| 6.2        | Bottom-up met focus op primes        | BU-RL     |

TABEL 6.1: Heuristieken gebruikt in de experimenten; VP + EL wordt ook gemeten zonder overhead en wordt gedenoteerd met VP + EL 2.

$\gamma_i \equiv k_i$  voor  $i \in \{1, \dots, N\}$ . De tweede mogelijkheid is de conjunctie uitvoeren op SDD's  $\delta_1, \dots, \delta_N$ , met  $\delta_i \equiv \neg k_i$  voor  $i \in \{1, \dots, N\}$ . Voor elke  $i \in \{1, \dots, N\}$  geldt dat  $\gamma_i \equiv \neg \delta_i$ . Omwille van eigenschap 2 zijn  $\gamma_i$  en  $\delta_i$  niet alleen even groot, ze hebben exact dezelfde structuur.  $\gamma_i$  en  $\delta_i$  verschillen enkel in de elementen van de meest rechtse vtreeknopen (de vtreeknopen tussen de wortelknoop en de meest rechtse bladknoop). Eender welk mogelijk tussenresultaat bij de disjunctie kunnen we schrijven als volgt:

$$\gamma_J = \bigvee_{j \in J} \gamma_j \equiv \bigvee_{j \in J} k_j, \text{ met } J \subseteq \{1, \dots, N\} \quad (6.1)$$

Op dezelfde manier kunnen we een mogelijk tussenresultaat voor de conjunctie beschrijven:

$$\delta_J = \bigwedge_{j \in J} \gamma_j \equiv \bigwedge_{j \in J} \neg k_j, \text{ met } J \subseteq \{1, \dots, N\} \quad (6.2)$$

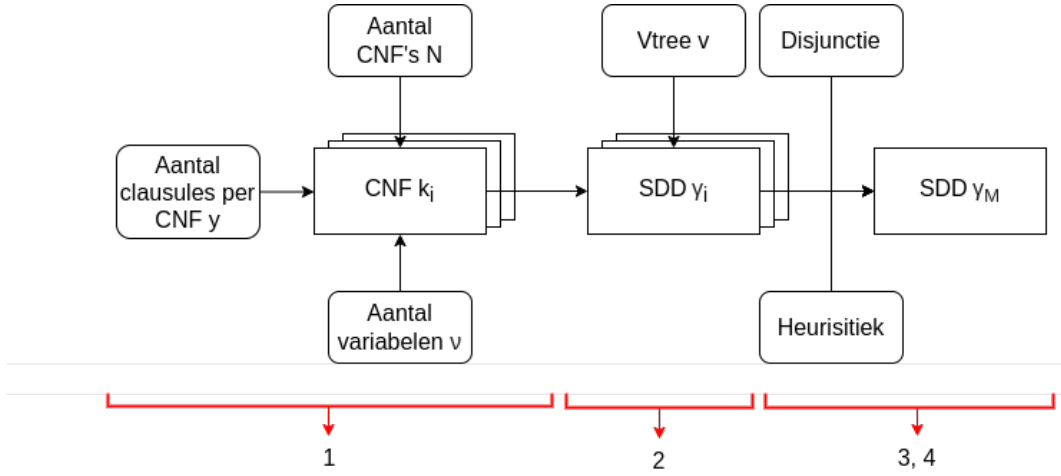
Volgens de wetten van de Morgan (tabel 2.2) mogen we de negatie naar buiten brengen en de conjunctie veranderen naar een disjunctie bij  $\delta_J$ :

$$\delta_J \equiv \neg \bigvee_{j \in J} k_j = \neg \gamma_J, \text{ met } J \subseteq \{1, \dots, N\} \quad (6.3)$$

We zien dat elk mogelijk tussenresultaat  $\delta_J = \neg \gamma_J$ , omwille van eigenschap 2 zal elk tussenresultaat exact dezelfde structuur hebben los van de operatie.

Het gevolg hiervan is dat onze heuristieken exact dezelfde Apply-volgorde zullen bepalen voor een bepaalde lijst van CNF's  $k_i$ , ongeacht de operatie. We voeren onze experimenten daarom enkel uit met als operatie de disjunctie.

De heuristieken die we besproken hebben zijn niet altijd logisch om te testen. Heuristiek 2 werkt enkel op CNF's, terwijl we heuristieken onderzoeken voor algemene rooted-DAG's. Heuristiek 1 willen we enkel testen in combinatie met een interne heuristiek. Voor we de analyse van de experimenten beginnen, geven we voor elke (combinatie van) heuristiek(en) die we testen een logische afkorting in tabel 6.1.



FIGUUR 6.1: Stappenplan voor experimenten.

## 6.1 Werkwijze

We geven de opstelling van één experiment, dit experiment voeren we meerdere malen uit om een algemeen beeld te verkrijgen van de efficiëntie van de heuristieken.

- 1 We genereren eerst  $N$  verschillende 3-CNF's, met  $y$  clausules, die stellen de verschillende  $k_i$  voor.
- 2 We compileren elke formule  $k_i$  naar een invoer-SDD  $\gamma_i$  met een vooraf bepaalde vtree  $v$ .
- 3 De disjunctie  $\gamma_M = \bigvee_{i=1}^N \gamma_i$  wordt uitgevoerd volgens een bepaalde heuristiek.
- 4 De totale compilatietijd  $ct$  en de groottes van de tussenresultaten worden gemeten.

Een diagram van dit stappenplan is te zien in figuur 6.1. Voor ons experiment zijn er naast de eerder vermelde parameters  $N$  en  $\nu$  nog 2 extra parameters, het aantal clausules per 3-CNF  $y$  en de vtree  $v$ .

In dit experiment worden alle SDD's  $\gamma_i, i \in 1, 2, \dots, N$  (invoer-SDD's genoemd) gecompileerd met behulp van 3-CNF's met eenzelfde ratio  $r$ . Alle invoer-SDD's zullen sterk gelijkaardig zijn qua structuur. In de praktijk is dat niet gegarandeerd, toch kunnen we enkele conclusies trekken over de werking van de verschillende heuristieken. In sectie 6.7 voeren we experimenten uit in een meer algemene setting, met SDD's gegenereerd vanuit 3-CNF's met een willekeurig aantal clausules.

We bespreken eerst alle parameters: waarvoor ze dienen, welk effect ze hebben en hoe ze van elkaar afhankelijk zijn.

- Het aantal variabelen  $\nu$  heeft een invloed op de grootte van de SDD's. We willen  $\nu$  niet te groot kiezen zodat de SDD's nog steeds in het computergeheugen

passen, maar ook niet te klein omdat de compilatietijden dan klein zijn en meer afhankelijk van externe factoren.

- De keuze van het aantal clausules per 3-CNF  $y$  is sterk gelinkt aan onze keuze van  $\nu$ . Zoals gezien in sectie 4.3 moeten we opletten op de ratio  $r = y/\nu$ . Als de ratio over de drempelwaarde  $r^*$  gaat, is er een grote kans dat de CNF niet voldaan kan worden. Dat willen we niet want een CNF die niet voldaan kan worden komt overeen met de logische formule  $f = \perp$ . Voor 3-CNF wordt de drempelwaarde geschat op  $r^* = 4,2$  [CA96b]. Naast de drempelwaarde voor onvoldaanbaarheid, is de ratio ook gelinkt aan de grootte van de compilatie. In zekere mate kunnen we de ratio beschouwen als parameter voor de complexiteit van een SDD: hoe groter een SDD, hoe complexer het is.
- Nog een belangrijke parameter is het aantal invoerSDD's  $N$ . Als  $N$  klein is bijvoorbeeld zijn er een klein aantal mogelijke sequenties voor een bottom-up compilatie en zullen de metrieken  $ct$  en  $g$  kleinere verschillen noteren tussen de heuristieken. De tijdscomplexiteit van de heuristieken is ook in functie van  $N$ . Zo is de tijdscomplexiteit van heuristiek 5 log-kwadratisch in  $N$ , terwijl die lineairitmisch of lineair is voor de andere heuristieken.
- Als laatste parameter hebben we de vtree  $v$ . De grootte van een SDD is, naast het aantal variabelen  $\nu$ , sterk afhankelijk van de vtree. De manier waarop we 3-CNF's genereren maakt dat elke variabele dezelfde kans op voorkomen heeft. We kunnen stellen dat elke variabele geacht wordt dezelfde invloed te hebben op de formule. Experimenteel stelden we vast dat we voor gebalanceerde en rechtslineaire vtrees meer variabelen kunnen toelaten voor we geheugen te kort komen. Vermoedelijk zullen we de meest representatieve resultaten krijgen door die twee types vtrees te gebruiken. Ook linkslineaire vtrees testen we, maar voor een lager aantal variabelen en op minder probleeminstanties.

In een gebalanceerde vtree ligt elke variabele, voor zover mogelijk, op dezelfde diepte. Hierdoor zullen heuristieken 6 en 6.2 zo goed als samenvallen. Voor rechts- en linkslineaire heuristieken verschillen de variabelenvolgordes bepaald door 6.1 en 6.2 enkel in de twee diepste variabelen. Deze heuristieken zullen dus ook zeer gelijkaardige resultaten vertonen voor die type vtrees. Omwille hiervan maar ook omdat in de praktijk verschillende types vtrees (zullen) worden gebruikt, zijn de experimenten met verschillende type vtrees belangrijk.

De volgorde van de variabelen in de vtree speelt ook een rol. Wij kiezen ervoor om tussen alle experimenten telkens dezelfde willekeurige volgorde aan te houden.

Met al deze overwegingen in de gedachten, kiezen we  $N = 20$ , de ratio  $r$  laten we verschillende waarden in het interval  $[0.5, 4.5]$  aannemen. Concreet doen we dat door  $\nu$  constant te houden en  $y = r * \nu$  te kiezen. Voor gebalanceerde en rechtslineaire vtrees kiezen we  $\nu = 24$ . Voor linkslineaire vtrees kiezen we  $\nu = 10$ .

## 6.2 SDD- en PySDD-bibliotheek

Alle experimenten in dit werk zijn uitgevoerd met behulp van de PySDD-bibliotheek [Mee23], een Pythonwrapper voor de SDD-bibliotheek geschreven door A. Darwiche in C [Dar11]. De bibliotheek biedt een interface voor het creëren, manipuleren en evalueren van SDDs. Meer informatie en documentatie is beschikbaar op de officiële GitHub pagina [Mee23]. Voor deze implementatie werd een specifieke fork gebruikt gemaakt door V. Derkinderen. Deze fork bevat een extra functie `local_vtree_element_count()`, die gebruikt wordt om het aantal elementen per vtreesknoop in een SDD te berekenen.

De tijdscomplexiteit van de heuristieken is afhankelijk van deze bibliotheek. De complexiteit van heuristiek 4 is eigenlijk  $O(\max_{\gamma_i}(|\gamma_i|) \times N \times \log(N))$ , maar in de bibliotheek wordt de grootte van SDD's tijdens de uitvoering automatisch bijgehouden. Het opvragen van de grootte kan in constante tijd, waardoor de complexiteit  $O(N \times \log(N))$  wordt.

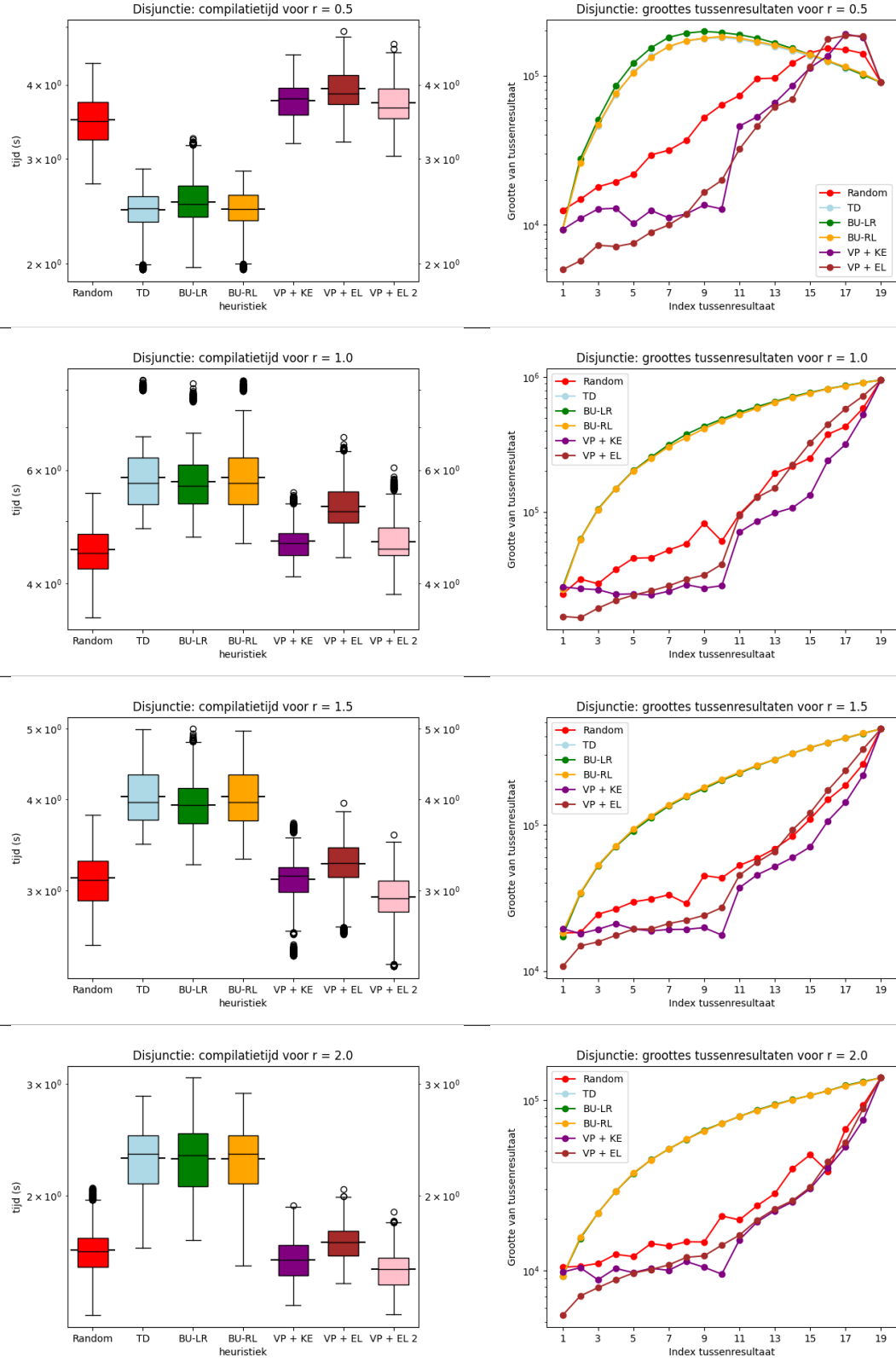
## 6.3 Resultaten gebalanceerde vtree

We voeren ons experiment telkens 2160 keer uit met als parameters  $\nu = 24, r \in \{0.5, 1, 1.5, \dots, 4.5\}$  en vtree  $v =$  gebalanceerd. Van elk experiment meten we de compilatietijd  $ct$  per heuristiek. We visualiseren deze resultaten met behulp van boxplots. Naast de boxplot zelf plotten we ook de gemiddelde compilatietijd, aangegeven door de zwarte strepen achter elke boxplot. Voor heuristiek 5 (VP + EL) geven we ook de compilatietijd gemeten zonder overhead (VP + EL 2). Dat doen we omdat de overhead van deze heuristiek dominant is, maar we verwachten dat die nog te verkleinen is met de juiste programmeertechnieken. Door ook de compilatietijden zonder overhead te plotten tonen we het mogelijke potentieel van deze heuristiek. Van elk experiment meten we ook de groottes van de tussenresultaten per heuristiek. Om de groottes te visualiseren nemen we het gemiddelde van elk experiment per tussenresultaat per heuristiek. De resultaten zijn te vinden in de tabellen 6.2, 6.3 en 6.4.

Wat ons meteen opvalt, is dat de drie heuristieken gebaseerd op een variabelenvolgorde gemiddeld resulteren in een tweemaal kleinere compilatietijd voor  $r = 0.5$ . Voor grotere  $r$  zijn deze heuristieken dan weer trager dan de bovengrens heuristieken. Een onderscheid maken tussen deze drie heuristieken is met deze resultaten niet mogelijk, omdat ze zo goed als samenvallen. Dat zien we niet alleen in de compilatietijden maar ook in de groottes van de tussenresultaten, die gemiddeld bijna gelijk zijn.

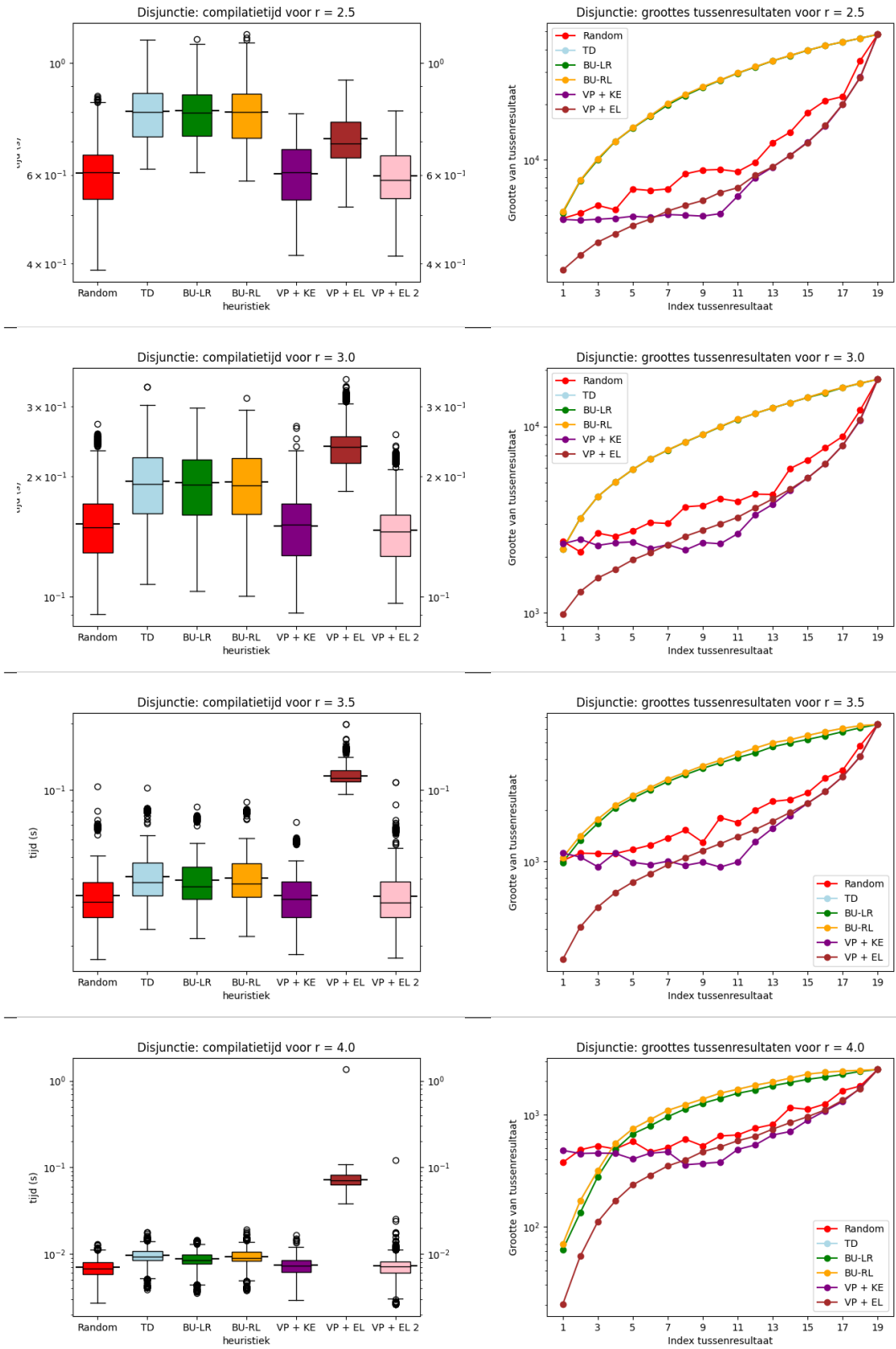
Het volgende dat ons opvalt, is dat de willekeurige heuristiek het onverwacht goed doet en voor sommige ratio's zelfs de snelste gemiddelde compilatietijd oplevert ( $r = 1$ ). Dat valt tot op zekere hoogte te verklaren. In dit experiment heeft elke 3-CNF dezelfde ratio, we kunnen stellen dat de invoerSDD's  $\gamma_i, i \in 1, \dots, N$  van gelijke complexiteit zijn. Hoe meer invoerSDD's aan een tussenresultaat  $\gamma_i, i > N$  gevoegd worden, hoe groter  $\gamma_i$  zal worden, dat kunnen we afleiden uit de groottes van de tussenresultaten

### 6.3. Resultaten gebalanceerde vtree

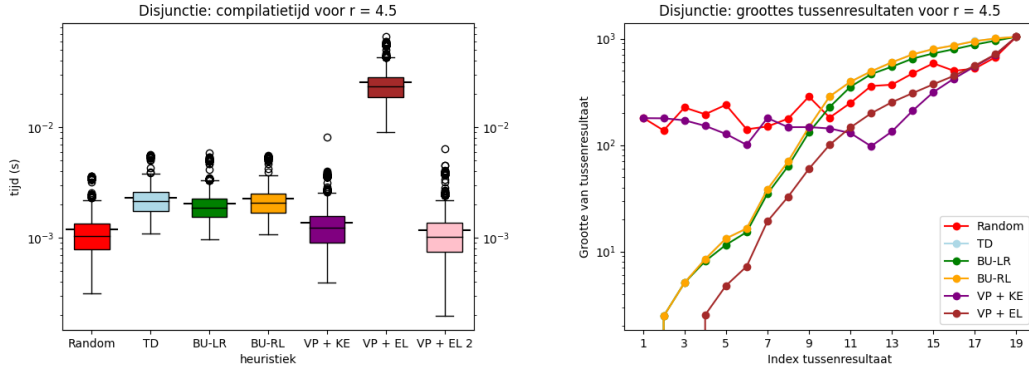


TABEL 6.2: Resultaten met gebalanceerde vtree en  $r = 0.5, 1, 1.5, 2$ .

### 6.3. Resultaten gebalanceerde vtree



TABEL 6.3: Resultaten met gebalanceerde vtree en  $r = 2.5, 3, 3.5, 4$ .

TABEL 6.4: Resultaten met gebalanceerde vtree en  $r = 4.5$ .

van de incrementele heuristieken (TD, BU-LR en BU-RL). Een willekeurige heuristiek zal gemiddeld gezien niet incrementeel compileren, daarom blijven de tussenresultaten relatief klein. De willekeurige heuristiek kiest de compilatievolgorde willekeurig, hierdoor bekommt die soms een goede compilatievolgorde en soms een slechte. Gelet op de vorige opmerking dat gemiddeld een relatief goede compilatievolgorde wordt gekozen omwille van de groottes verklaart dit enigszins de goede resultaten. De andere heuristieken werken volgens een bepaald stramien, een inefficiëntie van de heuristiek zal zich tonen in elk resultaat.

Kleine tussenresultaten zijn echter niet de volledige verklaring, want anders zou heuristiek 5.1.1 het snelste moeten zijn (zonder overhead erbij te rekenen, zie VP + EL 2). Ondanks dat VP + EL voor alle ratio's gemiddeld in de kleinste tussenresultaten resulteert, zijn de compilatietijden niet duidelijk lager.

Bij de grote ratio's  $r \geq 3$  zien we dat de overhead bij heuristiek 5.1.1 een grote invloed heeft op de totale compilatietijd. Als we overhead buiten beschouwing laten zien we dat deze heuristiek zeker niet trager is dan Random en VP + KE. Het kan daarom nuttig zijn om in de toekomst de SDD-bibliotheek aan te passen zodat het aantal elementen per vtreenoep in het programma worden opgeslagen, waardoor de overheadkost van  $O(\max_{\gamma_i}(|\gamma_i|) \times N + (\nu + \log(N^2)) \times N^2)$  naar  $O((\nu + \log(N^2)) \times N^2)$  kan gereduceerd worden.

We concluderen dat de willekeurige heuristiek samen met de twee heuristieken gebaseerd op een bovengrens ongeveer even goed presteren voor  $r > 0.5$ . Voor  $r \leq 0.5$  zijn de variabelenvolgorde heuristieken het snelste.

Een laatste opmerking bij deze resultaten zijn de compilatietijden voor de verschillende ratio's. Voor  $r = 1$  ligt de gemiddelde compilatietijd rond de 5 seconden, terwijl voor  $r = 2.5$  dat al verlaagt naar 5 deciseconden en voor  $r = 4$  de compilatietijden onder de centiseconde liggen. Voor  $\nu = 24$  is een heuristiek die goed werkt voor lage ratio's veel belangrijker dan een heuristiek voor hoge ratio's.

## 6.4 Resultaten rechtslineaire vtrees

Als volgende bekijken we de resultaten voor een rechtslineaire vtree. We gebruiken als parameter  $\nu = 24$  en we doen 2160 iteraties van het experiment. De resultaten geven we op dezelfde manier weer als bij de gebalanceerde vtrees in tabellen 6.5, 6.6 en 6.7.

Het eerste dat we kunnen opmerken is dat de drie heuristieken op basis van een variabelenvolgorde in alle gevallen twee keer trager zijn dan de snelste compilatietijd. Ook voor  $r = 0.5$  zien we dat, terwijl voor gebalanceerde vtrees en deze ratio dit type heuristieken juist sneller is. We merken ook een verschil tussen het gebruik van BU-LR en BU-RL, BU-LR leidt consistent tot grotere uitschieters.

We stellen vast dat de compilatietijden in het algemeen lager liggen dan voor de gebalanceerde vtrees, maar dat de groottes van de tussenresultaten wel ongeveer gelijk zijn. Het gevolg hiervan is dat de overhead voor VP + EL een groter effect heeft. VP + EL heeft wel potentieel, dat zien we als we de overhead buiten beschouwing laten (gegeven in VP + EL 2).

Waar bij de gebalanceerde vtrees VP + KE het ongeveer evengoed doet als VP + EL 2, zien we nu ook een onderscheid tussen deze twee. We kunnen hieruit concluderen dat VP + EL de structuur van de vtree beter in acht neemt om de compilatievolgorde te bepalen.

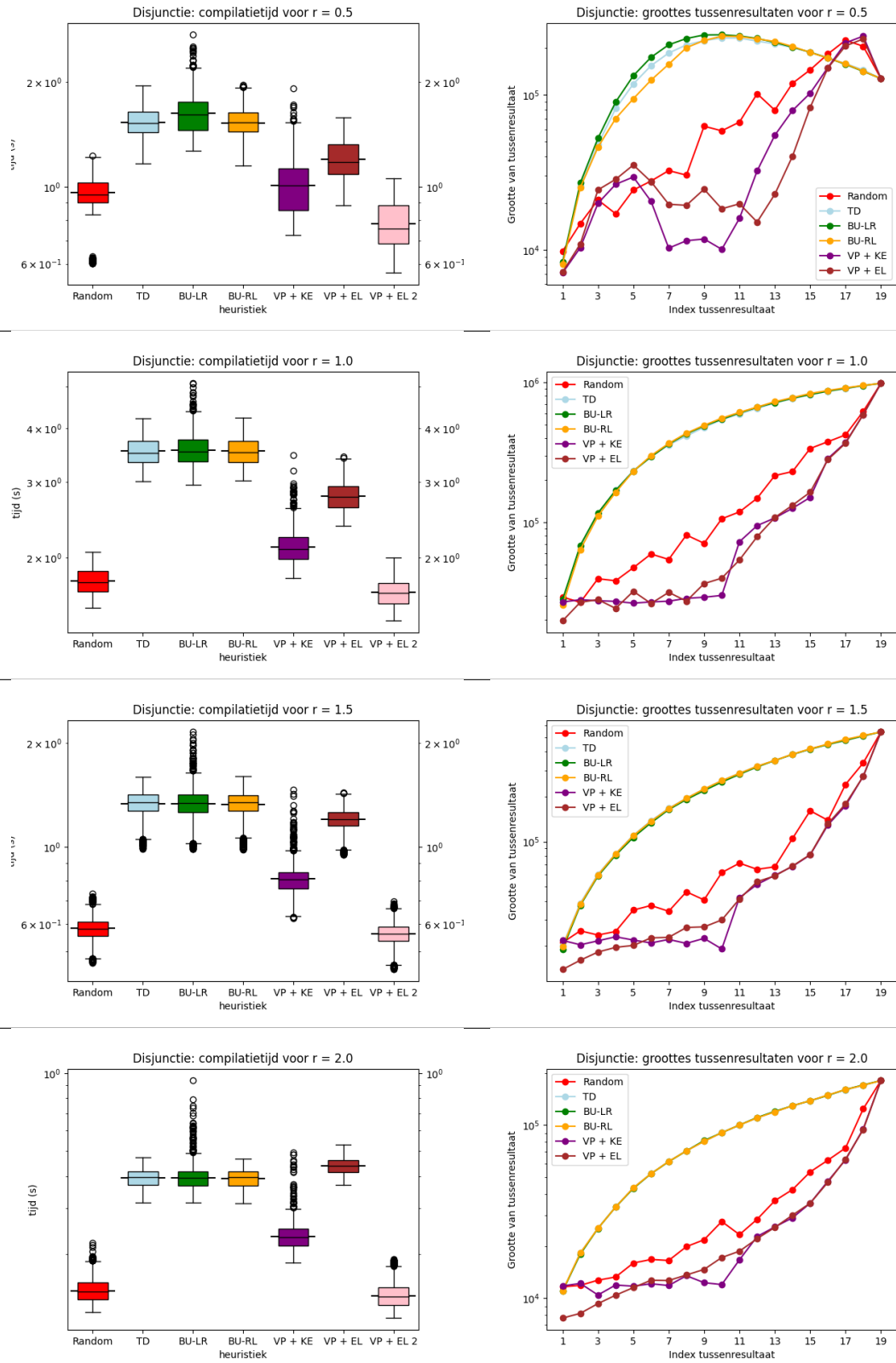
Net zoals bij de gebalanceerde vtrees doet de willekeurige heuristiek het bijzonder goed, de verklaring hiervoor is dezelfde als in de vorige sectie.

## 6.5 Resultaten linkslineaire vtrees

In deze sectie geven we de resultaten van linkslineaire vtrees. Voor linkslineaire vtrees kiezen we  $\nu = 10$  en doen we 720 iteraties. Het aantal variabelen is beduidend lager als voor de andere vtrees. Een grotere  $\nu$  leidde tot geheugentekortfouten tijdens de compilatie. De compilatietijden liggen in dezelfde grootteorde als bij de vorige vtrees, maar de groottes van de resultaten zijn beduidend kleiner. Ook het aantal iteraties ligt lager voor dit type vtree. Dat heeft twee redenen, ten eerste is het aantal mogelijke probleeminstanties kleiner dan wanneer  $\nu = 24$ , daarom is het verantwoord het aantal iteraties kleiner te nemen. Ten tweede zijn de resultaten minder representatief wegens het kleine aantal variabelen  $\nu$ , we hebben ervoor gekozen meer rekenkracht te voorzien voor andere experimenten. De resultaten zijn terug te vinden in tabellen 6.8, 6.9 en 6.10.

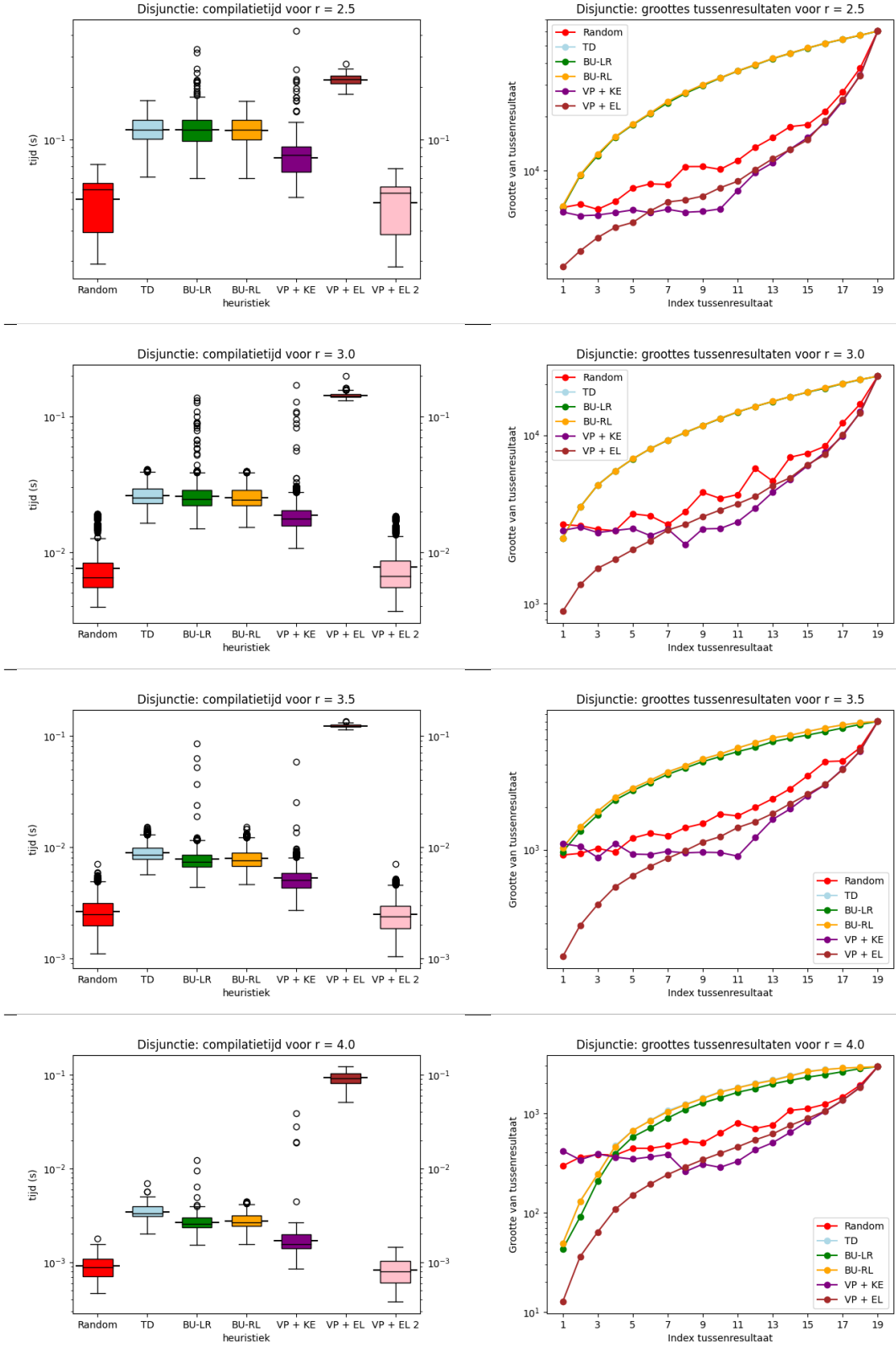
Over al onze experimenten heen kiezen we het aantal SDD's dat Applied wordt gelijk,  $N = 20$ . Omwille van het kleine aantal variabelen leidt het samennemen van 20 onderbepaalde SDD's (gegenereerd vanuit kleine ratio) tot zeer onderbepaalde SDD's die richting de triviale SDD  $\top$  gaan. Dit is voornamelijk te zien voor  $r = 0.5, 1, 1.5$ , waar de gemiddelde groottes van de tussenresultaten afnemen naar het einde toe. In dit geval is de incrementele compilatie van de variabelenvolgorde heuristieken

## 6.5. Resultaten linkslinaire vtrees

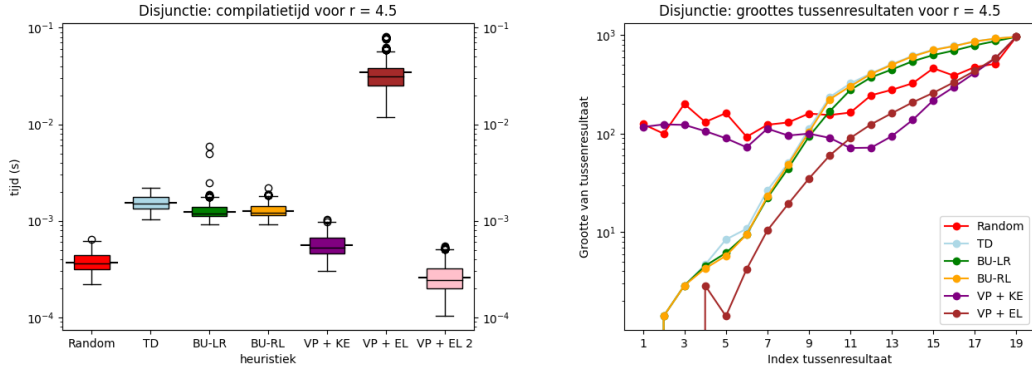


TABEL 6.5: Resultaten met rechtslineaire vtree en  $r = 0.5, 1, 1.5, 2$ .

## 6.5. Resultaten linkslinaire vtrees



TABEL 6.6: Resultaten met rechtslineaire vtrees en  $r = 2.5, 3, 3.5, 4$ .

TABEL 6.7: Resultaten met rechtslineaire vtree en  $r = 4.5$ .

voordelig, omdat de tussenresultaten gelijkmatig kleiner worden. Voor grotere  $r$  valt dit effect weg en resulteren de heuristieken in ongeveer gelijke compilatietijden. Omwille van de kleinere groottes van de tussenresultaten is de overhead voor VP + EL relatief klein.

## 6.6 Combinatie van heuristieken

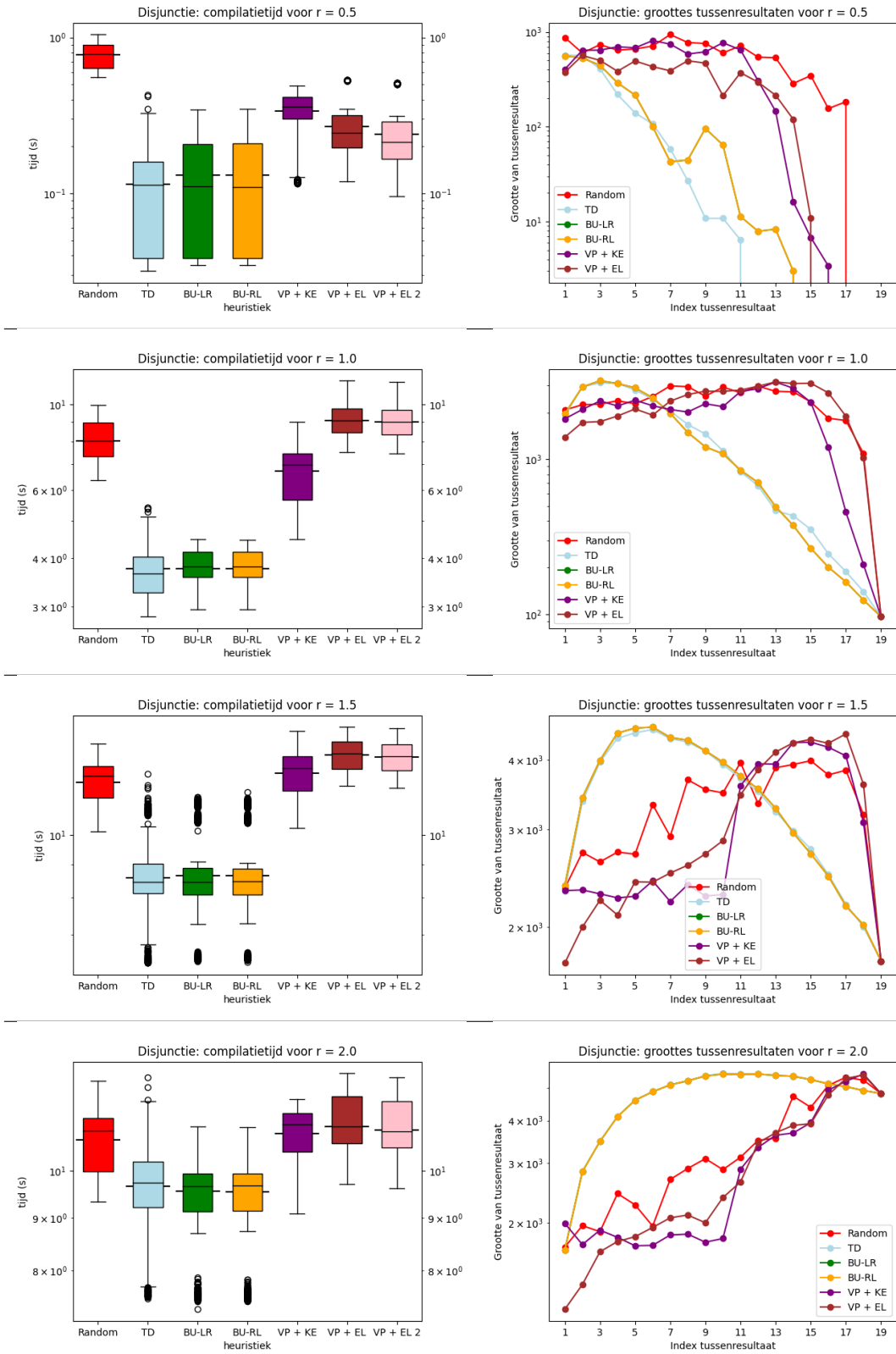
We merken uit de resultaten van secties 6.3, 6.4 en 6.5 dat er in bepaalde gevallen een heuristiek op basis van variabelenvolgorde te verkiezen is en voor andere gevallen een heuristiek op basis van bovengrens. De twee types heuristieken vullen elkaar wel aan, voor elke probleeminstantie is er een heuristiek die een goede compilatievolgorde vindt.

Idealiter willen we een heuristiek die op basis van de SDD's ofwel een variabelenvolgorde heuristiek toepast ofwel een bovengrens heuristiek. De vraag die we ons dan moeten stellen is op basis van wat we beslissen welke heuristiek verkiesbaar is.

Het is niet duidelijk wat precies maakt dat een variabelenvolgorde heuristiek te verkiezen is boven een bovengrens heuristiek. Als de grootte van het eindresultaat niet het grootste tussenresultaat is, zien we dat een incrementele compilatie heel goed is (gebalanceerde vtree en  $r = 0.5$ , linkslineaire vtree en  $r = 0.5, 1, 1.5, 2$ ). Enkel voor rechtslineaire vtree met  $r = 0.5$  zien we dat niet terugkomen. Bij rechtslineaire vtrees doen TD, BU-LR en BU-RL het over de hele lijn slechter, vermoedelijk is er iets anders aan de hand waardoor variabelenvolgorde heuristieken niet goed werken voor dit type vtrees.

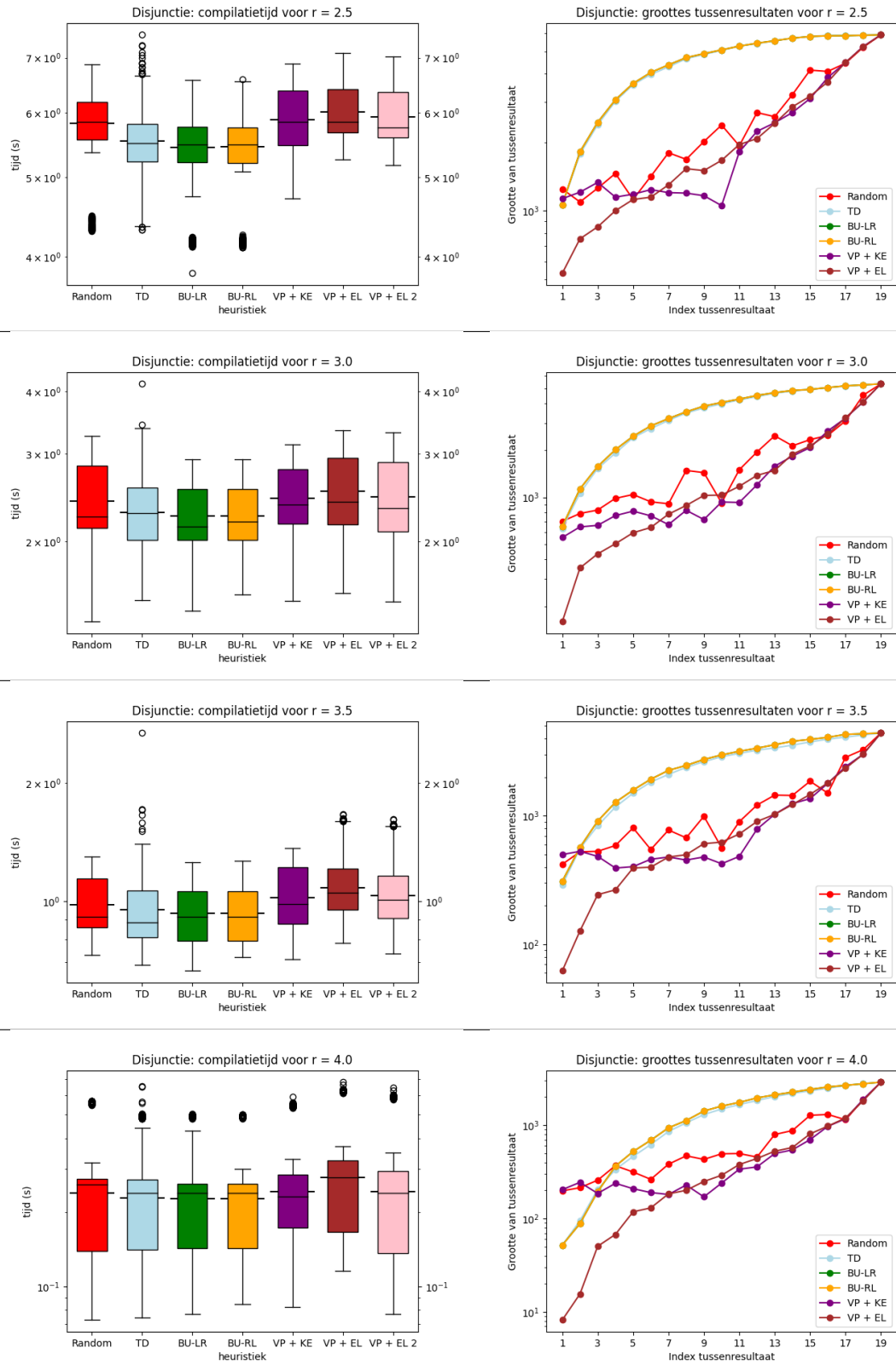
We weten op voorhand niet wat de grootte van het eindresultaat zal zijn. Wat wel mogelijk is de grootte van de invoerSDD's gebruiken als parameter. We kunnen een grenswaarde voor de grootte bepalen in functie van het aantal variabelen  $\nu$  in de probleeminstantie. Zolang SDD's onder die grenswaarde liggen gebruiken we een variabelenvolgorde heuristiek, als ze groter zijn gebruiken we een heuristiek op basis van bovengrens. Het idee hierachter is dat als de grootte onder de grenswaarde

## 6.6. Combinatie van heuristieken

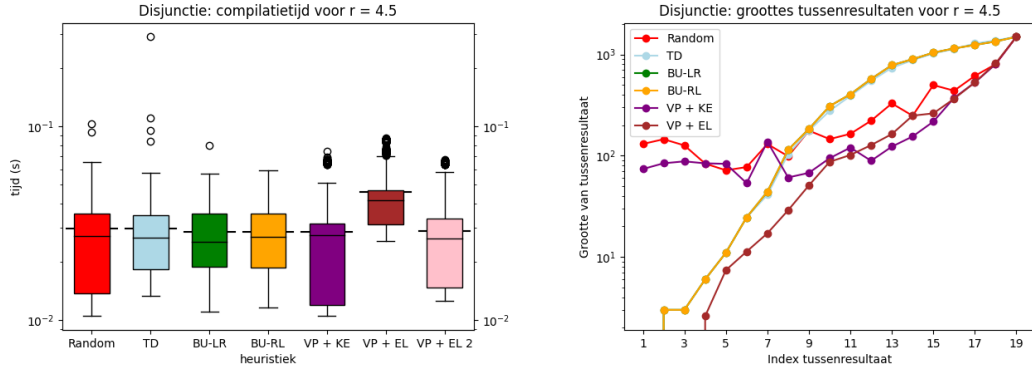


TABEL 6.8: Resultaten met linkse lineaire vtree en  $r = 0.5, 1, 1.5, 2$ .

## 6.6. Combinatie van heuristieken



TABEL 6.9: Resultaten met linkslineaire vtree en  $r = 2.5, 3, 3.5, 4$ .

TABEL 6.10: Resultaten met linkslineaire vtree en  $r = 4.5$ .

ligt, we een SDD hebben overeenkomstig met een 3-CNF met lage of hoge ratio  $r$ . Experimenteel hebben we gemerkt dat voor lage ratio's de variabelenvolgorde heuristieken goed werken, tenzij voor rechtslineaire vtrees. Voor hogere  $r$  ligt algemeen de compilatietijd veel lager en is het dus minder belangrijk om een optimale heuristiek te hebben. We leggen dus een link tussen SDD's en de grootte van de SDD's indirect via hypothetische 3-CNF's

Dit idee is in de praktijk helaas moeilijk uit te werken omdat de grootte van een SDD sterk afhangt van de vtree. Het is onduidelijk hoe we de grenswaarde moeten bepalen aan de hand van de vtree en het aantal variabelen.

Een tweede mogelijkheid die we hebben is gebaseerd op het aantal variabelen. Voor lage ratio's zijn waarschijnlijk niet alle variabelen aanwezig in de SDD (zie tabel 5.1). We kunnen op voorhand kijken hoeveel verschillende variabelen aanwezig zijn in de verschillende formules  $k_i$  van een probleeminstantie, dit duiden we aan met  $\nu$ . Zolang er twee of meer SDD's in de lijst zijn met het aantal variabelen lager dan  $\phi \times \nu$ , gebruiken we heuristiek 6.2 (BU-RL) voor die SDD's, zodra dat niet meer het geval is gebruiken we heuristiek 5 EL. De waarde  $\phi = 0.7$  resulteerde experimenteel in de laagste compilatietijden.

**Heuristiek 7 (Combinatie BU-RL en EL)** *Splits de SDD's op in twee verzamelingen BU en EL, BU bevat de SDD's met minder dan  $0.7 \times \nu$  variabelen en EL de rest.*

- $|BU| > 2$ : kies twee SDD's op basis van heuristiek 6.2 uit BU
- Anders: kies twee SDD's op basis van heuristiek 5 uit alle SDD's

De tijdscomplexiteit van deze heuristiek wordt gedomineerd door de complexiteit van heuristiek 5 en is daardoor  $O(\max_{\gamma_i}(|\gamma_i|) \times N + (\nu + \log(N^2)) \times N^2)$ . Net zoals we de mogelijke effectiviteit van heuristiek 5 meten door de overhead te negeren zullen we dat ook doen voor deze heuristiek.

## 6.7 Resultaten willekeurig aantal clausules

De experimenten die we tot nu toe hebben gedaan waren telkens met probleeminstanties gegenereerd met een vaste ratio  $r$ , elke 3-CNF  $k_i$  bevat hetzelfde aantal clausules. In de praktijk zijn de SDD's die Applied moeten worden van verschillende groottes en complexiteiten, daarom doen we een tweede experiment. In dit tweede experiment halen we het aantal clausules uit het stappenplan beschreven in figuur 6.1. Elke 3-CNF  $k_i$  wordt gegenereerd met een willekeurig aantal clausules  $y$ ,  $y \in \{1, 2, \dots, 4.5 * \nu\}$ . We voeren dit experiment uit met gebalanceerde vtree en  $\nu = 28$ , met rechtslineaire vtree en  $\nu = 28$ , met linkslineaire vtrees en  $\nu = 10$  en met willekeurige vtrees en  $\nu = 16$ . Telkens kiezen we  $N = 20$ . Voor elke probleeminstantie genereren we 67 willekeurige compilatiesequenties en de compilatiesequenties volgens de heuristieken gegeven in tabel 6.1. Heuristiek 7, aangeduid met (BU + EL) of (BU + EL 2) indien overhead wordt genegeerd testen we ook met dit experiment.

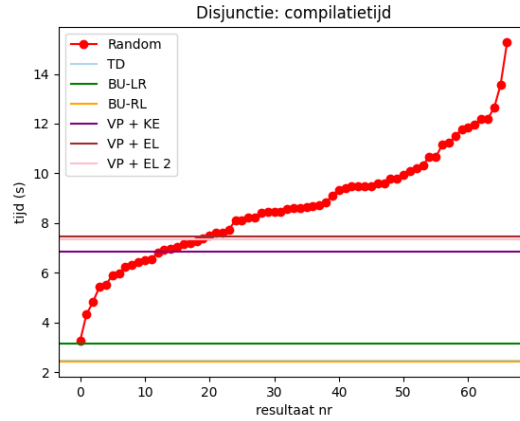
Het stappenplan van dit tweede experiment is als volgt:

- 1 We genereren eerst  $N$  verschillende 3-CNF's, met een willekeurig aantal clausules  $y$ ,  $y \in \{1, 2, \dots, 4.5 * \nu\}$ , die stellen de verschillende  $k_i$  voor.
- 2 We compileren elke formule  $k_i$  naar een SDD  $\gamma_i$  met een vooraf bepaalde vtree  $v$ .
- 3 De disjunctie  $\gamma_M = \bigvee_{i=1}^N \gamma_i$  wordt uitgevoerd volgens een bepaalde heuristiek en 67 keer met een willekeurige Apply-volgorde.
- 4 De totale compilatietijd  $ct$  wordt gemeten voor elke compilatiesequentie.

Omwillen van het willekeurige aantal clausules lopen de compilatietijden sterk uiteen voor verschillende probleeminstanties, het is daarom niet representatief om gemiddeldes te nemen over de verschillende probleeminstanties. Dit gemiddelde is vooral handig om te kijken hoe een heuristiek gemiddeld presteert ten opzichte van een willekeurige volgorde. Als oplossing genereren we voor elke probleeminstantie een aantal willekeurige compilatiesequenties en doen we de compilatie volgens de heuristieken die we willen testen. Nu kunnen we kijken hoeveel willekeurige compilatiesequenties beter zijn dan een compilatiesequentie volgens een heuristiek, we kunnen zo een relatieve score bepalen. Hoe lager de score is van een heuristiek, hoe beter. Het nadeel van deze werkwijze is dat we de effectief gemeten tijdsverschillen niet gebruiken en daardoor niets kunnen zeggen over de verhoudingen van de heuristieken in termen van tijdseenheden.

De grafiek in figuur 3.2 werd gegenereerd met behulp van een probleeminstantie uit dit experiment, met gebalanceerde vtree,  $N = 20$  en  $\nu = 28$ . Als we de compilatietijden van de heuristieken terug toevoegen krijgen we figuur 6.2.

Voor deze specifieke probleeminstantie zijn heuristieken TD, BU-LR en BU-RL sneller dan alle willekeurige compilaties. Er zijn 13 willekeurige compilaties sneller dan de compilatie als gevolg van heuristiek VP + KE. Voor VP + EL zijn er 20 willekeurige compilaties sneller en voor VP + EL 2 (gemeten zonder overhead) 19.



FIGUUR 6.2: Variatie van willekeurige compilatiesequenties samen met de compilatietijden van heuristieken voor een probleeminstantie met gebalanceerde vtree.

| heuristiek | gebalanceerd | rechtslineair | linkslineair | willekeurig  |
|------------|--------------|---------------|--------------|--------------|
| TD         | 14.44        | 55.48         | <b>5.78</b>  | 37.57        |
| BU-LR      | 15.26        | 50.27         | 26.22        | 36.1         |
| BU-RL      | <b>13.68</b> | 49.61         | 26.24        | 20.35        |
| VP + KE    | 33.3         | 41.48         | 8.24         | 35.3         |
| VP + EL    | 30.97        | 60.18         | 14.58        | 53.2         |
| VP + EL 2  | 27.98        | <b>12.97</b>  | 13.89        | 27.47        |
| BU + EL    | 28.01        | 61.9          | 12.67        | 51.99        |
| BU + EL 2  | 24.99        | 27.94         | 11.92        | <b>18.23</b> |

TABEL 6.11: Gemiddelde scores voor experimenten met willekeurig aantal clausules, de beste scores per vtreetype zijn vetgedrukt.

De scores voor deze probleeminstantie schrijven we als volgt:  $[0, 0, 0, 13, 20, 19]$ . Deze scores zijn in dezelfde volgorde als de legende in figuur 6.2.

We voeren het experiment 100 keer uit voor elke combinatie van parameters. De scores voor alle probleeminstanties geven we in appendix B, de gemiddelde scores geven we in tabel 6.11.

Voor gebalanceerde vtrees doen de variabelenvolgorde heuristieken het algemeen het beste, met een klein voordeel voor BU-RL. Voor rechtslineaire vtrees zien we dat VP + EL veel potentieel heeft indien de overhead gereduceerd kan worden. Voor linkslineaire vtrees is TD het snelste, in andere resultaten hebben we dat nog niet zien terugkomen. In linkslineaire vtrees wordt een vtreeknoop  $v$  zo opgesplitst dat  $v^r$  een variabele bevat en  $v^l$  de rest. Heuristiek TD zal in dit geval focussen op de subs hoog in de vtree. We kunnen stellen dat een focus op subs hoog in de vtree belangrijker is dan een focus op primes laag in de vtree. Voor willekeurige vtrees presteert de combinatie van heuristieken zonder overhead (BU + EL 2) het beste, met als tweede BU-RL. Dat bevestigt nogmaals het mogelijke potentieel van heuristiek 5. Wat interessant is, is de efficiëntie van BU-RL in vergelijking met BU-LR en TD. Voor

vaste types vtrees zien we weinig verschil tussen de verschillende variabelenvolgorde heuristieken, enkel voor linkslinaire vtrees is TD duidelijk beter. Willekeurige vtrees zijn elk experiment anders, toch zien we een duidelijk voordeel als we de focus te leggen op primes in plaats van op subs.

Heuristiek 7 (BU + EL) is een goed compromis tussen de twee heuristieken BU-RL en VP + EL. Voor linkslinaire en willekeurige vtrees bekomen we zelfs een gemiddeld snellere compilatietijd dan voor BU-RL of VP + EL apart. Toch vermoeden we dat er betere manieren zijn om de twee heuristieken te combineren.

## Hoofdstuk 7

# Besluit

In dit werk hebben we heuristieken onderzocht voor het bepalen van een efficiënte compilatiesequentie tijdens de bottom-up compilatie van SDD's. We identificeren twee types heuristieken, gebaseerd op variabelenvolgordes en gebaseerd op bovengrenzen na Apply. Beide types zijn ondersteund door een theoretische analyse van SDD's en de Apply-operatie.

Variabelenvolgorde heuristieken leggen een volgorde op aan de variabelen op basis van hoe ze voorkomen in de vtree. Daarna wordt gekeken naar welke SDD's welke variabelen bevatten om een compilatievolgorde te bepalen.

Bovengrens heuristieken proberen de grootte na Apply te schatten. Heuristiek 5 houdt hierbij ook rekening met het verschil tussen primes en subs en de variabelen aanwezig in de SDD's om de schatting bij te stellen.

Onze empirische analyse heeft aangetoond dat

1. Variabelenvolgorde heuristieken: Een goede compilatievolgorde bepalen in bepaalde probleeminstanties waarbij onder andere de invoerSDD's van lage complexiteit zijn en minder verschillende variabelen bevatten.
2. Bovengrens heuristieken: Bij probleeminstanties waarvoor de variabelenvolgorde heuristieken minder presteren, resulteren deze heuristieken in een snelle compilatie.

Ondanks uitgebreide analyse blijft het onduidelijk welke specifieke kenmerken de scenario's onderscheiden waarin variabelenvolgorde heuristieken beter presteren dan bovengrens heuristieken en vice versa. Dit onderscheid is voornamelijk waargenomen door empirische metingen in plaats van theoretische analyse. Wel vullen beide types elkaar aan, zodat in elk geval minstens één van de twee gemiddeld in een duidelijk betere compilatievolgorde resulteert dan een willekeurige compilatie.

Voor de variabelenvolgorde heuristieken merken we dat heuristiek 6.2 waarbij de focus op de primes wordt gelegd, het beste werkt. Heuristiek 5, die de strikste bovengrens implementeert, geeft de beste resultaten voor de bovengrens heuristieken.

Tenslotte hebben we een voorstel gedaan voor een combinatie van de heuristieken 6.2 en 5 op basis van het aantal variabelen in de SDD's, met als doel het beste van de twee types heuristieken te combineren. Deze combinatie geeft al veelbelovende resultaten, maar we vermoeden dat er betere manieren zijn om de twee types heuristieken te combineren.

## 7.1 Toekomstvisie

De eerste steen is gelegd voor het probleem van een efficiënte Apply-volgorde, we hebben heuristieken geïdentificeerd die een goede compilatievolgorde kunnen bepalen. Toch zijn er nog veel vragen onbeantwoord en is er nog ruimte voor verbetering. In deze sectie bespreken we tekortkomingen van ons onderzoek die in toekomstig onderzoek kunnen worden aangepakt.

Voorlopig is de potentiële efficiëntie van heuristiek 5 empirisch aangetoond door de overhead te negeren. Een praktisch intelligente implementatie die deze overhead minimaliseert, is nodig om de toepasbaarheid van deze heuristiek praktisch bruikbaar te maken. Voornamelijk het meten van het aantal elementen per vtreeknoop is een belangrijke factor in de overhead, dat kan opgelost worden door de aantallen tijdens de uitvoering van het programma op te slaan.

Het eerste experiment beschreven in figuur 6.1 hebben we niet uitgevoerd voor willekeurige vtrees. De reden hiervoor is dat een willekeurige vtree linkslinaair kan zijn. Voor linkslinaire vtrees komen we in de praktijk al snel geheugen tekort, we zouden dus enkel met een laag aantal variabelen kunnen testen. Verder liggen de compilatietijden verder uiteen tussen verschillende probleeminstanties, omdat de grootte van een SDD afhangt van de vtree. Het zou interessant zijn om onze heuristieken te toetsen op probleeminstanties waarvoor een vtree statisch op voorhand is bepaald. Het voordeel hiervan is dat de vtree een willekeurige structuur kan hebben en we dus een beter zicht krijgen op het gebruik van de vtree per heuristiek. Een goede vtree zal in theorie niet leiden tot geheugentekortfouten zoals gebeurt bij linkslinaire vtrees, waardoor we ons niet moeten beperken tot  $\nu = 10$  variabelen.

Verder kan er een beter onderscheid bepaald worden tussen de drie variabelenvolgorde heuristieken als we de heuristieken testen op SDD's die niet alle variabelen bevatten, hiervoor kunnen gespecialiseerde experimenten opgezet worden.

Tenslotte is meer theoretisch onderzoek nodig om de onderliggende factoren te begrijpen die de effectiviteit van variabelenvolgorde heuristieken versus bovengrens heuristieken bepalen. We vermoeden dat er een link is met de "moeilijkheid" van SDD's in termen van een fasetransitie. Zogenaamde "makkelijke" SDD's kunnen dan beter Applied worden volgens een variabelenvolgorde heuristiek. Toekomstig theoretisch werk kan dan ook leiden tot een betere combinatie van de twee types heuristieken.

# Bibliografie

- [AMS03] Fadi A. Aloul, Igor L. Markov, and Kareem A. Sakallah. FORCE: a fast and easy-to-implement variable-ordering heuristic. In *ACM Great Lakes Symposium on VLSI*, pages 116–119. ACM, 2003.
- [AMS04] Fadi A. Aloul, Igor L. Markov, and Kareem A. Sakallah. Mince: A static global variable-ordering heuristic for sat search and bdd manipulation. *J. Univers. Comput. Sci.*, 10:1562–1596, 2004.
- [Bov16a] Simone Bova. Sdds are exponentially more succinct than obdds. *CoRR*, abs/1601.00501, 2016.
- [Bov16b] Simone Bova. Sdds are exponentially more succinct than obdds. In *AAAI*, pages 929–935. AAAI Press, 2016.
- [Bry86] Randal E. Bryant. Graph-based algorithms for boolean function manipulation. *IEEE Trans. Computers*, 35(8):677–691, 1986.
- [BW96] Beate Bollig and Ingo Wegener. Improving the variable ordering of obdds is np-complete. *IEEE Trans. Computers*, 45(9):993–1002, 1996.
- [CA96a] James M. Crawford and Larry D. Auton. Experimental results on the crossover point in random 3-sat. *Artif. Intell.*, 81(1-2):31–57, 1996.
- [CA96b] James M. Crawford and Larry D. Auton. Experimental results on the crossover point in random 3-satf. *Artificial Intelligence*, 81(1):31–57, 1996. *Frontiers in Problem Solving: Phase Transitions and Complexity*.
- [CD08] Mark Chavira and Adnan Darwiche. On probabilistic inference by weighted model counting. *Artif. Intell.*, 172(6-7):772–799, 2008.
- [CD13a] Arthur Choi and Adnan Darwiche. Dynamic minimization of sentential decision diagrams. In *Proceedings of the Twenty-Seventh AAAI Conference on Artificial Intelligence*, AAAI’13, page 187–194. AAAI Press, 2013.
- [CD13b] Arthur Choi and Adnan Darwiche. Dynamic minimization of sentential decision diagrams. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2013.

- 
- [CKD13] Arthur Choi, Doga Gizem Kisa, and Adnan Darwiche. Compiling probabilistic graphical models using sentential decision diagrams. In *European Conference on Symbolic and Quantitative Approaches to Reasoning and Uncertainty*, 2013.
- [CKT91] Peter C. Cheeseman, Bob Kanefsky, and William M. Taylor. Where the really hard problems are. In *IJCAI*, pages 331–340. Morgan Kaufmann, 1991.
- [Dar00] Adnan Darwiche. On the tractable counting of theory models and its application to belief revision and truth maintenance. *CoRR*, cs.AI/0003044, 2000.
- [Dar11] Adnan Darwiche. Sdd: A new canonical representation of propositional knowledge bases. In *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence (IJCAI)*, pages 819–826, 2011.
- [dBD15] Guy Van den Broeck and Adnan Darwiche. On the role of canonicity in knowledge compilation. In *AAAI*, pages 1641–1648. AAAI Press, 2015.
- [dCM21] Alexis de Colnet and Stefan Mengel. Lower bounds on intermediate results in bottom-up knowledge compilation. *ArXiv*, abs/2112.12430, 2021.
- [Der23] Vincent Derkinderen. The role of counting in probabilistic reasoning, 2023.
- [DKL<sup>+</sup>21] Meihua Dang, Pasha Khosravi, Yitao Liang, Antonio Vergari, and Guy Van den Broeck. Juice: A julia package for logic and probabilistic circuits. In *AAAI Conference on Artificial Intelligence*, 2021.
- [DM11] Adnan Darwiche and Pierre Marquis. A knowledge compilation map. *CoRR*, abs/1106.1819, 2011.
- [GRM20] Rahul Gupta, Subhajit Roy, and Kuldeep S. Meel. Phase transition behavior in knowledge compilation. In *CP*, volume 12333 of *Lecture Notes in Computer Science*, pages 358–374. Springer, 2020.
- [GWY15] Jian Gao, Jianan Wang, and Minghao Yin. Experimental analyses on phase transitions in compiling satisfiability problems. *Sci. China Inf. Sci.*, 58(3):1–11, 2015.
- [HD04] Jinbo Huang and Adnan Darwiche. Using DPLL for efficient OBDD construction. In *SAT (Selected Papers)*, volume 3542 of *Lecture Notes in Computer Science*, pages 157–172. Springer, 2004.
- [HKZDM19] Evert Heylen, Samuel Kolb, and Pedro Miguel Zuidberg Dos Martires. How to maximize exploitable structure in wmi, 2019.

- 
- [LBD<sup>+</sup>17] Anna L. D. Latour, Behrouz Babaki, Anton Dries, Angelika Kimmig, Guy Van den Broeck, and Siegfried Nijssen. Combining stochastic constraint optimization and probabilistic programming - from knowledge compilation to constraint solving. In *CP*, volume 10416 of *Lecture Notes in Computer Science*, pages 495–511. Springer, 2017.
- [LBdB17] Yitao Liang, Jessa Bekker, and Guy Van den Broeck. Learning the structure of probabilistic sentential decision diagrams. In *UAI*. AUAI Press, 2017.
- [Mee23] Wannes Meert. Pysdd. <https://github.com/wannesm/PySDD>, dec 2023. Accessed: 2024-05-21.
- [MSL92] David G. Mitchell, Bart Selman, and Hector J. Levesque. Hard and easy distributions of SAT problems. In *AAAI*, pages 459–465. AAAI Press / The MIT Press, 1992.
- [NW07] Nina Narodytska and Toby Walsh. Constraint and variable ordering heuristics for compiling configuration problems. In *IJCAI*, pages 149–154, 2007.
- [OD18] Umut Oztok and Adnan Darwiche. An exhaustive DPLL algorithm for model counting. *J. Artif. Intell. Res.*, 62:1–32, 2018.
- [PD08] Knot Pipatsrisawat and Adnan Darwiche. New compilation languages based on structured decomposability. In *AAAI*, pages 517–522. AAAI Press, 2008.
- [PD10] Thammanit Pipatsrisawat and Adnan Darwiche. A lower bound on the size of decomposable negation normal form. In *AAAI*, pages 345–350. AAAI Press, 2010.
- [RN20] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson, 2020.
- [Rud93] Richard Rudell. Dynamic variable ordering for ordered binary decision diagrams. In *ICCAD*, pages 42–47. IEEE Computer Society / ACM, 1993.
- [VL20] Lieuwe Vinkhuijzen and Alfons Laarman. Symbolic model checking with sentential decision diagrams. In *SETTA*, volume 12153 of *Lecture Notes in Computer Science*, pages 124–142. Springer, 2020.
- [VRdBR14] Jonas Vlasselaer, Joris Renkens, Guy Van den Broeck, and Luc De Raedt. Compiling probabilistic logic programs into sentential decision diagrams. 2014.
- [XCD12] Yexiang Xue, Arthur Choi, and Adnan Darwiche. Basing decisions on sentences in decision diagrams. In *AAAI*, pages 842–849. AAAI Press, 2012.

## Bijlage A

### Engelse termen

In deze bijlage vermelden we woorden die afkomstig zijn van een Engelstalig woord. De bronnen waarop dit werk steunt zijn voornamelijk Engelstalig, met deze lijst maken we het makkelijker om meer informatie te winnen over bepaalde termen. De lijst is alfabetisch gesorteerd.

---

|                                                 |                                             |
|-------------------------------------------------|---------------------------------------------|
| Afbeelding                                      | Mapping                                     |
| Bladknoop                                       | Leafnode                                    |
| Bondigheid                                      | Succintness                                 |
| Behandelbaarheid                                | Tractability                                |
| Canoniciteit                                    | Canonicity                                  |
| Conditionering                                  | Conditioning                                |
| Consistentie                                    | Consistency                                 |
| (Begrensde) conjunctie                          | (Bounded) conjunction                       |
| Clausale implicatie                             | Clausal entailment                          |
| Clausule                                        | Clause                                      |
| Decompositie                                    | Decomposition                               |
| (Deel)verzameling                               | (Sub)set                                    |
| (Sterk) determinisme                            | (Strong) determinism                        |
| (Begrensde) disjunctie                          | (Bounded) disjunction                       |
| Equivalentie                                    | Equivalence                                 |
| Fasettransitie                                  | Phase-transition                            |
| Gecomprimeerd                                   | Compressed                                  |
| Geldigheid                                      | Validity                                    |
| Genormaliseerd                                  | Normalised                                  |
| Gestructureerde opsplitsbaarheid                | (Structured) decomposability                |
| Gerichte Acyclische Grafe                       | Directed Acyclic Graph                      |
| Gulzig                                          | Greedy                                      |
| Implicatie                                      | Implicant                                   |
| Kennisbasis                                     | Knowledge basis                             |
| Modeltellen                                     | Model counting                              |
| Modelenumeratie                                 | Model enumeration                           |
| Negatie                                         | Negation                                    |
| Onder-/overbepaald                              | Under-/overconstrained                      |
| Padbreedte                                      | Pathwidth                                   |
| Probabilistische inferentie                     | Probabilistic reasoning                     |
| Stochastische beperking-optimalisatie-problemen | Stochastic constraint optimization problems |
| (Super)verzameling                              | (Super)set                                  |
| Symbolische modelverificatie                    | Symbolic model checking                     |
| Vergeten (van singletons)                       | (Singleton) forgetting                      |
| Zin implicatie                                  | Sentential entailment                       |

TABEL A.1: Vertalingen van Engels jargon.

## Bijlage B

### Resultaten sectie 6.7

In volgende tabellen geven we scores van alle experimenten gedaan met willekeurig aantal clausules, zie sectie 6.7. Tabel B.1 bevat de resultaten voor de gebalanceerde vtree. De resultaten voor rechtslineaire, linkslineaire en willekeurige vtrees zijn terug te vinden in respectievelijk tabel B.2, tabel B.3 en tabel B.4. Voor elke probleeminstantie hebben we de laagste score vetgedrukt aangeduid.

TABEL B.1: Resultaten voor gebalanceerde vtrees en  $\nu = 28$ .

| TD       | BU-LR     | BU-RL    | VP+KE    | VP+EL    | VP+EL 2   | BU+EL | BU+EL 2  |
|----------|-----------|----------|----------|----------|-----------|-------|----------|
| <b>3</b> | 8         | <b>3</b> | 41       | 21       | 18        | 22    | 20       |
| <b>0</b> | <b>0</b>  | <b>0</b> | 17       | 10       | 9         | 13    | 9        |
| <b>0</b> | 12        | 2        | 40       | 66       | 63        | 66    | 65       |
| 43       | <b>37</b> | 41       | 44       | 55       | 49        | 57    | 49       |
| <b>0</b> | 3         | <b>0</b> | 20       | 48       | 44        | 48    | 44       |
| 7        | 12        | <b>6</b> | 60       | 57       | 54        | 58    | 57       |
| 34       | 40        | 33       | 43       | 33       | 30        | 9     | <b>8</b> |
| 57       | 24        | 57       | <b>0</b> | 11       | 11        | 11    | 11       |
| 4        | 30        | <b>1</b> | 28       | 32       | 28        | 35    | 28       |
| 4        | 52        | <b>3</b> | 59       | 34       | 28        | 42    | 33       |
| <b>3</b> | 66        | <b>3</b> | 10       | 55       | 51        | 56    | 55       |
| 32       | 33        | 25       | 51       | 22       | <b>17</b> | 27    | 22       |
| <b>0</b> | 14        | <b>0</b> | 63       | 47       | 44        | 52    | 46       |
| <b>0</b> | <b>0</b>  | <b>0</b> | 50       | 8        | 8         | 10    | 8        |
| 31       | <b>1</b>  | 30       | 54       | 35       | 29        | 56    | 51       |
| <b>0</b> | <b>0</b>  | <b>0</b> | 63       | 53       | 49        | 65    | 65       |
| 34       | <b>5</b>  | 33       | 12       | 27       | 17        | 22    | 14       |
| 62       | <b>37</b> | 58       | 55       | 56       | 55        | 58    | 57       |
| <b>1</b> | 4         | <b>1</b> | 22       | 34       | 29        | 36    | 33       |
| <b>0</b> | 16        | <b>0</b> | 55       | <b>0</b> | <b>0</b>  | 4     | 1        |
| <b>0</b> | 1         | <b>0</b> | 27       | 24       | 23        | 27    | 24       |
| <b>0</b> | 37        | <b>0</b> | 42       | 44       | 39        | 40    | 35       |
| <b>0</b> | 17        | <b>0</b> | 42       | 51       | 45        | 51    | 47       |

TABEL B.1: Resultaten voor gebalanceerde vtrees en  $\nu = 28$ .

| TD       | BU-LR     | BU-RL     | VP+KE     | VP+EL    | VP+EL 2   | BU+EL    | BU+EL 2   |
|----------|-----------|-----------|-----------|----------|-----------|----------|-----------|
| 4        | 22        | 4         | 23        | 59       | 57        | 60       | 59        |
| 0        | 0         | 0         | 41        | 27       | 24        | 36       | 30        |
| 0        | 0         | 0         | 2         | 4        | 2         | 5        | 4         |
| 50       | 45        | 52        | <b>42</b> | 43       | <b>42</b> | 43       | 43        |
| 19       | 65        | <b>17</b> | 31        | 34       | 30        | 51       | 47        |
| 45       | 0         | 44        | 5         | 21       | 20        | 22       | 22        |
| <b>3</b> | <b>3</b>  | <b>3</b>  | 46        | 61       | 60        | 36       | 35        |
| 6        | 35        | <b>5</b>  | 58        | 11       | 10        | 12       | 9         |
| 0        | 0         | 0         | 15        | 28       | 26        | 30       | 28        |
| 0        | 0         | 0         | 19        | 66       | 66        | 67       | 66        |
| 6        | 0         | 6         | 55        | 40       | 39        | 41       | 39        |
| 5        | 17        | <b>2</b>  | 40        | 35       | 34        | 38       | 35        |
| 15       | 28        | <b>14</b> | 42        | 65       | 63        | 65       | 65        |
| 0        | 0         | 0         | 9         | 4        | 3         | 12       | 10        |
| 67       | 49        | 67        | <b>37</b> | 49       | 47        | 50       | 47        |
| 0        | 0         | 0         | 29        | 29       | 25        | 33       | 31        |
| 53       | 54        | 53        | 13        | <b>0</b> | <b>0</b>  | <b>0</b> | <b>0</b>  |
| 3        | <b>2</b>  | 3         | <b>2</b>  | 10       | 3         | 11       | 5         |
| 67       | 2         | 67        | <b>0</b>  | 11       | 9         | 13       | 9         |
| 1        | 9         | <b>0</b>  | 51        | 27       | 25        | 25       | 23        |
| 49       | 60        | 47        | <b>16</b> | 29       | 25        | 64       | 61        |
| <b>1</b> | 2         | <b>1</b>  | 45        | 9        | 8         | 9        | 6         |
| 3        | <b>2</b>  | <b>2</b>  | 4         | 12       | 10        | 14       | 11        |
| <b>4</b> | 10        | <b>4</b>  | 57        | 49       | 46        | 48       | 46        |
| 2        | <b>0</b>  | 2         | 51        | 2        | 2         | 4        | 3         |
| 29       | 26        | 25        | 61        | 21       | 20        | 18       | <b>15</b> |
| 0        | 0         | 0         | 33        | 24       | 21        | 27       | 21        |
| 0        | 7         | 0         | 51        | 47       | 44        | 46       | 38        |
| <b>2</b> | 3         | <b>2</b>  | 22        | 37       | 33        | 57       | 55        |
| 1        | 11        | <b>0</b>  | 40        | 61       | 59        | 58       | 58        |
| 18       | 38        | <b>14</b> | 54        | 57       | 51        | 54       | 49        |
| 60       | <b>0</b>  | 52        | 2         | 2        | 2         | 3        | 2         |
| 41       | <b>25</b> | 40        | 53        | 35       | 31        | 36       | 31        |
| 0        | 10        | 0         | 45        | 41       | 35        | 43       | 42        |
| 0        | 0         | 0         | 59        | 59       | 57        | 41       | 34        |
| 0        | 4         | 0         | 55        | 56       | 54        | 54       | 48        |
| 0        | 0         | 0         | 13        | 12       | 8         | 1        | 1         |
| 11       | 0         | 16        | <b>0</b>  | 16       | 11        | 16       | 13        |
| 15       | 9         | 12        | 50        | 9        | 9         | 9        | <b>6</b>  |
| 34       | 56        | <b>27</b> | 46        | 58       | 56        | 56       | 52        |
| <b>1</b> | 29        | <b>1</b>  | 29        | 26       | 23        | 17       | 12        |
| 36       | 24        | 32        | 47        | 31       | 25        | 24       | <b>20</b> |

TABEL B.1: Resultaten voor gebalanceerde vtrees en  $\nu = 28$ .

| TD        | BU-LR     | BU-RL     | VP+KE    | VP+EL    | VP+EL 2   | BU+EL    | BU+EL 2  |
|-----------|-----------|-----------|----------|----------|-----------|----------|----------|
| <b>0</b>  | 1         | <b>0</b>  | 48       | 1        | 1         | 2        | 1        |
| <b>0</b>  | 1         | <b>0</b>  | 66       | 27       | 19        | 7        | 6        |
| 67        | 2         | 67        | <b>0</b> | <b>0</b> | <b>0</b>  | <b>0</b> | <b>0</b> |
| <b>0</b>  | 1         | <b>0</b>  | 64       | 66       | 65        | 10       | 9        |
| <b>1</b>  | <b>1</b>  | <b>1</b>  | 14       | 10       | 7         | 12       | 10       |
| 49        | <b>0</b>  | 49        | 5        | <b>0</b> | <b>0</b>  | <b>0</b> | <b>0</b> |
| <b>13</b> | 44        | <b>13</b> | 24       | 53       | 48        | 46       | 44       |
| 13        | <b>11</b> | 12        | 48       | 27       | 21        | 16       | 12       |
| 4         | 16        | <b>3</b>  | 36       | 38       | 31        | 10       | 7        |
| <b>0</b>  | <b>0</b>  | <b>0</b>  | 55       | 27       | 21        | 6        | 5        |
| 31        | 17        | 31        | 14       | 10       | 8         | <b>0</b> | <b>0</b> |
| 46        | 23        | 40        | 32       | 18       | <b>15</b> | 30       | 24       |
| 34        | 24        | 34        | 34       | 24       | 21        | 11       | <b>9</b> |
| <b>0</b>  | <b>0</b>  | <b>0</b>  | 13       | 20       | 19        | 18       | 16       |
| <b>0</b>  | <b>0</b>  | <b>0</b>  | <b>0</b> | 4        | 3         | 4        | 3        |
| <b>4</b>  | 40        | <b>4</b>  | 30       | 36       | 32        | 19       | 12       |
| 58        | <b>14</b> | 58        | 26       | 50       | 46        | 43       | 37       |
| <b>0</b>  | 2         | <b>0</b>  | 2        | 2        | 2         | 2        | 2        |
| 11        | 8         | <b>1</b>  | 36       | 20       | 17        | 3        | <b>1</b> |
| 65        | 17        | 65        | 36       | 8        | 7         | 7        | <b>5</b> |
| <b>3</b>  | 26        | <b>3</b>  | 43       | 53       | 45        | 41       | 40       |
| <b>0</b>  | <b>0</b>  | <b>0</b>  | 34       | 31       | 29        | 27       | 26       |
| <b>1</b>  | 14        | <b>1</b>  | 32       | 41       | 39        | 20       | 16       |
| 67        | <b>0</b>  | 66        | 1        | <b>0</b> | <b>0</b>  | <b>0</b> | <b>0</b> |
| <b>1</b>  | 3         | <b>1</b>  | 42       | 30       | 25        | 45       | 43       |
| <b>0</b>  | <b>0</b>  | <b>0</b>  | <b>0</b> | 1        | <b>0</b>  | <b>0</b> | <b>0</b> |
| <b>2</b>  | 37        | <b>2</b>  | 37       | 51       | 46        | 33       | 23       |
| <b>0</b>  | <b>0</b>  | <b>0</b>  | <b>0</b> | 8        | 7         | 8        | 7        |
| <b>0</b>  | 4         | <b>0</b>  | 36       | 44       | 43        | 31       | 22       |
| <b>0</b>  | 31        | <b>0</b>  | 51       | 38       | 32        | 20       | 16       |
| <b>0</b>  | 33        | <b>0</b>  | 13       | 51       | 41        | 14       | 9        |
| <b>1</b>  | 26        | <b>1</b>  | 25       | 27       | 25        | 12       | 8        |
| <b>6</b>  | 33        | <b>6</b>  | 36       | 49       | 42        | 46       | 35       |
| 1         | 1         | <b>0</b>  | 59       | 58       | 56        | 14       | 11       |
| <b>0</b>  | <b>0</b>  | <b>0</b>  | 47       | 34       | 30        | 30       | 27       |

TABEL B.2: Resultaten voor rechtslineaire vtrees en  $\nu = 28$ .

| TD | BU-LR | BU-RL | VP+KE | VP+EL | VP+EL 2   | BU+EL | BU+EL 2 |
|----|-------|-------|-------|-------|-----------|-------|---------|
| 44 | 44    | 44    | 42    | 58    | <b>23</b> | 64    | 40      |
| 47 | 57    | 46    | 47    | 60    | <b>5</b>  | 65    | 30      |
| 67 | 67    | 67    | 67    | 67    | <b>16</b> | 67    | 56      |

---

TABEL B.2: Resultaten voor rechte lineaire vrees en  $\nu = 28$ .

| TD | BU-LR     | BU-RL     | VP+KE    | VP+EL | VP+EL 2   | BU+EL | BU+EL 2 |
|----|-----------|-----------|----------|-------|-----------|-------|---------|
| 67 | 67        | 67        | 67       | 67    | <b>23</b> | 67    | 43      |
| 54 | 62        | 59        | 38       | 67    | <b>8</b>  | 67    | 35      |
| 67 | 67        | 67        | 67       | 67    | <b>4</b>  | 67    | 44      |
| 67 | 67        | 67        | 59       | 67    | <b>10</b> | 67    | 32      |
| 67 | 47        | <b>6</b>  | 12       | 65    | <b>8</b>  | 66    | 17      |
| 67 | 67        | 67        | 58       | 67    | <b>9</b>  | 67    | 27      |
| 67 | 67        | 67        | 67       | 67    | <b>40</b> | 67    | 67      |
| 67 | 67        | 67        | 42       | 64    | <b>7</b>  | 65    | 43      |
| 67 | 67        | 67        | 65       | 67    | <b>13</b> | 67    | 40      |
| 67 | 67        | 67        | 52       | 67    | <b>8</b>  | 67    | 60      |
| 51 | 66        | 55        | 46       | 64    | <b>0</b>  | 66    | 18      |
| 67 | 67        | 67        | 13       | 66    | <b>3</b>  | 67    | 13      |
| 67 | 65        | 67        | 45       | 64    | <b>23</b> | 67    | 30      |
| 35 | 6         | 2         | 23       | 67    | <b>0</b>  | 67    | 8       |
| 67 | 67        | 67        | 55       | 67    | <b>3</b>  | 67    | 41      |
| 67 | 67        | 67        | 59       | 67    | <b>10</b> | 67    | 59      |
| 67 | 67        | 67        | 66       | 67    | <b>22</b> | 67    | 63      |
| 25 | 36        | <b>17</b> | 36       | 51    | 20        | 58    | 32      |
| 67 | 67        | 67        | 49       | 67    | <b>16</b> | 67    | 35      |
| 67 | 67        | 67        | 67       | 67    | <b>20</b> | 67    | 64      |
| 67 | 67        | 67        | 63       | 67    | <b>2</b>  | 67    | 43      |
| 67 | 54        | 63        | 54       | 54    | <b>2</b>  | 58    | 29      |
| 12 | 9         | <b>2</b>  | 9        | 63    | <b>2</b>  | 63    | 8       |
| 67 | 63        | <b>0</b>  | 30       | 58    | 29        | 59    | 27      |
| 67 | 43        | 67        | 32       | 64    | <b>12</b> | 67    | 27      |
| 67 | 29        | 16        | <b>5</b> | 48    | 27        | 52    | 33      |
| 67 | 67        | 67        | 42       | 67    | <b>40</b> | 67    | 59      |
| 67 | 67        | 67        | 53       | 67    | <b>45</b> | 67    | 53      |
| 2  | 15        | <b>1</b>  | 24       | 37    | 5         | 47    | 15      |
| 67 | 67        | 67        | 53       | 60    | <b>14</b> | 67    | 53      |
| 67 | 53        | 67        | 53       | 67    | <b>52</b> | 67    | 53      |
| 66 | 67        | 66        | 44       | 63    | <b>7</b>  | 67    | 43      |
| 67 | 67        | 67        | 67       | 67    | <b>22</b> | 67    | 67      |
| 62 | 26        | 67        | 22       | 59    | <b>5</b>  | 63    | 17      |
| 67 | <b>30</b> | 66        | 56       | 67    | 39        | 67    | 62      |
| 37 | 50        | 37        | 38       | 60    | <b>5</b>  | 63    | 24      |
| 67 | 67        | 20        | 3        | 28    | <b>0</b>  | 36    | 1       |
| 62 | 61        | 61        | 26       | 67    | <b>1</b>  | 67    | 31      |
| 67 | 41        | 40        | 19       | 58    | <b>8</b>  | 66    | 36      |
| 67 | 67        | 67        | 62       | 67    | <b>20</b> | 67    | 58      |
| 67 | 67        | 67        | 57       | 67    | <b>3</b>  | 67    | 34      |
| 67 | 67        | 67        | <b>7</b> | 67    | 20        | 67    | 31      |

TABEL B.2: Resultaten voor rechte lineaire vrees en  $\nu = 28$ .

| TD       | BU-LR    | BU-RL    | VP+KE     | VP+EL | VP+EL 2   | BU+EL | BU+EL 2  |
|----------|----------|----------|-----------|-------|-----------|-------|----------|
| 11       | 11       | 11       | 11        | 28    | <b>4</b>  | 43    | 7        |
| 67       | 67       | 67       | 67        | 67    | <b>64</b> | 67    | 67       |
| 61       | 29       | 66       | 5         | 38    | <b>2</b>  | 38    | <b>2</b> |
| 67       | 67       | 67       | 67        | 67    | <b>18</b> | 67    | 63       |
| 57       | 42       | 42       | 31        | 57    | <b>6</b>  | 65    | 31       |
| 67       | 66       | 67       | 49        | 66    | <b>6</b>  | 67    | 46       |
| <b>1</b> | 11       | <b>1</b> | 16        | 42    | <b>1</b>  | 42    | <b>1</b> |
| 67       | 67       | 67       | 59        | 67    | <b>30</b> | 67    | 54       |
| 67       | 67       | 67       | 67        | 67    | <b>5</b>  | 67    | 67       |
| 65       | 21       | 67       | <b>13</b> | 49    | <b>13</b> | 59    | 21       |
| 67       | 67       | 67       | 65        | 67    | <b>23</b> | 67    | 58       |
| 62       | 50       | 47       | 47        | 66    | <b>10</b> | 67    | 36       |
| 67       | 67       | 67       | 66        | 67    | <b>15</b> | 67    | 62       |
| 67       | 67       | 67       | 63        | 67    | <b>39</b> | 67    | 60       |
| 5        | 4        | 3        | 9         | 46    | <b>1</b>  | 47    | 5        |
| 48       | 25       | 7        | 24        | 48    | <b>4</b>  | 48    | 11       |
| 67       | 67       | 67       | 63        | 67    | <b>34</b> | 67    | 57       |
| 67       | 67       | 67       | 63        | 64    | <b>21</b> | 67    | 22       |
| 67       | 67       | 67       | 67        | 67    | <b>25</b> | 67    | 58       |
| 67       | 67       | 67       | 51        | 67    | <b>8</b>  | 67    | 51       |
| 20       | 30       | <b>4</b> | <b>4</b>  | 55    | 28        | 47    | <b>4</b> |
| 64       | 56       | 67       | 52        | 57    | <b>9</b>  | 66    | 28       |
| 67       | 30       | 35       | <b>0</b>  | 34    | <b>0</b>  | 38    | <b>0</b> |
| 66       | 67       | 64       | 49        | 65    | <b>0</b>  | 67    | 34       |
| 8        | 17       | <b>7</b> | 21        | 33    | 8         | 49    | 15       |
| 58       | <b>0</b> | <b>0</b> | 9         | 57    | <b>0</b>  | 58    | <b>0</b> |
| 67       | 67       | 67       | 67        | 67    | <b>43</b> | 67    | 62       |
| 67       | 67       | 67       | 66        | 67    | <b>1</b>  | 67    | 15       |
| 67       | 67       | 67       | 52        | 60    | 15        | 60    | <b>5</b> |
| 67       | 67       | 67       | 56        | 64    | <b>3</b>  | 65    | <b>3</b> |
| 67       | 67       | 67       | 67        | 67    | 8         | 67    | <b>0</b> |
| 67       | 67       | 67       | 66        | 67    | <b>25</b> | 67    | 42       |
| 67       | 67       | 67       | 67        | 67    | 45        | 67    | <b>0</b> |
| 18       | 10       | 25       | 7         | 37    | 3         | 37    | <b>0</b> |
| 1        | <b>0</b> | 1        | 1         | 49    | <b>0</b>  | 50    | <b>0</b> |
| 67       | 67       | 67       | 67        | 67    | 19        | 67    | <b>0</b> |
| 67       | 6        | 51       | 38        | 67    | <b>3</b>  | 67    | 6        |
| 7        | 14       | 7        | 10        | 39    | 8         | 37    | <b>0</b> |
| 67       | 67       | 67       | 52        | 64    | 13        | 64    | <b>0</b> |
| 67       | 67       | 55       | 55        | 67    | <b>0</b>  | 66    | <b>0</b> |
| 67       | 57       | 67       | 4         | 60    | <b>0</b>  | 60    | <b>0</b> |
| 4        | 9        | 4        | 14        | 31    | <b>0</b>  | 45    | 4        |

TABEL B.2: Resultaten voor rechte lineaire vrees en  $\nu = 28$ .

| TD       | BU-LR    | BU-RL    | VP+KE | VP+EL | VP+EL 2   | BU+EL | BU+EL 2   |
|----------|----------|----------|-------|-------|-----------|-------|-----------|
| 67       | 67       | 67       | 64    | 67    | <b>31</b> | 67    | 32        |
| 67       | <b>0</b> | 67       | 24    | 64    | <b>0</b>  | 62    | <b>0</b>  |
| 67       | 66       | 67       | 52    | 64    | 10        | 65    | <b>5</b>  |
| 18       | 15       | 15       | 22    | 67    | <b>0</b>  | 67    | <b>0</b>  |
| 67       | 67       | 67       | 66    | 67    | 2         | 67    | <b>0</b>  |
| <b>1</b> | 2        | <b>1</b> | 7     | 56    | 2         | 60    | 6         |
| 22       | 42       | 24       | 20    | 55    | <b>19</b> | 60    | <b>19</b> |
| 67       | 67       | 62       | 47    | 65    | <b>4</b>  | 67    | 6         |
| 22       | 42       | 21       | 17    | 67    | <b>3</b>  | 67    | 10        |
| 41       | 41       | 41       | 32    | 59    | <b>4</b>  | 62    | 6         |
| 67       | 67       | 67       | 67    | 67    | <b>6</b>  | 67    | 12        |
| 66       | 20       | 5        | 5     | 49    | 5         | 49    | <b>0</b>  |
| 37       | 37       | 9        | 37    | 37    | <b>0</b>  | 38    | <b>0</b>  |

TABEL B.3: Resultaten voor links lineaire vrees en  $\nu = 10$ .

| TD       | BU-LR    | BU-RL    | VP+KE     | VP+EL    | VP+EL 2  | BU+EL    | BU+EL 2  |
|----------|----------|----------|-----------|----------|----------|----------|----------|
| <b>0</b> | 16       | 18       | <b>0</b>  | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| <b>0</b> | 57       | 57       | 1         | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| 7        | 66       | 66       | <b>0</b>  | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| <b>0</b> | <b>0</b> | <b>0</b> | 1         | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| 20       | 60       | 60       | <b>14</b> | 40       | 37       | 43       | 43       |
| <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b>  | 28       | 25       | 30       | 28       |
| 33       | 48       | 48       | <b>0</b>  | 14       | 12       | 15       | 15       |
| <b>6</b> | 40       | 40       | 40        | 18       | 14       | 29       | 25       |
| 25       | 38       | 38       | <b>0</b>  | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| <b>0</b> | 63       | 63       | <b>0</b>  | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| 1        | 5        | 5        | <b>0</b>  | 21       | 21       | 25       | 23       |
| <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b>  | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b>  | 30       | 30       | 33       | 33       |
| 1        | 66       | 66       | <b>0</b>  | 11       | 10       | 13       | 11       |
| <b>0</b> | 60       | 60       | <b>0</b>  | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| <b>0</b> | 67       | 67       | 2         | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| <b>0</b> | <b>0</b> | <b>0</b> | 24        | 63       | 63       | 63       | 63       |
| <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b>  | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| <b>0</b> | <b>0</b> | <b>0</b> | 24        | 39       | 38       | 40       | 39       |
| <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b>  | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| <b>0</b> | 67       | 67       | <b>0</b>  | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b>  | 23       | 23       | 35       | 30       |
| <b>0</b> | 67       | 67       | <b>0</b>  | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| <b>0</b> | 1        | 1        | <b>0</b>  | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |
| <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b>  | <b>0</b> | <b>0</b> | <b>0</b> | <b>0</b> |

TABEL B.3: Resultaten voor linkslineaire vtrees en  $\nu = 10$ .

| TD | BU-LR | BU-RL | VP+KE | VP+EL | VP+EL 2 | BU+EL | BU+EL 2 |
|----|-------|-------|-------|-------|---------|-------|---------|
| 18 | 67    | 67    | 1     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 2  | 67    | 67    | 2     | 0     | 0       | 0     | 0       |
| 13 | 26    | 26    | 0     | 0     | 0       | 0     | 0       |
| 0  | 56    | 56    | 0     | 0     | 0       | 0     | 0       |
| 2  | 20    | 20    | 63    | 62    | 58      | 65    | 64      |
| 0  | 0     | 0     | 0     | 13    | 13      | 14    | 14      |
| 0  | 5     | 5     | 17    | 40    | 40      | 40    | 39      |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 10 | 34    | 34    | 66    | 47    | 47      | 47    | 47      |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 0  | 66    | 66    | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 16 | 38    | 38    | 0     | 45    | 44      | 44    | 43      |
| 0  | 0     | 0     | 5     | 0     | 0       | 0     | 0       |
| 0  | 67    | 67    | 1     | 0     | 0       | 0     | 0       |
| 0  | 50    | 50    | 0     | 0     | 0       | 0     | 0       |
| 39 | 66    | 65    | 22    | 36    | 31      | 27    | 24      |
| 4  | 4     | 4     | 37    | 63    | 61      | 43    | 41      |
| 0  | 66    | 66    | 0     | 0     | 0       | 0     | 0       |
| 1  | 9     | 9     | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 26    | 46    | 43      | 34    | 33      |
| 0  | 67    | 67    | 28    | 47    | 45      | 44    | 41      |
| 0  | 65    | 65    | 0     | 52    | 49      | 46    | 43      |
| 24 | 59    | 58    | 13    | 31    | 31      | 29    | 29      |
| 0  | 0     | 0     | 43    | 40    | 35      | 23    | 19      |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 1     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 15    | 8     | 6       | 4     | 4       |
| 30 | 45    | 45    | 2     | 26    | 23      | 27    | 25      |
| 0  | 6     | 6     | 3     | 0     | 0       | 0     | 0       |
| 47 | 67    | 67    | 2     | 0     | 0       | 0     | 0       |
| 0  | 65    | 65    | 1     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 0  | 65    | 65    | 0     | 0     | 0       | 0     | 0       |
| 0  | 57    | 57    | 0     | 0     | 0       | 0     | 0       |
| 0  | 66    | 66    | 1     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 11    | 22    | 20      | 14    | 14      |
| 0  | 45    | 44    | 20    | 17    | 17      | 20    | 19      |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 52 | 38    | 37    | 1     | 62    | 62      | 60    | 60      |

TABEL B.3: Resultaten voor linkslineaire vtrees en  $\nu = 10$ .

| TD | BU-LR | BU-RL | VP+KE | VP+EL | VP+EL 2 | BU+EL | BU+EL 2 |
|----|-------|-------|-------|-------|---------|-------|---------|
| 0  | 1     | 1     | 1     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 2     | 0     | 0       | 0     | 0       |
| 24 | 67    | 67    | 32    | 4     | 1       | 3     | 1       |
| 56 | 67    | 67    | 0     | 0     | 0       | 0     | 0       |
| 0  | 41    | 44    | 30    | 27    | 27      | 22    | 20      |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 8  | 16    | 16    | 7     | 15    | 14      | 22    | 16      |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 7     | 48    | 46      | 48    | 47      |
| 0  | 0     | 0     | 2     | 43    | 43      | 21    | 18      |
| 2  | 66    | 66    | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 31    | 49    | 45      | 5     | 3       |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 30    | 65    | 65      | 45    | 43      |
| 64 | 62    | 61    | 47    | 48    | 48      | 40    | 38      |
| 8  | 32    | 32    | 2     | 46    | 44      | 27    | 27      |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 1     | 0     | 0       | 0     | 0       |
| 20 | 57    | 57    | 0     | 34    | 32      | 32    | 31      |
| 1  | 48    | 48    | 50    | 65    | 65      | 43    | 36      |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 10 | 24    | 24    | 1     | 0     | 0       | 1     | 1       |
| 20 | 60    | 61    | 0     | 13    | 11      | 8     | 6       |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 14 | 9     | 10    | 57    | 43    | 38      | 33    | 27      |
| 0  | 65    | 65    | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 37    | 14    | 12      | 10    | 9       |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |

TABEL B.4: Resultaten voor willekeurige vtrees en  $\nu = 16$ .

| TD | BU-LR | BU-RL | VP+KE | VP+EL | VP+EL 2 | BU+EL | BU+EL 2 |
|----|-------|-------|-------|-------|---------|-------|---------|
| 67 | 67    | 67    | 2     | 26    | 0       | 28    | 7       |
| 67 | 67    | 67    | 67    | 67    | 67      | 67    | 67      |
| 14 | 18    | 17    | 43    | 67    | 67      | 67    | 21      |
| 67 | 67    | 67    | 21    | 18    | 0       | 39    | 2       |
| 35 | 26    | 26    | 52    | 67    | 67      | 67    | 64      |

TABEL B.4: Resultaten voor willekeurige vtrees en  $\nu = 16$ .

| TD        | BU-LR     | BU-RL     | VP+KE     | VP+EL | VP+EL 2   | BU+EL    | BU+EL 2   |
|-----------|-----------|-----------|-----------|-------|-----------|----------|-----------|
| 9         | 9         | 9         | 7         | 41    | <b>3</b>  | 47       | 6         |
| <b>4</b>  | 67        | 67        | 9         | 67    | 54        | 67       | 42        |
| 67        | 67        | 67        | <b>0</b>  | 67    | 67        | 67       | 8         |
| 65        | 65        | 65        | <b>0</b>  | 67    | 3         | 67       | <b>0</b>  |
| 67        | 67        | 67        | <b>0</b>  | 67    | 16        | 67       | 1         |
| 35        | 32        | 32        | <b>22</b> | 67    | 67        | 67       | 67        |
| 67        | 67        | 67        | <b>0</b>  | 67    | <b>0</b>  | 67       | <b>0</b>  |
| 41        | 61        | 61        | 65        | 67    | <b>3</b>  | 67       | 63        |
| 65        | <b>31</b> | <b>31</b> | 67        | 67    | 67        | 67       | 67        |
| 67        | 67        | 67        | 43        | 67    | 67        | 67       | <b>40</b> |
| <b>0</b>  | <b>0</b>  | <b>0</b>  | <b>0</b>  | 58    | <b>0</b>  | 59       | <b>0</b>  |
| <b>9</b>  | 10        | 10        | 43        | 67    | 61        | 67       | 48        |
| 52        | 37        | 37        | <b>35</b> | 67    | 62        | 67       | 53        |
| 11        | 7         | <b>3</b>  | 57        | 67    | 67        | 67       | 51        |
| 61        | 61        | 59        | <b>0</b>  | 67    | <b>0</b>  | 65       | <b>0</b>  |
| 66        | 67        | 67        | <b>58</b> | 67    | 67        | 67       | 67        |
| 63        | 63        | 64        | <b>10</b> | 61    | 53        | 36       | 22        |
| 67        | 48        | 48        | <b>13</b> | 67    | 67        | 67       | 31        |
| 27        | <b>19</b> | 22        | 28        | 61    | 45        | 65       | 54        |
| 67        | 67        | 67        | 30        | 67    | 6         | 67       | <b>1</b>  |
| 65        | 65        | 65        | 15        | 18    | 15        | 11       | <b>8</b>  |
| <b>0</b>  | <b>0</b>  | <b>0</b>  | <b>0</b>  | 30    | <b>0</b>  | 25       | <b>0</b>  |
| <b>0</b>  | 64        | 64        | 1         | 61    | <b>0</b>  | 61       | <b>0</b>  |
| 2         | 7         | 7         | <b>0</b>  | 15    | <b>0</b>  | <b>0</b> | <b>0</b>  |
| 1         | 66        | 66        | 1         | 1     | <b>0</b>  | 1        | <b>0</b>  |
| <b>0</b>  | <b>0</b>  | <b>0</b>  | 12        | 60    | 26        | 60       | 26        |
| 67        | 67        | 67        | 27        | 67    | 46        | 67       | <b>0</b>  |
| 6         | 6         | 7         | <b>0</b>  | 7     | 1         | 7        | 2         |
| 67        | 67        | 67        | 64        | 67    | <b>55</b> | 67       | 67        |
| <b>23</b> | 24        | <b>23</b> | <b>23</b> | 57    | 40        | 59       | 50        |
| 1         | 1         | 1         | 2         | 67    | <b>0</b>  | 67       | <b>0</b>  |
| 67        | 67        | 67        | <b>8</b>  | 67    | 67        | 67       | 26        |
| 63        | <b>0</b>  | <b>0</b>  | 47        | 66    | 36        | 67       | 60        |
| 2         | 1         | 1         | <b>0</b>  | 21    | 8         | 8        | 2         |
| 66        | 67        | 67        | 63        | 67    | 66        | 67       | <b>41</b> |
| <b>0</b>  | 64        | 64        | 1         | 67    | <b>0</b>  | 67       | <b>0</b>  |
| <b>0</b>  | <b>0</b>  | <b>0</b>  | 5         | 62    | 18        | 62       | 26        |
| 67        | <b>6</b>  | <b>6</b>  | 67        | 67    | 67        | 67       | 9         |
| <b>0</b>  | <b>0</b>  | <b>0</b>  | 37        | 67    | 25        | 67       | 37        |
| 60        | 54        | 54        | <b>0</b>  | 67    | 9         | 67       | <b>0</b>  |
| 67        | 67        | 67        | <b>6</b>  | 67    | 59        | 67       | 17        |
| <b>0</b>  | <b>0</b>  | <b>0</b>  | 7         | 64    | 7         | 57       | <b>0</b>  |

TABEL B.4: Resultaten voor willekeurige vtrees en  $\nu = 16$ .

| TD | BU-LR | BU-RL | VP+KE | VP+EL | VP+EL 2 | BU+EL | BU+EL 2 |
|----|-------|-------|-------|-------|---------|-------|---------|
| 1  | 1     | 1     | 0     | 67    | 0       | 67    | 0       |
| 0  | 0     | 0     | 0     | 67    | 1       | 67    | 0       |
| 0  | 3     | 1     | 19    | 67    | 3       | 67    | 0       |
| 67 | 67    | 67    | 57    | 67    | 67      | 67    | 23      |
| 62 | 43    | 0     | 27    | 67    | 21      | 67    | 2       |
| 67 | 67    | 67    | 37    | 67    | 65      | 67    | 8       |
| 29 | 3     | 19    | 64    | 67    | 62      | 67    | 47      |
| 0  | 0     | 0     | 0     | 67    | 0       | 67    | 0       |
| 12 | 67    | 67    | 15    | 67    | 67      | 67    | 12      |
| 0  | 0     | 0     | 0     | 67    | 0       | 67    | 2       |
| 0  | 0     | 0     | 62    | 67    | 28      | 67    | 30      |
| 0  | 0     | 0     | 0     | 0     | 0       | 0     | 0       |
| 26 | 0     | 2     | 45    | 67    | 48      | 67    | 14      |
| 0  | 65    | 66    | 34    | 3     | 3       | 0     | 0       |
| 67 | 67    | 67    | 58    | 67    | 67      | 67    | 54      |
| 46 | 2     | 2     | 4     | 67    | 0       | 67    | 0       |
| 0  | 0     | 0     | 3     | 67    | 35      | 67    | 13      |
| 67 | 67    | 67    | 50    | 67    | 67      | 67    | 55      |
| 1  | 67    | 66    | 1     | 67    | 1       | 67    | 0       |
| 18 | 16    | 11    | 0     | 12    | 3       | 0     | 0       |
| 51 | 52    | 50    | 6     | 1     | 0       | 0     | 0       |
| 67 | 67    | 37    | 0     | 67    | 19      | 67    | 0       |
| 36 | 45    | 45    | 1     | 64    | 27      | 67    | 62      |
| 65 | 65    | 65    | 65    | 67    | 9       | 67    | 0       |
| 33 | 32    | 31    | 0     | 67    | 0       | 67    | 0       |
| 10 | 65    | 0     | 0     | 21    | 0       | 11    | 0       |
| 0  | 0     | 0     | 0     | 67    | 0       | 67    | 0       |
| 54 | 49    | 52    | 34    | 41    | 32      | 18    | 9       |
| 63 | 63    | 63    | 40    | 67    | 62      | 67    | 9       |
| 58 | 60    | 58    | 0     | 9     | 3       | 4     | 2       |
| 65 | 63    | 63    | 63    | 67    | 66      | 67    | 53      |
| 64 | 0     | 0     | 2     | 41    | 0       | 37    | 0       |
| 0  | 0     | 0     | 1     | 67    | 0       | 67    | 0       |
| 67 | 67    | 67    | 38    | 67    | 67      | 67    | 28      |
| 67 | 67    | 67    | 0     | 67    | 17      | 67    | 0       |
| 12 | 66    | 9     | 0     | 1     | 0       | 11    | 0       |
| 67 | 17    | 65    | 3     | 67    | 23      | 67    | 0       |
| 15 | 15    | 15    | 9     | 67    | 31      | 67    | 9       |
| 67 | 67    | 67    | 23    | 67    | 67      | 67    | 8       |
| 50 | 4     | 4     | 0     | 0     | 0       | 0     | 0       |
| 0  | 0     | 0     | 0     | 36    | 0       | 28    | 0       |
| 50 | 0     | 0     | 0     | 16    | 0       | 6     | 0       |

---

TABEL B.4: Resultaten voor willekeurige vtrees en  $\nu = 16$ .

| <b>TD</b> | <b>BU-LR</b> | <b>BU-RL</b> | <b>VP+KE</b> | <b>VP+EL</b> | <b>VP+EL 2</b> | <b>BU+EL</b> | <b>BU+EL 2</b> |
|-----------|--------------|--------------|--------------|--------------|----------------|--------------|----------------|
| 2         | 61           | 62           | 4            | 4            | <b>0</b>       | 4            | <b>0</b>       |
| 65        | 65           | 65           | <b>57</b>    | 67           | 64             | 67           | 66             |
| 48        | 52           | 48           | <b>0</b>     | 54           | <b>0</b>       | 56           | <b>0</b>       |
| 66        | 66           | 66           | 29           | 66           | <b>23</b>      | 67           | 65             |
| 67        | 67           | 67           | 10           | 67           | 61             | 59           | <b>9</b>       |
| 4         | 4            | 4            | <b>2</b>     | 14           | 5              | 32           | 20             |
| <b>0</b>  | 67           | 67           | <b>0</b>     | <b>0</b>     | <b>0</b>       | <b>0</b>     | <b>0</b>       |
| <b>0</b>  | <b>0</b>     | <b>0</b>     | 6            | 31           | 13             | 22           | 11             |
| <b>0</b>  | <b>0</b>     | <b>0</b>     | 9            | 25           | 3              | <b>0</b>     | <b>0</b>       |
| 59        | 59           | 56           | 49           | 67           | 29             | 67           | <b>1</b>       |
| <b>11</b> | 67           | 67           | 50           | 67           | 67             | 67           | 58             |